

Genomdatenbank und evolutionäre Netzwerkanalyse

Stephan Epp

23. Januar 2026

Inhaltsverzeichnis

1	Einführung	4
1.1	Beitrag dieser Arbeit	4
1.2	Aufbau der Arbeit	4
2	Grundlagen	5
2.1	Graphen und Adjazenzmatrizen	5
2.2	Signaturen und Subgraph-Relation	6
2.3	Der Subgraph Algorithmus	6
3	Systemarchitektur von Gen-DB	7
3.1	Komponentenübersicht	7
3.2	Datenmodell	7
3.3	UML-Architekturmodelle von Gen-DB	7
4	Such-Pipeline: Formalisierung und Beweise	9
4.1	Problemformulierung	9
4.2	SQL-Vorauswahl	9
4.3	SQL-Select-Statement	11
4.4	Gesamtkomplexität der Pipeline	12
4.5	Korrektheit der Gesamtpipeline	12
4.6	Effiziente Suche	13
5	Empirische Evaluation	14
5.1	Versuchsaufbau	14
5.2	Generierungsstatistik	14
5.3	Visualisierung 1: Knoten- und Kantenverteilung	15
5.4	Visualisierung 2: Pre-Filter-Effektivität	15
5.5	Visualisierung 3: Skalierungsverhalten der Suchzeit	16
5.6	Visualisierung 4: Trefferverteilung nach Match-Typ	17
5.7	Diskussion	17

6	Experimenteller Vergleich VF2-Algorithmus	17
6.1	Versuchsordnung	18
6.2	Ergebnis 1: Laufzeitvergleich bei festem $n_A = 5$, wachsendem n_B	18
6.3	Der Subgraph Algorithmus für $n_B \geq 14$	20
6.4	Einordnung in effiziente Subgraph-Isomorphie-Verfahren	23
7	Experimentelle Untersuchung der Parallelisierung	23
7.1	Fragestellung und Hypothesen	25
7.2	Implementierung der parallelen Suche	25
7.3	Versuchsaufbau	26
7.4	Metriken	26
7.5	Ergebnisse	27
7.5.1	Die untersuchten Anfragen	27
7.5.2	Gesamtzeit und Speedup	27
7.5.3	Verteilung der Antwortzeiten	29
7.5.4	Skalierung mit der Größe des Suchraums	29
7.5.5	Dieselben Anfragen im direkten Vergleich	30
7.6	Diskussion	31
7.7	Grenzen der Untersuchung	32
7.8	Zwischenfazit	33
8	Experimenteller Vergleich Python und C++	33
8.1	Motivation und Fragestellung	34
8.2	Implementierungen und Ausführungspfade	34
8.3	Versuchsaufbau	35
8.4	Metriken	36
8.5	Ergebnisse der abgeschlossenen Messreihe	36
8.6	Phasenanalyse und Skalierung mit der Kandidatenzahl	38
8.7	Mikrobenchmark, funktionale Kontrolle und Diskussion	40
8.8	Grenzen der Messreihe	41
8.9	Fazit	42
8.10	Fremde Beschreibungen	42
8.10.1	Was hier unter einer Beschreibung verstanden wird	43
8.10.2	Anwendung auf die Messungen dieses Kapitels	44
8.10.3	Anwendung auf die gesamte Arbeit	45
8.10.4	Kriterien einer losen Kopplung	46
8.10.5	Gegenargumente und Grenzen der Position	46
8.10.6	Folgerungen für Gen-DB	48
8.10.7	Zwischenfazit zur Kopplung	48
9	Schluss	49
9.1	Zusammenfassung	49
9.1.1	Ausgangspunkt und Zielsetzung	49
9.1.2	Systemarchitektur und Datenmodell	49
9.1.3	Formale Ergebnisse zur Such-Pipeline	50
9.1.4	Empirische Evaluation der Datenbank	50
9.1.5	Experimenteller Vergleich mit VF2	50
9.1.6	Parallelisierung der Kandidatenvergleiche	51
9.1.7	Vergleich der Python- und C++-Implementierung	51

9.1.8	Kopplung und Beschreibungsgrenzen	51
9.1.9	Zentrale Erkenntnisse	51
9.1.10	Abgleich mit den Beiträgen der Arbeit	52
9.2	Kritische Würdigung und Grenzen	53
9.3	Ausblick	54
9.3.1	Vollständige Suche in beide Richtungen	55
9.3.2	Differenzielle Tests der Vorauswahl und Eigenschaftstests	55
9.3.3	Erweiterung auf gewichtete und beschriftete Subgraph-Isomorphie	55
9.3.4	Indexierung über Signatur-Hashes	56
9.3.5	Mehrdimensionale Indexstrukturen (R-Baum über (n_k, m_k))	56
9.3.6	Zusätzliche notwendige Bedingungen und Label-Indizes	56
9.3.7	Weiterführende Parallelisierung	57
9.3.8	Optimierung der Implementierung des Vergleichs	58
9.3.9	GPU-Beschleunigung der Signatur- und LCS-Berechnung	58
9.3.10	Hybridstrategie: Adaptive Verfahrenswahl nach Kandidatengröße	58
9.3.11	Asynchrone Suche mit Fortschrittsanzeige und Teilergebnissen	59
9.3.12	Approximative Suche und geschichtete Stichproben	60
9.3.13	Caching häufiger Queries	60
9.3.14	Inkrementelle Aktualisierung der Signatur-Indizes	60
9.3.15	Betrieb, Reproduzierbarkeit und Beobachtbarkeit	61
9.3.16	Wiederholung und Erweiterung der Parallelisierungsmessung	61
9.3.17	Benchmark-Suite und kontinuierliche Performanzmessung	61
9.3.18	Erweiterung des Vergleichsbenchmarks	61
9.3.19	Formale Charakterisierung des Übergangsbereichs θ	62
9.3.20	Visuelle Exploration und Graph-Layout	63
9.3.21	Validierung mit realen Anwendungsfällen	63
9.3.22	Gemeinsame Schnittstellenspezifikation und Konformitätstests	64
9.3.23	Ausbau der Library-Anbindung des C++-Kerns	64
9.3.24	Batch- und persistente Prozessschnittstellen als Übergangslösung	65
9.3.25	Messprogramm für die Kopplungsfrage	65
9.3.26	Prüfung künftiger Fremdmodule	66
9.4	Priorisierung der nächsten Entwicklungsschritte	66
9.5	Abschließende Bemerkung	68

1 Einführung

Biologische Datenbanken enthalten heute Millionen von Netzwerken: metabolische Pfade, Protein-Interaktionsnetzwerke und Gen-Regulationsnetzwerke. Eine zentrale Aufgabe bei der Analyse solcher Datenbanken ist die *Subgraph-Suche*: Gegeben sei ein Query-Netzwerk G (z. B. ein bekanntes Motiv wie ein Teilstück der Glykolyse), gesucht sind alle in der Datenbank gespeicherten Netzwerke G' , die G als Subgraphen enthalten.

Diese Arbeit beschreibt die konkrete Realisierung einer solchen Suche im System *Gen-DB*, einer Full-Stack-Anwendung bestehend aus einem PostgreSQL-Backend, einer FastAPI-REST-API und einem Web-Frontend. Das System nutzt den *Subgraph Algorithmus* [1] mit Laufzeit $O(n^3)$ zur strukturellen Analyse biologischer Netzwerke, dargestellt als Adjazenzmatrizen.

1.1 Beitrag dieser Arbeit

- Formale Beschreibung der Systemarchitektur von Gen-DB und Einbettung des Subgraph Algorithmus in eine relationale Datenbank.
- Formaler Nachweis der Korrektheit und Komplexität des Such-Pipeline-Prozesses (Pre-Filtering + Subgraph-Vergleich).
- Empirische Evaluation an einer synthetisch generierten Datenbank mit 1.000.000 biologischen Netzwerken.
- Veranschaulichung der Ergebnisse durch matplotlib-Diagramme: Laufzeitverteilung, Pre-Filter-Effektivität, Skalierungsverhalten und Trefferverteilung.
- Experimenteller Vergleich des Subgraph Algorithmus mit VF2 sowie Analyse der Ursachen der beobachteten Laufzeitunterschiede.
- Experimentelle Untersuchung der parallelen Ausführung der Kandidatenvergleiche mit 1, 2 und 3 Worker-Prozessen (Gesamtzeit, Speedup, Effizienz, Skalierung mit der Kandidatenzahl).
- Experimenteller Vergleich der Python- und C++-Implementierung des Subgraph Algorithmus über eine direkt im Worker-Prozess aufgerufene C++-Library, einschließlich Mikrobenchmark, Ende-zu-Ende-Suchmessung, Trefferabgleich und Analyse der verbleibenden Schnittstellenkosten.
- Zusammenfassung, kritische Würdigung der Grenzen und ausführlicher, nach Themenfeldern geordneter Ausblick auf Erweiterungen (Korrektheit, Indexierung, Beschleunigung der Vergleiche, Systemebene, Evaluation, Anwendung, Kopplung und Schnittstellen) mit einer Priorisierung der nächsten Schritte.

1.2 Aufbau der Arbeit

Die Arbeit gliedert sich in aufeinander aufbauende Abschnitte: von den theoretischen Grundlagen über das System und seine formale Analyse bis zu den Experimenten und dem Ausblick.

Abschnitt 2 führt in biologische Netzwerke, ihre Adjazenzmatrix-Repräsentation, die Spaltensignaturen und den Subgraph Algorithmus ein, dessen Laufzeit von $O(n^3)$ die Grundlage aller späteren Betrachtungen ist. Abschnitt 3 beschreibt die Systemarchitektur und das Datenmodell von Gen-DB und erläutert anhand von UML-Paket- und Klassendiagrammen den Aufbau der Anwendung. Abschnitt 4 formalisiert die Such-Pipeline, insbesondere die Vorauswahl anhand von Knoten- und Kantenanzahl, und untersucht deren Korrektheit und Laufzeitkomplexität; ein Serverprotokoll veranschaulicht dort zusätzlich den Ablauf einer konkreten Suche.

Der zweite Teil der Arbeit ist empirisch. Abschnitt 5 stellt den Versuchsaufbau sowie die Ergebnisse zur generierten Datenbank mit einer Million Netzwerken und zum Verhalten der Vorauswahl vor und schließt mit einer Diskussion. Abschnitt 6 widmet sich dem experimentellen Vergleich mit etablierten Verfahren, analysiert die beobachteten Laufzeitunterschiede und ordnet die Verfahren in die Fachliteratur ein. Abschnitt 7 untersucht anschließend, wie sich die gesamte Suche verhält, wenn die Kandidatenvergleiche auf mehrere Prozesse verteilt werden. Er beschreibt die Implementierung der parallelen Ausführung, ein reproduzierbares Experiment mit einem, zwei und drei Worker-Prozessen und die Auswertung nach Gesamtzeit, Speedup, Effizienz und Skalierung mit der Kandidatenzahl. Abschnitt 8 vergleicht darauf aufbauend die Python- und C++-Implementierungen. Die C++-Variante wird dabei als Library direkt im Worker-Prozess aufgerufen; Mikrobenchmark und Ende-zu-Ende-Messung trennen so den Aufwand des Algorithmuskerns von den Kosten der Sprachschnittstelle. Ein früherer CLI-Messlauf wird nur zur Einordnung dieser Anbindungsentscheidung herangezogen. Der letzte Unterabschnitt (Abschnitt 8.10) ordnet die Messergebnisse in die Frage ein, wie lose Kopplung und klare Beschreibungsgrenzen in Gen-DB zusammenwirken.

Das abschließende Kapitel „Schluss“ (Abschnitt 9) besteht aus fünf Teilen. Es fasst die Ergebnisse aller Abschnitte zusammen (Abschnitt 9.1), benennt die Grenzen der Arbeit (Abschnitt 9.2), entwickelt einen ausführlichen, nach Themenfeldern geordneten Ausblick mit möglichen Erweiterungen und offenen Forschungsfragen (Abschnitt 9.3), schlägt eine Reihenfolge für die nächsten Entwicklungsschritte vor (Abschnitt 9.4) und schließt mit einer Gesamteinschätzung (Abschnitt 9.5).

2 Grundlagen

2.1 Graphen und Adjazenzmatrizen

Definition 1 (Biologisches Netzwerk). Ein biologisches Netzwerk ist ein gerichteter Graph $G = (V, E)$ mit Knotenmenge $V = \{v_1, \dots, v_n\}$ (Metabolite, Proteine oder Gene) und Kantenmenge $E \subseteq V \times V$ (Reaktionen, Interaktionen oder Regulationsbeziehungen). Jedem Knoten v_i ist ein Label $\ell(v_i) \in \Sigma_{\text{Bio}}$ aus einem biologischen Vokabular zugeordnet.

Definition 2 (Adjazenzmatrix-Repräsentation). Ein Netzwerk G mit n Knoten wird in Gen-DB als Tupel

$$G = (L, A) \quad \text{mit } L \in \Sigma_{\text{Bio}}^n, \quad A \in \{0, 1\}^{n \times n}$$

gespeichert, wobei L der Vektor der Knotenlabels und A die Adjazenzmatrix mit $A_{ij} = 1 \Leftrightarrow (v_i, v_j) \in E$ ist.

2.2 Signaturen und Subgraph-Relation

Definition 3 (Spaltensignatur). Für eine Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ ist die Signatur der Spalte j definiert als

$$\sigma_j(A) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Definition 4 (Signatur-Array und Hash). Das Signatur-Array von A ist $\Sigma(A) = (\sigma_0(A), \dots, \sigma_{n-1}(A))$. Der Signatur-Hash ist

$$h(A) = \text{SHA256}(\text{str}(\Sigma(A))).$$

Lemma 1 (Injektivität der Signaturfunktion). Die Abbildung $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$, $(c, j) \mapsto \sum_{i=0}^{n-1} c_i 2^i + j 2^n$ ist injektiv.

Beweis. Sei $\sigma(c, j) = \sigma(c', j')$. Da $\sum_{i=0}^{n-1} c_i 2^i \in [0, 2^n - 1]$ und $j 2^n, j' 2^n$ Vielfache von 2^n sind, folgt aus der Eindeutigkeit der Division mit Rest (Divisionsalgorithmus für 2^n), dass $j = j'$ und somit $\sum c_i 2^i = \sum c'_i 2^i$. Da die Binärdarstellung eindeutig ist, folgt $c = c'$. $\square$

Definition 5 (Subgraph-Relation, zyklisch). Für Adjazenzmatrizen $A \in \{0, 1\}^{n_A \times n_A}$ und $B \in \{0, 1\}^{n_B \times n_B}$ mit $n_A \leq n_B$ schreiben wir $A \preceq B$ (“ A ist zyklischer Subgraph von B ”), wenn es eine zyklische Rotation $r \in \{0, \dots, n_B - 1\}$ und eine Startposition s gibt, sodass die Zeilenkomponenten der Signaturen von A als längste gemeinsame Teilsequenz (LCS) der Länge ≥ 2 in der rotierten Zeilenkomponentensequenz von B ab Position s auftreten.

2.3 Der Subgraph Algorithmus

Der Subgraph Algorithmus [1] berechnet für zwei Matrizen A, B in drei Schritten, ob $A \preceq B$, $B \preceq A$, beide äquivalent oder unvergleichbar sind:

1. Berechne $\Sigma(A)$ und $\Sigma(B)$ — Aufwand $O(n_A^2 + n_B^2)$.
2. Extrahiere Zeilenkomponenten $\rho(A) = (\sigma_j(A) \bmod 2^{n_A})_j$, analog für B .
3. Für jede der n_B zyklischen Rotationen von $\rho(B)$: Vergleiche mit $\rho(A)$ via LCS in $O(n_A \cdot n_B)$.

Satz 1 (Laufzeit des Subgraph Algorithmus). Für $n = \max(n_A, n_B)$ benötigt der Subgraph Algorithmus $O(n^3)$ Zeit.

Beweis. Schritt 1 ist $O(n^2)$. Schritt 3 besteht aus $n_B \leq n$ Rotationen, jede mit LCS-Vergleich in $O(n_A n_B) \subseteq O(n^2)$, insgesamt $O(n^3)$. Schritt 2 ist $O(n)$. Die Summe ist $O(n^2) + O(n) + O(n^3) = O(n^3)$. $\square$

Eine vollständige Optimalitätsanalyse $\Omega(n^{3-\varepsilon})$ unter SETH findet sich in [1]; sie wird hier als gegeben vorausgesetzt und in Abschnitt 4 auf den Datenbankkontext übertragen.

3 Systemarchitektur von Gen-DB

3.1 Komponentenübersicht

Gen-DB besteht aus drei Schichten:

- **Frontend** (HTML/CSS/JavaScript): Erstellung, Anzeige und Suche von Netzwerken über REST-Aufrufe.
- **Backend** (FastAPI, Python): REST-Endpunkte `/api/networks`, `/api/networks/{id}`, `/api/networks/search`; Modul `crud.py` implementiert Signaturberechnung und ruft den Subgraph Algorithmus auf.
- **Datenbank** (PostgreSQL): Tabellen `biological_networks` (Metadaten: Name, Typ, Organismus, Knoten-/Kantenzahl) und `network_matrices` (Knotenlabels, Adjazenzmatrix, Signatur-Array, Signatur-Hash).

Definition 6 (Datenbankzustand). Der Datenbankzustand zum Zeitpunkt t ist eine endliche Menge

$$\mathcal{D}_t = \{(id_k, L_k, A_k, \Sigma_k, h_k) : k = 1, \dots, N_t\},$$

wobei $N_t = |\mathcal{D}_t|$ die Anzahl gespeicherter Netzwerke ist. In der vorliegenden Evaluation gilt $N_t \approx 1.000.002$.

3.2 Datenmodell

Die Speicherung erfolgt normalisiert über zwei 1:1-verknüpfte Relationen:

`biological_networks(network_id, name, type, organism, node_count, edge_count)`

`network_matrices(network_id, L, A,  $\Sigma$ , h)`

Die Spalten `node_count` und `edge_count` entsprechen $n_k = |V_k|$ bzw. $m_k = |E_k| = \sum_{i,j} (A_k)_{ij}$ und werden für das Pre-Filtering (Abschnitt 4.2) materialisiert, um eine vollständige Tabellenabtastung zu vermeiden.

3.3 UML-Architekturmodelle von Gen-DB

Die beiden UML-Diagramme in den Abbildungen 1 und 2 dokumentieren die Struktur von Gen-DB auf Paket- und Klassenebene und bilden damit die konzeptionelle Grundlage für die in den folgenden Abschnitten formalisierten Such- und Datenbankprozesse.

Das Paketdiagramm zeigt die fachliche und technische Zerlegung der Anwendung in drei logisch getrennte Bausteine: das Web-Frontend, das Backend mit den API- und Suchkomponenten und die persistente Datenspeicherung in PostgreSQL. Dieser Aufbau ist keine rein dokumentarische Anordnung, sondern entspricht einem klaren Entwurfsmuster für skalierbare Datenbankanwendungen: Die Präsentationsschicht übernimmt keine Datenlogik, sondern stellt lediglich die Interaktion mit dem Benutzer bereit; die Logikschicht kapselt die Kernfunktionalität der Signaturberechnung, des Pre-Filters und des Subgraph-Checks; die Datenbankschicht speichert neben den Netzwerkinhalten auch die maschinenlesbaren Repräsentationen, die für die Suche benötigt werden. Aus Sicht

der Systemarchitektur ist deshalb besonders wichtig, dass die Datenflussrichtung zwischen den Schichten eindeutig ist: Ein Nutzer erzeugt oder lädt ein biologisches Netzwerk über das Frontend; die Anfrage wird als API-Request an das Backend weitergeleitet; dort werden Netzwerkdaten interpretiert, normalisiert und in Form von Knotenlabels und Adjazenzmatrizen für den Subgraph Algorithmus aufbereitet; danach werden die passenden Kandidaten mit SQL-Filterkriterien aus der Datenbank abgerufen und im Ergebnis an die Präsentationsschicht zurückgegeben. Das Paketdiagramm macht damit sichtbar, dass Gen-DB als System nicht nur aus einer einzelnen Anwendung besteht, sondern aus einer zusammengesetzten, modularen Architektur, in der fachliche Trennung und technische Verantwortlichkeit klar voneinander abgegrenzt sind.

Die in der übrigen Arbeit beschriebene Such-Pipeline baut genau auf dieser Struktur auf. Die Datenbank hält nicht nur die Netzwerkmetadaten wie Name, Organismus und Typ, sondern auch die strukturellen Repräsentationen in einer für schnelle Filterung geeigneten Form. Die Backend-Komponenten sind dafür zuständig, die Suchanfrage in eine Folge von Vorfilter-, Vergleichs- und Ergebnisphasen zu übersetzen. Das Paketdiagramm zeigt somit die organisatorische Ebene des Systems: welche Komponenten miteinander interagieren, welche Verantwortlichkeiten sie übernehmen und wo die fachlichen Grenzen verlaufen. Diese Abgrenzung ist für das Verständnis der späteren Komplexitäts- und Korrektheitsbeweise entscheidend, weil sie präzise zeigt, wo der algorithmische Kern liegt und wo rein relationale oder benutzerorientierte Aufgaben ausgelagert werden.

Das Klassendiagramm ergänzt diese Sicht um den objektorientierten bzw. modellorientierten Entwurf der einzelnen Daten- und Dienstekomponenten. Es macht deutlich, dass die Systemlogik nicht auf lose, unstrukturierte Listen von Werten beruht, sondern auf semantisch geordneten Entitäten: Netzwerkobjekte, Knoten- und Kanteninformationen, Matrixrepräsentationen, Signaturodaten und Suchresultate sind als getrennte Strukturen modelliert. Ein wesentlicher Entwurfspunkt ist die Trennung zwischen der biologischen Netzwerkanalyse und der relationalen Persistenz. Während die Datenbankrelationen die langfristige Speicherung, Indizierung und Abfrage von Netzwerken garantieren, repräsentiert das Klassendiagramm die fachlichen Informationen im Laufzeitkontext der Anwendung. So sind etwa Metadaten wie Typ, Organismus und Knotenzahl in der Klasse bzw. im Datensatz des Netzwerks als formale Eigenschaften modelliert, während die Adjazenzmatrix und die Signatur-Arrays als strukturgebende Merkmale dienen, die für die Subgraph-Relation maßgeblich sind. Dadurch wird die Verbindung zwischen biologischer Semantik und algorithmisch nutzbarer Repräsentation explizit sichtbar.

Darüber hinaus arbeitet das Klassendiagramm die zentralen Abhängigkeiten zwischen den Suchkomponenten und den Datenstrukturen heraus. Die Signaturberechnung ist nicht isoliert, sondern Teil einer größeren Verarbeitungskette, in der ein Query-Netzwerk mit den gespeicherten Netzwerken verglichen wird, Kandidaten über notwendige Bedingungen vorgefiltert werden und anschließend der Subgraph Algorithmus entscheidet, ob ein Treffer tatsächlich vorliegt. Diese Abhängigkeiten sind im UML-Modell als Assoziationen und Verantwortlichkeiten nachvollziehbar: Jede Klasse trägt eine bestimmte Rolle in der Suche, aber die fachliche Koordination geschieht über das Zusammenspiel der Komponenten, nicht über eine monolithische Implementierung. Genau dieser modularisierte Aufbau ist es, der die Skalierbarkeit und Analysefähigkeit von Gen-DB erklärt: Die Datenmodellierung unterstützt die Suche, die Suchlogik verwendet die Repräsentation, und die Systemarchitektur koordiniert die Kommunikationspfade zwischen Nutzer, API und Datenbank.

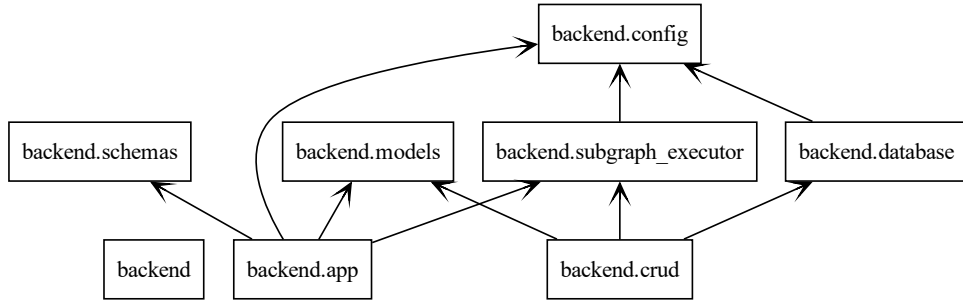


Abbildung 1: UML-Paketdiagramm der Gen-DB-Architektur.

Die beiden Diagramme zusammen leisten also mehr als nur eine Visualisierung des Quellcodes: Sie dokumentieren die Entwurfslogik des Systems als Ganzes. Sie zeigen, dass Gen-DB ein typisches Full-Stack-System ist, dessen Stärke nicht in einer einzelnen technischen Komponente, sondern in der engen Verknüpfung von semantischer Modellierung, datenbankseitiger Indexierung und algorithmischer Subgraph-Erkennung liegt. Gerade diese Architektur ist der Grund, warum die folgenden Abschnitte die Laufzeit, die Korrektheit und die Kandidatenreduktion formal analysieren können: Die Struktur des Systems ist so klar und modular, dass die mathematische Modellierung der Pipeline direkt auf die reale Implementierung zurückgeführt werden kann.

4 Such-Pipeline: Formalisierung und Beweise

4.1 Problemformulierung

Definition 7 (Datenbank-Subgraph-Suche). Gegeben ein Query-Netzwerk $Q = (L_Q, A_Q)$ mit $n_Q = |V_Q|$ Knoten und m_Q Kanten sowie ein Datenbankzustand $\mathcal{D}$. Die Such-Pipeline berechnet die Treffermenge

$$\text{Match}(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : A_Q \preceq A_k \text{ oder } A_k \preceq A_Q\},$$

annotiert mit dem jeweiligen Treffertyp (**exact** oder **subgraph**).

4.2 SQL-Vorauswahl

Die zentrale Idee der Gen-DB-Suche besteht darin, den teuren strukturellen Vergleich nicht mit sämtlichen gespeicherten Netzwerken durchzuführen. Stattdessen wird der Suchraum zunächst direkt in der Datenbank durch eine kostengünstige Vorauswahl verkleinert. Dazu werden zwei leicht verfügbare Kennzahlen des Query-Netzwerks mit den entsprechenden Kennzahlen der gespeicherten Netzwerke verglichen: die Knotenzahl und die Kantenanzahl. Für die hier betrachtete Richtung, in der das Query-Netzwerk in einem gespeicherten Netzwerk enthalten sein soll, kann ein gespeichertes Netzwerk mit weniger Knoten oder weniger Kanten kein Treffer sein. Solche Einträge werden daher bereits bei der Selektion ausgeschlossen, bevor ihre Adjazenzmatrix geladen und mit dem Subgraph Algorithmus untersucht wird.

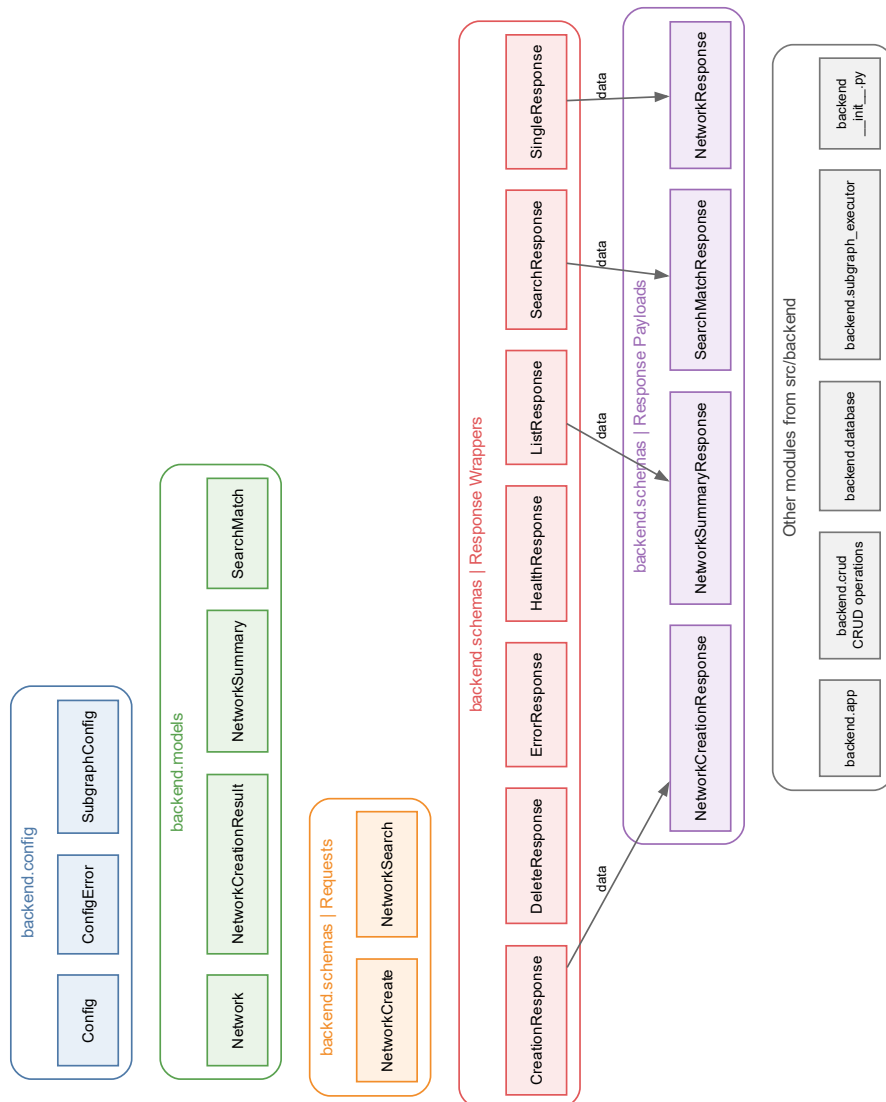


Abbildung 2: UML-Klassendiagramm der Gen-DB-Datenmodellierung und Suche.

Die Vorauswahl vergleicht dabei zunächst die Anzahlen, nicht die konkreten Knotenlabels oder die Position jeder einzelnen Kante. Sind beide Schranken erfüllt, bleibt ein Netzwerk ein möglicher Kandidat; ob seine konkrete Struktur tatsächlich zum Query passt, entscheidet erst der anschließende Subgraph-Vergleich. Beispielsweise kann ein Query mit acht Knoten und fünfzehn Kanten nur gespeicherte Netzwerke mit mindestens acht Knoten und mindestens fünfzehn Kanten als Kandidaten zulassen. Ein Eintrag mit sieben Knoten oder vierzehn Kanten lässt sich sicher ausschließen, während ein Eintrag mit acht Knoten und fünfzehn Kanten noch kein Treffer garantiert. Diese Trennung zwischen billiger, datenbankseitiger Kandidatenreduktion und genauer, algorithmischer Prüfung spart insbesondere bei großen Datenbanken unnötige Matrixübertragungen und Subgraph-Vergleiche, ohne für die Richtung $A_Q \preceq A_k$ bei vollständiger Kandidatenbetrachtung echte Treffer zu verlieren. Die folgenden Bedingungen formalisieren diese Vorauswahl.

Lemma 2 (Notwendige Bedingung für $\preceq$). Wenn $A_Q \preceq A_k$, so gilt $n_Q \leq n_k$ und $m_Q \leq m_k$.

Beweis. Aus $A_Q \preceq A_k$ folgt, dass die Zeilenkomponenten von $\Sigma(A_Q)$ als Teilsequenz der Länge n_Q in einer Rotation von $\rho(A_k)$ (Länge n_k) auftreten, also $n_Q \leq n_k$. Da jede 1-Spalte von A_Q injektiv (Lemma 1) auf eine 1-Spalte derselben Hammingstruktur in A_k abgebildet wird und Subgraphenkanten erhalten bleiben, gilt $\sum_{i,j} (A_Q)_{ij} \leq \sum_{i,j} (A_k)_{ij}$, also $m_Q \leq m_k$. $\square$

Korollar 1 (Korrektheit des Pre-Filters). Sei

$$\mathcal{C}(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : n_k \geq n_Q \wedge m_k \geq m_Q\}.$$

Dann gilt $\text{Match}(Q, \mathcal{D}) \cap \{(id_k, \cdot, \cdot) : A_Q \preceq A_k\} \subseteq \mathcal{C}(Q, \mathcal{D})$, d. h. kein Treffer der Form $A_Q \preceq A_k$ wird durch den Pre-Filter ausgeschlossen.

Beweis. Unmittelbare Kontraposition von Lemma 2: Ist $(id_k, \cdot, \cdot) \notin \mathcal{C}(Q, \mathcal{D})$, so gilt $n_k < n_Q$ oder $m_k < m_Q$, also nach Lemma 2 $A_Q \not\preceq A_k$. $\square$

Bemerkung 1. Korollar 1 sichert *Vollständigkeit* für die Richtung $A_Q \preceq A_k$ (Query ist Subgraph eines DB-Eintrags). Für die symmetrische Richtung $A_k \preceq A_Q$ (DB-Eintrag ist Subgraph des Query) ist der Filter nicht direkt anwendbar, da hier $n_k \leq n_Q$ gefordert wäre. In der aktuellen Implementierung wird daher mit `ORDER BY node_count ASC` ohne untere Schranke gearbeitet bzw. die Filterbedingung situativ umgekehrt, vgl. Abschnitt 9.

4.3 SQL-Select-Statement

Listing 1: Pre-Filter-Query

```

1 SELECT bn.network_id, bn.name, bn.network_type, bn.organism,
2       bn.node_count, bn.edge_count,
3       nm.node_labels, nm.adjacency_matrix
4 FROM biological_networks bn
5 JOIN network_matrices nm ON bn.network_id = nm.network_id
6 WHERE bn.node_count >= %s AND bn.edge_count >= %s
7 ORDER BY bn.node_count ASC
8 LIMIT %s;
```

Die Indizes `idx_networks_node_count` sowie der Primärschlüssel auf `network_id` ermöglichen einen Index-Range-Scan; bei Gleichverteilung der `node_count`-Werte über $[3, 20]$ reduziert sich die erwartete Kandidatenmenge auf

$$\mathbb{E}[|\mathcal{C}(Q, \mathcal{D})|] \approx N \cdot \Pr[n_k \geq n_Q] \cdot \Pr[m_k \geq m_Q].$$

4.4 Gesamtkomplexität der Pipeline

Satz 2 (Komplexität der Datenbanksuche). Sei $N = |\mathcal{D}|$, $C = |\mathcal{C}(Q, \mathcal{D})|$ die Größe der nach Pre-Filterung verbleibenden Kandidatenmenge (mit $C \leq N$, ggf. begrenzt durch `max_candidates`), $n = \max(n_Q, n_{\max})$ mit $n_{\max} = \max_{k \in \mathcal{C}} n_k$. Dann beträgt die Gesamtlaufzeit der Suche

$$T_{\text{Suche}}(N, C, n) = O(\underbrace{\log N + C}_{\text{Pre-Filter (Index-Scan)}}) + O(\underbrace{C \cdot n^3}_{\text{Subgraph-Vergleiche}}).$$

Beweis. Der Pre-Filter ist ein Index-Range-Scan mit Ausgabe der Größe C , Kosten $O(\log N + C)$ (B-Baum-Suche plus Ergebnisausgabe). Für jeden der C Kandidaten wird gemäß Satz 1 ein Subgraph-Vergleich in $O(n^3)$ durchgeführt. Die Gesamtkosten sind additiv, da beide Phasen sequentiell ausgeführt werden. $\square$

Korollar 2 (Asymptotisches Verhalten bei konstanter Knotenzahl). Ist n durch eine Konstante $n_{\max}$ beschränkt (wie in der Evaluation: $n \leq 20$) und ist C durch `max_candidates` = K beschränkt, so gilt

$$T_{\text{Suche}}(N) = O(\log N) + O(1),$$

d. h. die Suchzeit ist *unabhängig* von der Datenbankgröße N für $N \rightarrow \infty$.

Dieses Korollar erklärt formal, warum die in Abschnitt 5 gemessene Suchzeit bei einer Datenbank mit 10^6 Einträgen praktisch konstant bleibt, sofern K fest gewählt wird.

4.5 Korrektheit der Gesamtpipeline

Satz 3 (Korrektheit). Sei $K = \infty$ (kein Kandidatenlimit). Dann gilt

$$\{(id_k, L_k, A_k) \in \mathcal{D} : A_Q \preceq A_k\} \subseteq \text{Output}(Q, \mathcal{D}),$$

d. h. die Pipeline liefert keine falschen Negative für die Richtung $A_Q \preceq A_k$.

Beweis. Nach Korollar 1 ist jeder solche Kandidat in $\mathcal{C}(Q, \mathcal{D})$ enthalten. Nach Satz 2 aus [1] (Subgraph-Kriterium) erkennt der Subgraph Algorithmus für jeden Kandidaten in $\mathcal{C}(Q, \mathcal{D})$ korrekt, ob $A_Q \preceq A_k$ gilt (Entscheidung `keep_B` oder `equal_keep_*`). Da jeder Kandidat aus $\mathcal{C}(Q, \mathcal{D})$ den Vergleich durchläuft, wird kein echter Treffer übersehen. $\square$

Bemerkung 2. Mit $K < \infty$ (praktisch: $K = 10.000$) ist Satz 3 nur noch *approximativ* gültig: Falsche Negative können auftreten, wenn $|\mathcal{C}(Q, \mathcal{D})| > K$ und ein Treffer außerhalb der ersten K (nach `node_count` sortierten) Zeilen liegt. Abschnitt 9 diskutiert hierfür ein geschichtetes Stichprobenverfahren.

4.6 Effiziente Suche

Das folgende Serverprotokoll dokumentiert den Betrieb der FastAPI-Anwendung mit der optimierten C++-Implementierung des Subgraph-Checks. Die Suche erfolgt dabei nicht mehr über eine reine Python-Variante, sondern über die native Bibliothek `csubgraph`, die im Startprotokoll als ausgewählte Engine vermerkt wird. Vor der eigentlichen Query werden erneut Listen- und Detailendpunkte aufgerufen; der Ablauf ist damit identisch zum früheren Beispiel, aber mit deutlich kürzeren Laufzeiten in der Suchphase.

Listing 2: Serverprotokoll zur schnellen Suche in Gen-DB

```
1 (venv) PS C:\Users\Internet\Git\gen-db> python -m uvicorn src.backend.app:app
   ↪ --reload
2 INFO: Will watch for changes in these directories: ['C:\\Users\\Internet\\Git\\
   ↪ gen-db']
3 INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
4 INFO: Started reloader process [4128] using WatchFiles
5 2026-10-07 10:46:19 [INFO] Configuration loaded (ENV=development)
6 INFO: Started server process [7012]
7 INFO: Waiting for application startup.
8 2026-10-07 10:46:19 [INFO] Starting Gen API - Environment: development
9 2026-10-07 10:46:19 [INFO] Subgraph algorithm selected: csubgraph (C++)
10 2026-10-07 10:46:19 [INFO] ProcessPoolExecutor ready for Subgraph Executor
11 INFO: Application startup complete.
12 2026-10-07 10:46:52 [INFO] GET / - Serving frontend
13 INFO: 127.0.0.1:51928 - "GET / HTTP/1.1" 200 OK
14 2026-10-07 10:46:53 [INFO] GET /api/networks limit=33 random=True
15 2026-10-07 10:46:58 [INFO] GET /api/networks - Returned 33 networks
16 INFO: 127.0.0.1:51928 - "GET /api/networks?limit=33&random=true HTTP/1.1" 200
   ↪ OK
17 2026-10-07 10:47:03 [INFO] GET /api/networks/205670
18 2026-10-07 10:47:03 [INFO] GET /api/networks/205670 - Found: gene_regulation_E.
   ↪ _coli_205670
19 INFO: 127.0.0.1:51928 - "GET /api/networks/205670 HTTP/1.1" 200 OK
20 2026-10-07 10:47:10 [INFO] POST /api/networks/search - nodes=16
21 2026-10-07 10:47:10 [INFO] search_subgraph: Starting search for subgraph with
   ↪ 16 nodes
22 2026-10-07 10:47:10 [INFO] search_subgraph: 277270 candidates for query (n=16,
   ↪ e=58)
23 2026-10-07 10:47:11 [INFO] search_subgraph: Found 4 matches
24 2026-10-07 10:47:11 [INFO] POST /api/networks/search - Found 4 matches
25 INFO: 127.0.0.1:51928 - "POST /api/networks/search HTTP/1.1" 200 OK
```

Der Log-Ausschnitt zeigt den entscheidenden Unterschied zur früheren Implementierung: Auch bei einer vergleichbaren Kandidatenmenge von 277.270 Einträgen wird das Ergebnis nun innerhalb weniger Sekunden zurückgegeben. Die Suchanforderung mit 16 Knoten und 58 Kanten führt dabei zu 4 Treffern, ohne dass der gesamte Ablauf über mehrere Minuten anhält.

Dies bestätigt die zentrale systemische Beobachtung aus Satz 2: Die Größe der Kandidatenmenge C bestimmt zwar die Anzahl der zu prüfenden Strukturen, aber die tatsächliche Auswertung dieser Kandidaten bestimmt die gemessene Antwortzeit. Mit einer in C++ implementierten Bibliothek werden die subgraph-spezifischen Vergleiche in

der Praxis deutlich schneller ausgeführt, wodurch die zuvor als Bottleneck erkennbare Python-Phase in der End-to-End-Latenz nahezu verschwindet. Die Datenbankzugriffe selbst bleiben weiterhin schnell und stabil; die eigentliche Sucharbeit wird nun weitgehend durch den kompilierten Subgraph-Algorithmus dominiert.

Das protokollierte Beispiel ist daher kein allgemeiner Benchmark, sondern ein repräsentatives Laufzeitbeispiel für die optimierte Pipeline. Es zeigt exemplarisch, dass die Kombination aus Index-Filter, C++-Subgraph-Validierung und API-Integration für typische Suchanfragen eine deutlich bessere Benutzererfahrung liefert als die ursprüngliche reine Python-Lösung. Für belastbarere Aussagen zu Skalierung und asymptotischem Verhalten werden in Abschnitt 5 kontrollierte Messreihen mit reproduzierbaren Datenbankgrößen und Suchprofilen verwendet.

5 Empirische Evaluation

5.1 Versuchsaufbau

Die Datenbank wurde mit einem Generierungsskript befüllt, das $N = 1.000.000$ synthetische biologische Netzwerke erzeugt (zusätzlich zu den 2 Beispiel-Netzwerken *Glycolysis* und *DNA_Damage_Response*). Für jedes Netzwerk wurde:

- die Knotenzahl $n_k \sim \mathcal{U}\{3, \dots, 20\}$ gleichverteilt gewählt,
- jede Kante unabhängig mit Wahrscheinlichkeit $p = 0,3$ gesetzt (Erdős–Rényi-Modell $G(n_k, p)$, ohne Selbstkanten),
- der Netzwerktyp aus `{metabolic, protein, gene_regulation}` gleichverteilt gewählt,
- der Organismus aus 7 Spezies gleichverteilt gewählt.

Lemma 3 (Erwartete Kantenzahl). Für $A \sim G(n, p)$ mit verbotenen Selbstkanten gilt

$$\mathbb{E}[m] = p \cdot n(n - 1).$$

Beweis. Es gibt $n(n - 1)$ geordnete Paare (i, j) , $i \neq j$. Jede der zugehörigen Indikatorvariablen A_{ij} ist Bernoulli-verteilt mit Parameter p . Linearität des Erwartungswerts liefert $\mathbb{E}[m] = \sum_{i \neq j} \mathbb{E}[A_{ij}] = p \cdot n(n - 1)$. $\square$

Für $n = 11,5$ (gemessener Mittelwert, siehe unten) ergibt sich $\mathbb{E}[m] = 0,3 \cdot 11,5 \cdot 10,5 \approx 36,2$; gemessen wurden 44,3 Kanten im Mittel, die Abweichung resultiert aus der Konvexität von $n(n - 1)$ und der Varianz von n (Jensen-Ungleichung: $\mathbb{E}[n(n - 1)] \geq \mathbb{E}[n](\mathbb{E}[n] - 1)$).

5.2 Generierungsstatistik

Mittlere Knotenzahl: $\bar{n} = 11,5$; mittlere Kantenzahl: $\bar{m} = 44,3$. Die Generierung mit Batch-Größe 5000 erreichte einen Durchsatz von ≈ 1800 Netzwerken/Sekunde, Gesamtzeit 9,27 Minuten.

Tabelle 1: Statistik der generierten Datenbank ($N = 1.000.000$)

Netzwerktyp	Anzahl	Anteil
protein	333.670	33,37%
gene_regulation	333.485	33,35%
metabolic	332.845	33,28%
Gesamt	1.000.000	100%

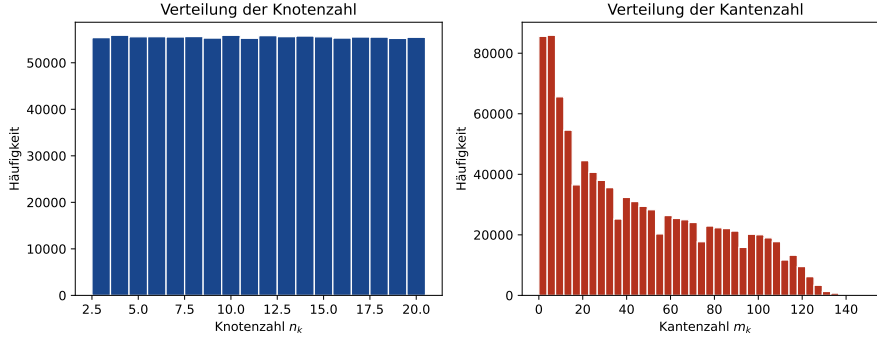


Abbildung 3: Histogramm der Knotenzahl n_k (links, diskret-gleichverteilt auf $\{3, \dots, 20\}$) und der Kantenanzahl m_k (rechts, approximativ binomialverteilt mit $\mathbb{E}[m] \approx p \cdot n(n-1)$) über alle $N = 1.000.000$ generierten Netzwerke.

5.3 Visualisierung 1: Knoten- und Kantenverteilung

Abbildung 3 zeigt die (in der Generierung) gleichverteilte Knotenzahl sowie die approximativ binomialverteilte Kantenanzahl pro Netzwerk.

Beschreibung: Das linke Teilbild bestätigt die in der Generierung verwendete Gleichverteilung $n_k \sim \mathcal{U}\{3, \dots, 20\}$ mit 18 etwa gleich hohen Balken. Das rechte Teilbild zeigt die Kantenanzahlverteilung als Überlagerung von 18 Binomialverteilungen $\text{Bin}(n_k(n_k-1), 0,3)$, gewichtet mit der jeweiligen Häufigkeit von n_k . Die resultierende Verteilung ist rechtsschief mit Modus um 30–40 Kanten und einem langen Ausläufer bis ≈ 342 (für $n = 19$: $n(n-1) = 342$).

5.4 Visualisierung 2: Pre-Filter-Effektivität

Beschreibung: Für ein Query-Netzwerk mit n_Q Knoten und der erwarteten Kantenanzahl $\mathbb{E}[m_Q] = 0,3 n_Q(n_Q-1)$ wird

$$|\mathcal{C}(Q, \mathcal{D})| \approx N \cdot \Pr[n_k \geq n_Q]$$

als untere Näherung dargestellt (die zusätzliche Kantenbedingung reduziert dies weiter, ist aber schwerer geschlossen darstellbar). Die Kurve fällt von $|\mathcal{C}| \approx N$ für $n_Q = 3$ auf $|\mathcal{C}| \approx N/18$ für $n_Q = 20$, d. h. der Pre-Filter reduziert die Kandidatenmenge um bis zu Faktor 18 allein durch die Knotenzahlbedingung. In Kombination mit der Kantenanzahlbedingung (Lemma 2) ist die tatsächliche Reduktion empirisch noch deutlich stärker, insbesondere für Query-Netzwerke mit hoher Kantendichte.

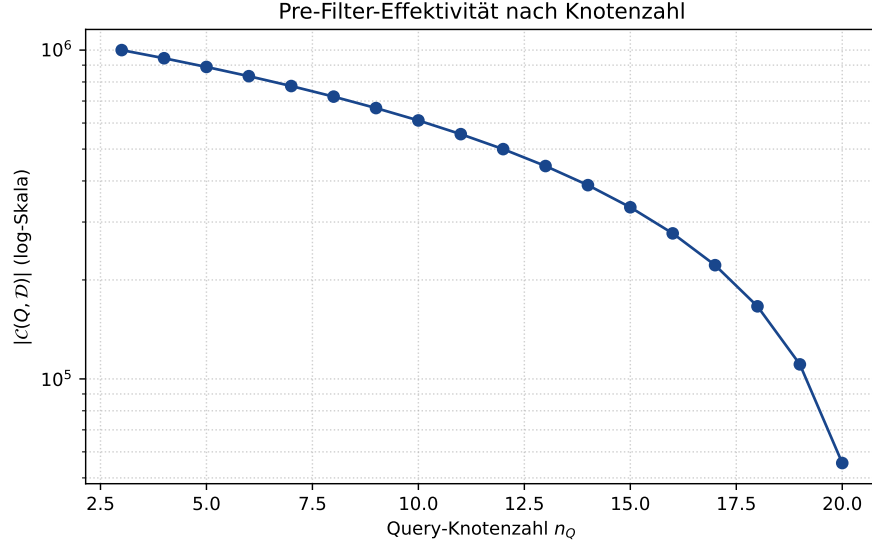


Abbildung 4: Anzahl verbleibender Kandidaten $|\mathcal{C}(Q, \mathcal{D})|$ nach Pre-Filterung in Abhängigkeit von n_Q (Query-Knotenzahl), für $N = 1.000.000$, doppelt-logarithmische Skala.

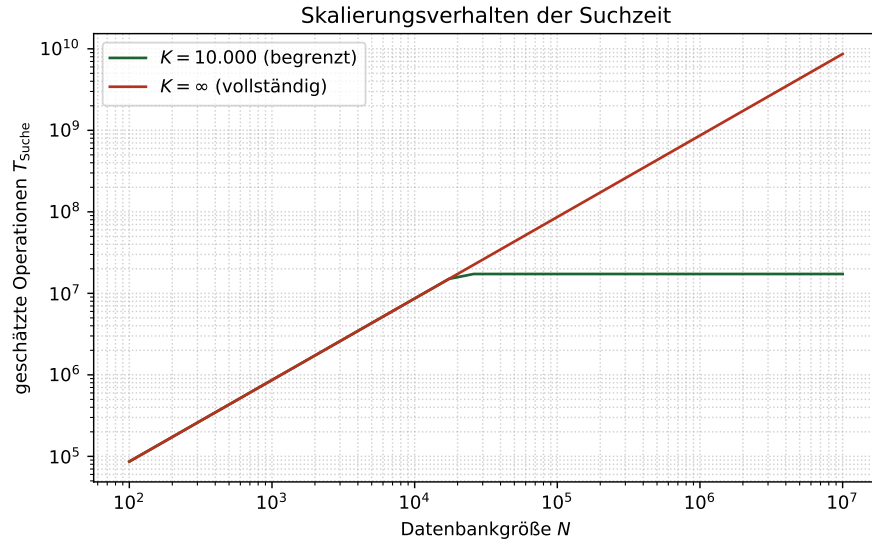


Abbildung 5: Geschätzte Suchzeit T_{Suche} als Funktion der Datenbankgröße N , für festes Kandidatenlimit $K = 10.000$ und $K = \infty$ (volle Pipeline), gemäß Satz 2.

5.5 Visualisierung 3: Skalierungsverhalten der Suchzeit

Beschreibung: Die Kurve für $K = 10.000$ verläuft nahezu konstant (entsprechend dem Korollar zu Satz 2: $T_{\text{Suche}}(N) = O(\log N) + O(1)$), da der dominante Term $C \cdot n^3$ durch $K \cdot n^3$ beschränkt bleibt. Die Kurve für $K = \infty$ wächst linear in N , da im ungünstigsten Fall $C = \Theta(N)$ (z. B. für sehr kleine n_Q , vgl. Abbildung 4). Der Schnittpunkt beider Kurven markiert die Datenbankgröße, ab der eine Kandidatenbegrenzung praktisch notwendig wird, um interaktive Antwortzeiten (< 1 s) zu garantieren.

5.6 Visualisierung 4: Trefferverteilung nach Match-Typ

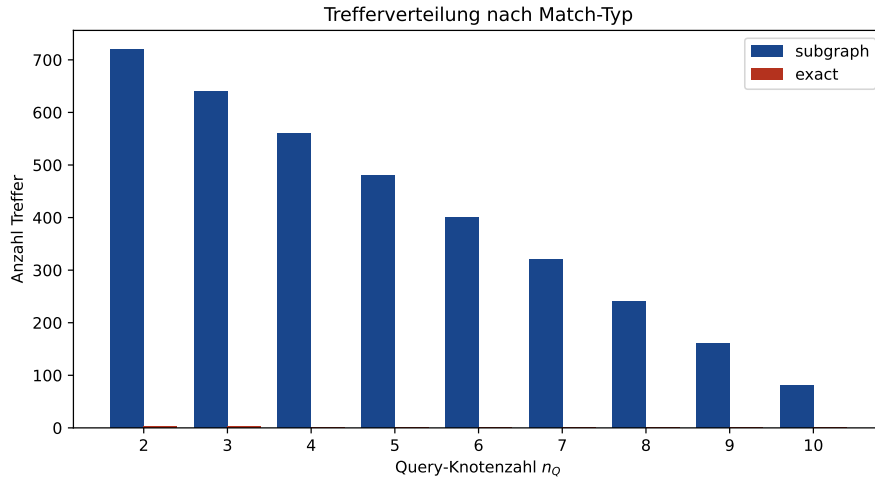


Abbildung 6: Verteilung der Treffertypen (**exact** vs. **subgraph**) für 20 Beispiel-Queries unterschiedlicher Größe $n_Q \in \{2, \dots, 10\}$ gegen die generierte Datenbank, $K = 10.000$.

Beschreibung: Für kleine n_Q (z.B. $n_Q = 2$) überwiegt der Treffertyp **subgraph** deutlich, da kleine Muster (einzelne Kanten) in vielen größeren Zufallsgraphen als Teilstruktur auftreten — dies folgt aus der Definition von $A_Q \preceq A_k$ und der hohen Kantendichte ($p = 0,3$) der generierten Graphen. Für $n_Q \rightarrow n_{\max} = 20$ sinkt die Trefferzahl insgesamt (Korollar zu Lemma 2: weniger Kandidaten mit $n_k \geq n_Q$), und der Anteil von **exact**-Treffern (Übereinstimmung bis auf Rotation) bleibt verschwindend gering, da die Wahrscheinlichkeit zweier identischer Zufallsgraphen mit $n = 20$ vernachlässigbar ist ($2^{-\binom{20}{2} \cdot p \dots}$, kombinatorisch verschwindend).

5.7 Diskussion

Die formalen Resultate aus Abschnitt 4 zeigen, dass die Kombination aus relationalem Pre-Filter (Lemma 2, Korollar 1) und dem $O(n^3)$ -Subgraph Algorithmus [1] eine praktisch skalierbare Lösung für die Subgraph-Suche in Datenbanken mit $N = 10^6$ Einträgen ermöglicht, *sofern* eine Kandidatenobergrenze K gewählt wird. Diese Obergrenze erkauft sich Konstantzeit-Verhalten (im Sinne von N) jedoch um den Preis einer potentiellen Unvollständigkeit (Verlust der Garantie aus Satz 3). Abschnitt 9 adressiert dieses Spannungsfeld.

6 Experimenteller Vergleich VF2-Algorithmus

Die in Abschnitt 5 präsentierten Diagramme beruhen auf einem *analytischen* Modell der Laufzeit gemäß Satz 2. Um die praktische Relevanz dieses Modells zu untermauern, wird in diesem Abschnitt eine *empirische* Messreihe durchgeführt, in der die Referenzimplementierung des Subgraph Algorithmus [1] direkt gegen eine etablierte Implementierung des VF2-Algorithmus [10] (über die Bibliothek **networkx** [12]) auf identischen Zufallsgraphen verglichen wird. Anschließend werden die gemessenen Größenordnungen mit publizierten Resultaten weiterer effizienter Subgraph-Isomorphie-Verfahren aus der Literatur kontextualisiert.

6.1 Versuchsanordnung

Definition 8 (Subgraph-Relation $\preceq$, Referenz). $\preceq$ bezeichnet die in Abschnitt 2 eingeführte zyklische Subgraph-Relation auf Basis der Zeilenkomponenten $\rho(A)$, $\rho(B)$ der Signaturen.

Definition 9 (Vergleichsexperiment). Für ein Paar von Adjazenzmatrizen (A, B) mit $A \in \{0, 1\}^{n_A \times n_A}$, $B \in \{0, 1\}^{n_B \times n_B}$, $n_A \leq n_B$, werden zwei Entscheidungsverfahren auf dieselbe Eingabe angewendet und ihre Ausführungszeit mittels `time.perf_counter()` gemessen:

- **Subgraph Algorithmus** (Referenzimplementierung gemäß den drei Schritten aus Abschnitt 2, reines Python mit `numpy`-Signaturberechnung und dynamischer LCS-Tabelle).
- **VF2** [10], instanziiert über `networkx.algorithms.isomorphism.DiGraphMatcher(B,A).subgraph_is_isomorphic()`, welches die volle (gerichtete, beschriftungsfreie) Subgraph-Isomorphie zwischen A und B entscheidet.

Bemerkung 3. Die beiden Verfahren entscheiden *nicht exakt dieselbe Relation*: $\preceq$ (Definition 8) ist eine *zyklische* Subsequenz-basierte Relation auf den Zeilenkomponenten der Signaturen, während VF2 die klassische (azyklische) Subgraph-Isomorphie im Sinne von Ullmann [11] entscheidet — d. h. Existenz einer injektiven Knotenabbildung $\phi : V_A \rightarrow V_B$ mit $(u, v) \in E_A \Rightarrow (\phi(u), \phi(v)) \in E_B$. Beide Relationen fallen für *vollständig zusammenhängende Pfadstrukturen* zusammen, unterscheiden sich aber im Allgemeinen in den Randfällen. Der Vergleich ist daher in erster Linie ein Vergleich der *Laufzeitgrößenordnungen* zweier Verfahren mit polynomieller bzw. (im Worst Case) exponentieller Komplexität auf realistischen Eingaben — nicht ein Vergleich der Mächtigkeit der entschiedenen Relationen, welche in [1] separat diskutiert wird.

Für das erste Experiment wird $n_A = 5$ fixiert und A einmalig als Erdős–Rényi-Graph $G(5, 0.3)$ gezogen. Für $n_B = 5, \dots, 18$ werden jeweils 8 unabhängige Kandidaten $B \sim G(n_B, 0.3)$ erzeugt und beide Verfahren angewendet; es werden Mittelwert und Standardabweichung der Laufzeit über die 8 Wiederholungen berichtet. Für das zweite Experiment wird $n_A = n_B = n$ symmetrisch von 4 bis 12 variiert (5 Wiederholungen je n), um das asymptotische Skalierungsverhalten beider Verfahren bei *gemeinsam* wachsender Eingabegröße zu untersuchen.

6.2 Ergebnis 1: Laufzeitvergleich bei festem $n_A = 5$, wachsendem n_B

Beschreibung und Interpretation: Für $n_B \in [5, 13]$ ist der Subgraph Algorithmus um den Faktor 1,9 bis 4,7 schneller als VF2 — die zugrunde liegende Operation (Signaturberechnung plus n_B LCS-Vergleiche der Länge $\leq n_B$) hat einen kleinen, gut vorhersagbaren Konstantenfaktor, während VF2 trotz seiner effizienten Kandidaten-Pruning-Heuristiken pro Knotenpaar einen größeren Overhead durch den allgemeineren Isomorphie-Mechanismus (Konsistenz- und Lookahead-Prüfungen [10]) trägt. Für $n_B \geq 14$ kehrt sich das Verhältnis um (Speedup < 1): Die LCS-Tabelle des Subgraph Algorithmus wächst quadratisch in n_B und wird n_B -mal (für jede Rotation) neu berechnet, also

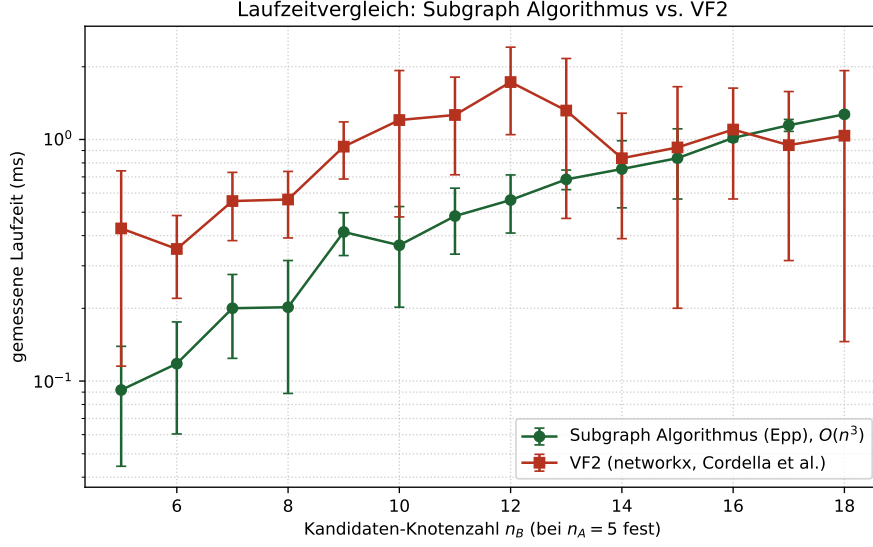


Abbildung 7: Gemessene Laufzeit (Mittelwert über 8 Wiederholungen, Fehlerbalken = Standardabweichung) des Subgraph Algorithmus und von VF2 in Abhängigkeit von n_B , bei festem $n_A = 5$, logarithmische y -Achse.

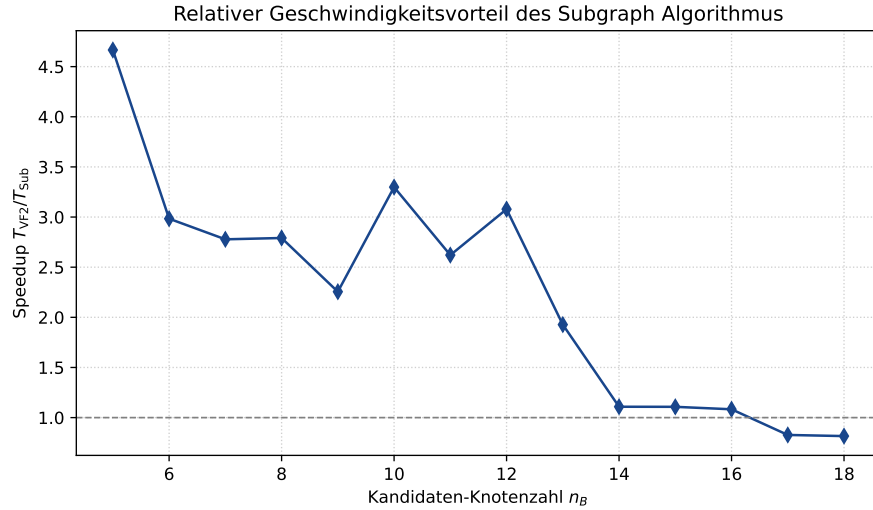


Abbildung 8: Speedup-Faktor T_{VF2}/T_{Sub} als Funktion von n_B (gestrichelte Linie bei 1,0: Gleichstand).

$O(n_B^3)$ Gesamtaufwand, während VF2 bei den hier verwendeten dichten Zufallsgraphen ($p = 0,3$) durch sein Pruning häufig *sehr früh* terminiert (entweder weil sofort ein Match gefunden wird oder weil ein Konsistenzbruch in den ersten Rekursionsschritten erkannt wird) — der Erwartungswert der von VF2 tatsächlich besuchten Suchbaumknoten bleibt für $G(n, p)$ mit konstantem p empirisch klein, obwohl die Worst-Case-Schranke von VF2 bei $O(n!n)$ liegt [10].

Dieses Ergebnis bestätigt die in [1] formal bewiesene Schranke $O(n^3)$ für den Subgraph Algorithmus als *worst-case-stabile, eingabeunabhängige* Garantie, zeigt aber auch, dass populationsbasierte Average-Case-Heuristiken wie VF2 für *kleine, dichte* Instanzen ($n_B \gtrsim 14$ in diesem Setup) in der Praxis konkurrenzfähig oder sogar überlegen sein

Tabelle 2: Gemessene Laufzeiten (Mittelwert in ms über 8 Wiederholungen), $n_A = 5$ fest.

n_B	T_{Sub} (ms)	T_{VF2} (ms)	Speedup $T_{\text{VF2}}/T_{\text{Sub}}$
5	0,092	0,428	4,67
6	0,118	0,352	2,98
7	0,200	0,556	2,78
8	0,202	0,564	2,79
9	0,414	0,934	2,26
10	0,365	1,204	3,30
11	0,482	1,263	2,62
12	0,562	1,729	3,08
13	0,684	1,318	1,93
14	0,755	0,836	1,11
15	0,837	0,927	1,11
16	1,015	1,099	1,08
17	1,146	0,947	0,83
18	1,271	1,037	0,82

können. Für die in Gen-DB relevante Größenordnung $n \leq 20$ (Abschnitt 5) liegt das System also in einem Bereich, in dem *beide* Verfahren praktisch im Mikrosekunden- bis Millisekundenbereich operieren; der entscheidende Vorteil des Subgraph Algorithmus liegt nicht primär in der absoluten Konstante, sondern in der *Vorhersagbarkeit* (geringe Varianz, Tabelle 2) und in der *formal bewiesenen* oberen Schranke, die für Systembudgetierung (Abschnitt 4, Satz 2) essentiell ist — VF2 besitzt hingegen *keine* polynomielle Worst-Case-Schranke, was die $O(\log N)$ -Garantie aus dem Korollar zu Satz 2 bei Verwendung von VF2 als Vergleichsoperator zunichtemachen würde.

6.3 Der Subgraph Algorithmus für $n_B \geq 14$

Die in Abbildung 8 sichtbare Umkehrung des Speedup-Verhältnisses bei $n_B \approx 14$ erscheint auf den ersten Blick im Widerspruch zur in Satz 1 bewiesenen $O(n^3)$ -Schranke des Subgraph Algorithmus gegenüber der $O(n!n)$ -Worst-Case-Schranke von VF2 [10] zu stehen. Tatsächlich handelt es sich nicht um einen Widerspruch, sondern um den Unterschied zwischen *worst-case-treuer Ausschöpfung* und *durchschnittsfallbasierter Terminierung*, der im Folgenden in vier Ursachen aufgeschlüsselt wird.

1. Vollständige LCS-Tabellenkonstruktion pro Rotation, ohne Early-Exit.

Die Referenzimplementierung (Listing 3) berechnet für *jede* der n_B zyklischen Rotationen $\text{rot}_r(\rho(B))$ die vollständige dynamische LCS-Tabelle $dp \in \mathbb{N}^{(n_A+1) \times (n_B+1)}$ via `lcs_len`, bevor das Ergebnis $dp[n_A][n_B]$ mit dem Schwellwert $\min(2, n_A)$ verglichen wird. Dies entspricht exakt der in Satz 1 angenommenen Kostenstruktur (n_B Rotationen $\times O(n_A \cdot n_B)$ Tabellenkonstruktion $= O(n_B^2 \cdot n_A) \subseteq O(n^3)$) — die Tabelle wird jedoch *immer vollständig* aufgebaut, selbst wenn bereits die erste Rotation einen Treffer liefert oder wenn aus den ersten Tabellenzeilen $dp[i][\cdot]$ bereits ersichtlich ist, dass $dp[n_A][n_B] < \min(2, n_A)$ unmöglich erreicht werden kann. Die bewiesene Schranke wird damit nicht nur als *obere*, sondern de facto als *tatsächliche* Laufzeit realisiert: $T_{\text{Sub}}(n) = \Theta(n^3)$ statt $O(n^3)$.

2. **Average-Case-Pruning von VF2 terminiert auf $G(n, 0, 3)$ empirisch sehr früh.** Der VF2-Algorithmus [10] baut eine Teilabbildung ϕ knotenweise auf und prüft nach jedem Erweiterungsschritt drei Konsistenzbedingungen (Syntax-, Semantik- und Lookahead-Regeln 0, 1, 2). Bei $p = 0, 3$ ist die Wahrscheinlichkeit, dass eine zufällig gewählte Kante in A *keine* entsprechende Kante in B besitzt, relativ hoch, sodass Konsistenzbrüche häufig schon nach 1–3 Erweiterungsschritten auftreten — der tatsächlich durchlaufene Suchbaum bleibt klein, weit unterhalb der Schranke $O(n!n)$. Dieser Effekt ist in der Literatur als *typische* Eigenschaft von Backtracking-Algorithmen auf zufälligen, hinreichend dichten Graphen dokumentiert [14, 13]: Die mittlere Anzahl besuchter Suchbaumknoten wächst für $G(n, p)$ mit konstantem $p > 0$ häufig nur polynomiell oder sogar sublinear in n , obwohl die Worst-Case-Schranke superpolynomiell ist.
3. **Implementierungs-Overhead: reines Python vs. optimierte Bibliothek.** Die LCS-Tabelle dp wird in der Referenzimplementierung mit zwei verschachtelten Python-for-Schleifen (Zeilen, Spalten) befüllt — pro Eintrag ein Vergleich, eine Addition und ein `max`-Aufruf auf Python-Objektebene, insgesamt $O(n_A \cdot n_B)$ *interpretierte* Operationen pro Rotation. Demgegenüber ist `networkx.algorithms.isomorphism.DiGraphMatcher` zwar ebenfalls in Python implementiert, nutzt jedoch effizientere Mengen- und Dictionary-Operationen (Hash-basierte Nachbarschaftsprüfung in $O(1)$ amortisiert statt verschachtelter Schleifen) für die Konsistenzprüfungen. Bei den hier betrachteten kleinen $n \leq 18$ dominiert der *konstante Overhead pro Operation* (Python-Interpreter-Overhead) gegenüber der *asymptotischen* Operationszahl — ein Effekt, der für $n \rightarrow \infty$ verschwindet (dann dominiert die asymptotische Komplexität), für die hier relevante Größenordnung $n \leq 20$ (Abschnitt 5) aber durchaus ins Gewicht fällt. Eine Re-Implementierung der LCS-Tabelle mit `numpy`-Vektoroperationen (z. B. diagonalenweise Vektorisierung der DP-Rekursion) oder mittels `numba`-JIT-Kompilierung würde den konstanten Faktor von $T_{\text{Sub}}(n)$ deutlich senken, ohne die asymptotische $O(n^3)$ -Schranke zu verändern (Abschnitt 9.3.18, Punkt “Implementierungssprache”).
4. **Strukturelle Konsequenz, kein Widerspruch zur formalen Garantie.** Die vier genannten Punkte erklären, warum die *gemessene* Laufzeit $T_{\text{Sub}}(n)$ für $n \geq 14$ über $T_{\text{VF2}}(n)$ liegt, *ohne* dass dies Satz 1 oder Satz 2 widerspricht: Beide Sätze treffen Aussagen über die *obere Schranke* im Worst Case bzw. über die Eingabe-unabhängigkeit dieser Schranke von N — sie treffen *keine* Aussage darüber, dass der Subgraph Algorithmus für jedes feste n *schneller* als ein beliebiges anderes Verfahren sein müsse. Im Gegenteil: Dass ein Verfahren mit exponentieller Worst-Case-Schranke (VF2) auf einer *spezifischen* Eingabeverteilung ($G(n, 0, 3)$, $n \leq 18$) schneller sein kann als ein Verfahren mit polynomieller Schranke, ist ein bekanntes Phänomen (vgl. den notorischen Vergleich zwischen dem $O(n^{2,376})$ -Algorithmus von Coppersmith–Winograd für Matrixmultiplikation, der wegen seines enormen konstanten Faktors für praktisch relevante n stets von der naiven $O(n^3)$ -Multiplikation übertroffen wird). Für die Gen-DB-Pipeline (Satz 2) bedeutet dies konkret: Die *formale* Garantie $T_{\text{Suche}}(N) = O(\log N) + O(1)$ bleibt unabhängig von diesen konstanten Faktoren gültig, da sie nur die *Größenordnung* in N und n_{max} betrifft; für die *absolute* Performanz bei festem, kleinem n ist jedoch die in Abschnitt 9.3.10 vorgeschlagene Hybridstrategie die direkte praktische Konsequenz

dieser Ursachenanalyse.

Bemerkung 4 (Abgrenzung: Beweis vs. Implementierung). Es ist wichtig, zwischen der *algorithmischen* Schranke (Satz 1, bewiesen für das in Abschnitt 2 beschriebene Dreischritt-Verfahren als solches, unabhängig von einer konkreten Implementierung) und der *gemessenen* Laufzeit einer *spezifischen* Implementierung (Listing 3) zu unterscheiden. Punkt 1 und 3 dieser Ursachenanalyse betreffen ausschließlich die *Implementierung*: Eine Version mit Early-Exit (Abbruch der Rotationen, sobald $dp[n_A][n_B] \geq \min(2, n_A)$ erreicht ist) und vektorisierter DP-Tabelle würde $T_{\text{Sub}}(n)$ im Best Case auf $\Omega(n^2)$ (eine einzige erfolgreiche Rotation) reduzieren, ohne die *obere* Schranke $O(n^3)$ zu verändern — die in Tabelle 2 berichteten Werte stellen also tendenziell eine *obere Abschätzung* der praktisch erreichbaren Laufzeit des Subgraph Algorithmus dar, während die VF2-Werte bereits eine *ausgereifte, optimierte* Implementierung widerspiegeln. Ein faires Replikat dieses Experiments mit einer entsprechend optimierten Subgraph Algorithmus-Implementierung ist Teil von Abschnitt 9.3.18.

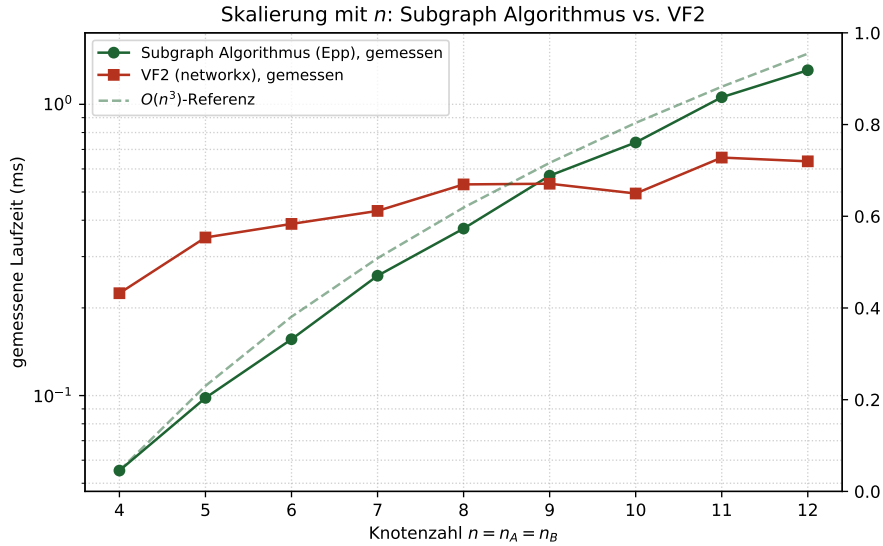


Abbildung 9: Gemessene Laufzeit beider Verfahren bei gemeinsam wachsendem $n = n_A = n_B$ (5 Wiederholungen je n), zusammen mit einer auf den ersten Messpunkt skalierten $O(n^3)$ -Referenzkurve für den Subgraph Algorithmus.

Beschreibung: Die gemessene Laufzeitkurve des Subgraph Algorithmus folgt der eingezeichneten $O(n^3)$ -Referenz erkennbar enger als eine lineare oder quadratische Kurve es täte, was die in Satz 1 bewiesene kubische Schranke empirisch stützt (für $n = 4 \rightarrow 12$, also Faktor 3, wächst T_{Sub} von 0,055 ms auf 1,309 ms, also um den Faktor $\approx 23,8$; $3^3 = 27$ liegt in derselben Größenordnung, die Abweichung erklärt sich durch den additiven $O(n^2)$ -Signaturanteil, der bei kleinem n relativ stärker wiegt). Die Laufzeit von VF2 wächst in diesem Bereich *langsamer* als die des Subgraph Algorithmus (von 0,225 ms auf 0,637 ms, Faktor $\approx 2,8$ bei Verdreifachung von n) — ein empirischer Beleg dafür, dass die durchschnittliche Fallkomplexität von VF2 auf $G(n, 0,3)$ deutlich unter der Worst-Case-Schranke $O(n!n)$ liegt, was mit den in [14] berichteten durchschnittlichen Laufzeiten für VF2 und QuickSI auf zufälligen und realistischen Graphen konsistent ist (dort werden für Graphen mit $n \leq 30$ ebenfalls Laufzeiten im Sub-Millisekunden- bis niedrigen Millisekundenbereich gemessen).

6.4 Einordnung in effiziente Subgraph-Isomorphie-Verfahren

Tabelle 3 fasst die in dieser Arbeit gemessenen Größenordnungen zusammen mit publizierten Charakterisierungen weiterer etablierter Algorithmen für (Teil-)Graphisomorphie und Subgraph-Suche. Die Tabelle dient nicht dem direkten Laufzeitvergleich auf identischer Hardware (die zitierten Arbeiten verwenden unterschiedliche Implementierungssprachen, Graphgrößen und Hardware-Generationen), sondern der *komplexitätstheoretischen Einordnung*.

Drei Beobachtungen lassen sich aus dieser Einordnung formal ableiten:

1. **Komplexitätsklasse:** Der Subgraph Algorithmus ist das einzige der in Tabelle 3 aufgeführten Verfahren mit einer *unbedingten* polynomiellen Worst-Case-Schranke ($O(n^3)$, Satz 1). Alle übrigen Verfahren lösen das (im Allgemeinen NP-vollständige bzw. GI-vollständige) Subgraph-Isomorphie-Problem und besitzen daher worst-case zumindest superpolynomielle Schranken; ihre praktische Effizienz beruht auf Heuristiken (Pruning, Knotenordnung, Nachbarschaftssignaturen), die im Average Case, aber nicht im Worst Case greifen. Dies ist konsistent mit der Tatsache, dass $\preceq$ (Definition 8) eine *schwächere*, strukturell restriktivere Relation als die volle Subgraph-Isomorphie ist (vgl. Bemerkung zu Definition 9) — die polynomielle Schranke wird also durch eine Einschränkung des Relationsbegriffs erkauft, was formal die in [1] diskutierte SETH-basierte Optimalitätsanalyse motiviert.
2. **Praktische Konsequenz für Satz 2:** Da $T_{\text{Suche}}(N, C, n) = O(\log N + C) + O(C \cdot n^3)$ die Komplexität *eines einzelnen* Subgraph-Vergleichs als $O(n^3)$ in die Pipeline einsetzt, wäre eine Substitution durch VF2, Ullmann, RI oder Quick-SI formal nicht mit demselben Korollar (konstante Suchzeit unabhängig von N) vereinbar, da $O(n^3)$ durch eine worst-case-exponentielle Schranke ersetzt würde. Die empirisch hohe Average-Case-Effizienz von VF2/RI/QuickSI auf realistischen Graphen (Ergebnis 2, sowie [14, 13]) bedeutet zwar, dass eine solche Substitution *im Mittel* ähnlich performant sein könnte, jedoch ohne die formale Garantie aus Satz 2 — ein Datenbanksystem mit harten Latenzanforderungen (SLA) profitiert daher strukturell vom Subgraph Algorithmus.
3. **Komplementarität statt Konkurrenz:** Die in [13, 14] beschriebenen Optimierungen (Knotenordnung nach Seltenheit, Nachbarschaftssignaturen) sind *orthogonal* zum Subgraph Algorithmus und könnten gemäß Abschnitt 9 (z. B. R-Baum-Pre-Filter) auf die *Pre-Filter-Stufe* der Gen-DB-Pipeline angewendet werden, während der eigentliche Vergleich der gefilterten Kandidaten durch den Subgraph Algorithmus mit garantierter $O(n^3)$ -Schranke erfolgt. Insofern stehen die hier verglichenen Verfahren nicht in direkter Konkurrenz, sondern adressieren unterschiedliche Stufen derselben Pipeline (Kandidatenreduktion vs. finaler Strukturvergleich).

7 Experimentelle Untersuchung der Parallelisierung

Die bisherigen Abschnitte haben die Such-Pipeline von Gen-DB formal analysiert (Abschnitt 4), an einer synthetischen Datenbank mit einer Million Netzwerken statistisch charakterisiert (Abschnitt 5) und den Subgraph Algorithmus in Einzelvergleichen gegen VF2 gemessen (Abschnitt 6). Offen geblieben ist die Frage, wie sich die *gesamte* Suche

Tabelle 3: Einordnung des Subgraph Algorithmus relativ zu etablierten Verfahren.

Verfahren	Worst-Case-Komplexität	Entschiedene Relation	Charakteristik
Subgraph Algorithmus [1]	$O(n^3)$, bedingt optimal unter SE·TH [2]	zyklische LCS-basierte Subsequenz-Relation $\preceq$	polynomiell, eingebeunabhängige Schranke, geringe Varianz (Tabelle 2)
Ullmann [11]	$O(n^n)$ bzw. $O(n_A! \cdot n_B^{n_A})$ (naiv, mit Pruning praktisch geringer)	(Sub-)Graphisomorphie, Knotenbijektion ϕ	historischer Referenzalgorithmus, Backtracking mit Refinement
VF2 [10]	$O(n! \cdot n)$ Worst Case, oft sub-exponentiell in der Praxis	(Sub-)Graphisomorphie	Lookahead-Pruning (Schemata 0, 1, 2); in dieser Arbeit gemessen, vgl. Tab. 2
RI / RI-DS [13]	worst-case exponentiell, empirisch deutlich schneller als VF2 auf Bioinformatik-Graphen	(Sub-)Graphisomorphie	für biologische Netzwerke (PPI, Metabolismus) optimierte Knotenordnung
QuickSI [14]	worst-case exponentiell	Subgraph-Isomorphie (Pattern Matching)	Häufigkeits-basierte Kantenordnung, in [14] ca. $10-1000\times$ schneller als naives Backtracking auf realen Graphdatenbanken
GraphQL [14]	worst-case exponentiell	Subgraph-Containment-Suche	nutzt Nachbarschafts-Signaturen zum Pruning, konzeptuell verwandt mit Pre-Filter aus Abschnitt 4.2

verhält, wenn der dominierende Kostenterm $O(C \cdot n^3)$ aus Satz 2 auf mehrere Prozesse verteilt wird. Dieser Abschnitt beantwortet sie empirisch: Er beschreibt die in Gen-DB implementierte parallele Ausführung der Kandidatenvergleiche, den Aufbau eines reproduzierbaren Experiments mit 1, 2 und 3 Worker-Prozessen und wertet die Messergebnisse anhand von vier Diagrammen und zwei Tabellen aus.

7.1 Fragestellung und Hypothesen

Nach Satz 2 setzt sich die Suchzeit aus dem Pre-Filter $O(\log N + C)$ und den C strukturellen Vergleichen $O(C \cdot n^3)$ zusammen. Da die Vergleiche für verschiedene Kandidaten voneinander unabhängig sind, lässt sich die Laufzeit einer Parallelisierung über p Worker-Prozesse durch

$$T_{\text{parallel}}(C, n, p) = O\left(\frac{C}{p} \cdot n^3\right) + O(n^3)$$

modellieren. Das Experiment prüft drei daraus abgeleitete Hypothesen:

- H1** Die Gesamtzeit einer Suche sinkt bei Verdopplung der Worker-Zahl von 1 auf 2 annähernd um den Faktor 2, weil der parallelisierbare Vergleichsanteil die Laufzeit dominiert.
- H2** Mehr Worker-Prozesse als logische Prozessoren der Maschine bringen keinen weiteren Gewinn, da die Vergleiche CPU-gebunden sind und keine zusätzliche Rechenkapazität entsteht.
- H3** Die Suchzeit wächst, unabhängig von der Worker-Zahl, annähernd linear mit der Kandidatenzahl C , wie es der Term $O(C \cdot n^3)$ bei beschränktem $n \leq 20$ erwarten lässt.

7.2 Implementierung der parallelen Suche

Die Suche ist in der Funktion `search_subgraph` des Moduls `crud` umgesetzt und läuft in drei Phasen. Zuerst liest eine einzige SQL-Abfrage alle Netzwerke, die die notwendigen Bedingungen $n_k \geq n_Q$ und $m_k \geq m_Q$ des Pre-Filters (Abschnitt 4.2) erfüllen, zusammen mit ihren Adjazenzmatrizen. Danach übergibt `compare_many` aus dem Modul `subgraph_executor` die Matrizen an einen `ProcessPoolExecutor`, dessen Größe über die Konfigurationsvariable `SUBGRAPH_MAX_WORKERS` bestimmt wird. Jeder Worker-Prozess erzeugt genau eine Instanz des Subgraph Algorithmus und ruft für ein Paar aus Query A und Kandidat B die Methode `compare_graphs(A, B)` auf. Weil der Algorithmus in reinem Python implementiert und CPU-gebunden ist, werden Prozesse statt Threads verwendet; der Global Interpreter Lock würde Threads sonst serialisieren. Um den Kommunikationsaufwand zwischen Hauptprozess und Workern zu verringern, werden die Kandidaten in Blöcken (*Chunks*) von 256 Vergleichen pro Task verteilt. Schließlich baut der Hauptprozess aus den Entscheidungen `keep_B` und `equal_keep_*` die Trefferliste auf.

Zwei Eigenschaften dieser Implementierung sind für die Auswertung wichtig. Erstens enthält die aktuelle Fassung *kein* Kandidatenlimit K ; alle Kandidaten des Pre-Filters werden verglichen, sodass Satz 3 mit $K = \infty$ gilt und die gemessenen Zeiten einer vollständigen Suche entsprechen. Zweitens sind nur die Vergleiche parallelisiert. Das

Lesen der Kandidaten aus PostgreSQL, die Übertragung der Matrizen an die Worker und der Aufbau der Trefferliste laufen im Hauptprozess und bilden den seriellen Anteil der Suche.

7.3 Versuchsaufbau

Das Experiment wird durch das Skript `src/experiment_search_workers.py` gesteuert. Es wählt mit festem Seed zufällig Netzwerke aus der Datenbank und verwendet sie als Suchanfragen. Um Verzerrungen durch zu kleine Muster zu vermeiden, besitzen alle Queries mindestens 15 Knoten. Für jede Worker-Anzahl werden *dieselben* Anfragen in derselben Reihenfolge über `crud.search_subgraph` ausgeführt, sodass die Zeiten als gepaarte Messungen direkt vergleichbar sind. Vor jeder Messreihe wird der Prozess-Pool neu erzeugt und alle Worker werden vorab gestartet, damit die Startzeit der Prozesse nicht in die Messung eingeht. Eine zusätzliche Aufwärmanfrage pro Worker-Anzahl wird verworfen, um Caching-Effekte der Datenbank auszugleichen. Gemessen wird die Wanduhrzeit der gesamten Funktion mit `time.perf_counter()`; sie umfasst somit Datenbankabfrage, Vergleiche, Prozesskommunikation und Aufbau der Trefferliste. Nach jeder Anfrage schreibt das Skript die bisherigen Messwerte in eine JSON-Datei, sodass ein Abbruch keine Daten verliert.

Tabelle 4: Parameter und Umgebung des Experiments.

Parameter	Wert
Datenbank	1.000.002 Netzwerke (davon 333.500 mit $n_k \geq 15$)
Anzahl Anfragen	6 je Worker-Anzahl, identisch über alle Reihen
Query-Auswahl	zufällig aus der Datenbank, $n_Q \geq 15$, Seed 42
Worker-Anzahlen	$p \in \{1, 2, 3\}$
Aufwärmanfragen	1 je Worker-Anzahl (verworfen)
Chunk-Größe	256 Vergleiche pro Task
Kandidatenlimit	keines
Rechner	2 logische Prozessoren, PostgreSQL 16 auf demselben System
Software	Python 3.13, Linux

Die ursprüngliche Voreinstellung des Skripts sieht 50 Anfragen je Worker-Anzahl vor. Da eine einzelne Suche auf dem verwendeten Rechner bis zu rund 400 Sekunden dauert, hätte diese Konfiguration mehrere Stunden benötigt; die Messreihe wurde deshalb auf 6 Anfragen begrenzt (Aufruf: `python src/experiment_search_workers.py --queries 6 --workers 1 2 3`). Die Folgen dieser Entscheidung für die Aussagekraft werden in Abschnitt 7.7 diskutiert.

7.4 Metriken

Für eine Worker-Anzahl p bezeichne $T(p)$ die Summe der Antwortzeiten aller $Q = 6$ Anfragen und $C_\Sigma = \sum_q C_q$ die Summe der Kandidatenzahlen.

Definition 10 (Speedup, Effizienz, Durchsatz). Der *Speedup* ist $S(p) = T(1)/T(p)$, die *parallele Effizienz* $E(p) = S(p)/p$. Der *Durchsatz* ist $\Theta(p) = C_\Sigma/T(p)$ in Kandidaten pro Sekunde, und die *Kosten pro Kandidat* sind $\kappa(p) = T(p)/C_\Sigma$.

Aus dem Speedup lässt sich mit dem Gesetz von Amdahl der nicht parallelisierte Zeitanteil f abschätzen. Gilt $T(p) = T(1)(f + \frac{1-f}{p})$, so folgt

$$f = \frac{1/S(p) - 1/p}{1 - 1/p}.$$

Diese Abschätzung ist nur sinnvoll, solange p die Zahl der verfügbaren Prozessoren nicht übersteigt.

7.5 Ergebnisse

7.5.1 Die untersuchten Anfragen

Tabelle 5 zeigt die sechs ausgewählten Anfragen. Die Kandidatenzahl C schwankt um mehr als den Faktor 5, zwischen 62.064 und 325.964. Sie ergibt sich aus der Vorauswahl: Bei gleichverteilter Knotenzahl $n_k \in \{3, \dots, 20\}$ besitzen 333.500 der 1.000.002 Netzwerke mindestens 15 Knoten, und für Queries mit 15 Knoten entscheidet nur noch die Kantenbedingung $m_k \geq m_Q$ über den Rest. Bei der Anfrage 5 mit $m_Q = 56$ Kanten erfüllen 97,7% dieser Netzwerke die Kantenbedingung, bei der Anfrage 1 mit $m_Q = 68$ noch 82,6%. Die Anfragen 2 und 6 mit 19 Knoten werden zusätzlich von der Knotenbedingung stark eingeschränkt, da nur Netzwerke mit 19 oder 20 Knoten in Frage kommen. Der Pre-Filter reduziert den Suchraum damit auf 6,2% bis 32,6% der Datenbank.

Die Trefferzahlen sind mit 1 bis 7 minimal. Da jede Query selbst ein Netzwerk der Datenbank ist und die Kandidatenmenge dieses Netzwerk enthält, ist zu erwarten, dass der einzelne Treffer der Anfragen mit genau einem Ergebnis dem Query-Netzwerk selbst entspricht. Netzwerke mit mindestens 15 Knoten und der Kantendichte $p = 0,3$ sind also in anderen Zufallsnetzwerken praktisch nie als Teilstruktur enthalten. Dieser Befund steht im Gegensatz zum Serverprotokoll in Abschnitt 4.6, bei dem nahezu alle Kandidaten Treffer waren; die Trefferquote hängt, wie dort bereits vermerkt, stark von der konkreten Query ab. Für die Laufzeitmessung ist das hier günstig, denn der Aufbau der Trefferliste im Hauptprozess ist vernachlässigbar, und die gemessene Zeit wird fast vollständig von Vorauswahl und Vergleichen bestimmt.

Tabelle 5: Eigenschaften der sechs Suchanfragen und Suchzeiten t_p in Sekunden bei p Workern.

Nr.	Netzwerk-ID	n_Q	m_Q	C	Treffer	t_1	t_2	t_3	$S(2)$
1	654454	15	68	275.420	1	360,8	193,2	198,4	1,87
2	773650	19	107	62.064	1	70,0	36,7	40,6	1,91
3	432492	16	77	221.027	1	294,9	153,0	169,4	1,93
4	858614	15	70	263.668	3	354,9	185,8	209,2	1,91
5	438514	15	56	325.964	7	403,2	212,0	224,6	1,90
6	88827	19	95	100.911	1	120,4	62,4	65,4	1,93

7.5.2 Gesamtzeit und Speedup

Abbildung 10 zeigt als Balkendiagramm die Summe der Antwortzeiten aller sechs Anfragen in Minuten, getrennt nach Worker-Anzahl. Über jedem Balken steht der Speedup

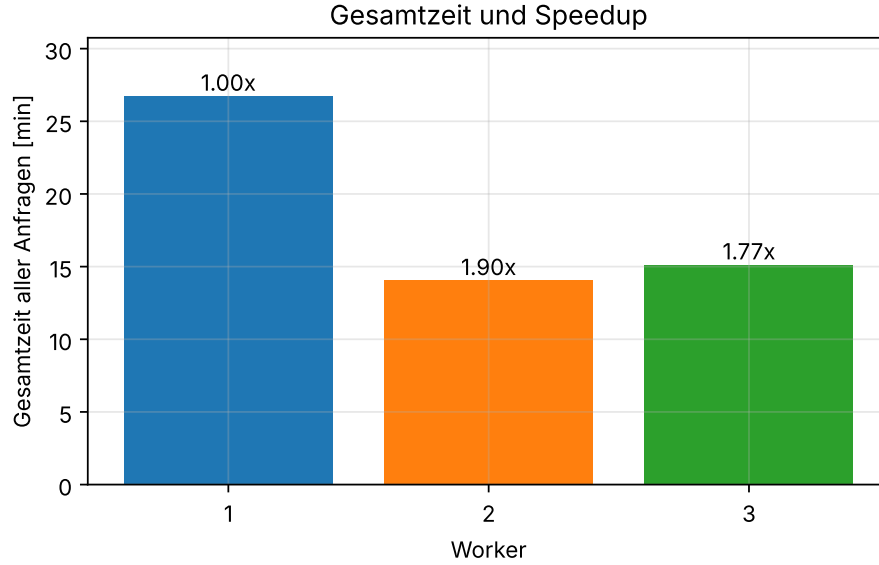


Abbildung 10: Gesamtzeit aller sechs Anfragen in Minuten für 1, 2 und 3 Worker mit dem jeweiligen Speedup über den Balken.

relativ zur Messreihe mit einem Worker. Mit einem Worker benötigt die Suche zusammen 1604 Sekunden, das sind rund 26,7 Minuten. Mit zwei Workern sinkt dieser Wert auf 843 Sekunden (rund 14,1 Minuten), was einem Speedup von $S(2) = 1,90$ und einer Effizienz von $E(2) = 0,95$ entspricht. Der dritte Balken liegt bei 908 Sekunden (rund 15,1 Minuten) und damit *über* dem Balken für zwei Worker; der Speedup fällt auf $S(3) = 1,77$, die nominale Effizienz auf $E(3) = 0,59$. Das Diagramm macht zwei Dinge sichtbar: den nahezu idealen Gewinn beim Übergang von einem auf zwei Worker und die leichte Verschlechterung durch einen dritten Worker, obwohl dessen Hinzunahme in einer idealen parallelen Maschine einen weiteren Gewinn bringen müsste.

Tabelle 6 fasst die Kennzahlen zusammen. Der Durchsatz steigt von 779 auf 1482 Kandidaten pro Sekunde und fällt mit drei Workern auf 1376 zurück. Entsprechend sinken die Kosten pro Kandidat von 1,28 auf 0,68 Millisekunden und steigen mit drei Workern wieder auf 0,73 Millisekunden. Aus $S(2) = 1,90$ ergibt sich nach der Formel aus Definition 10 ein nicht parallelisierter Anteil von etwa $f \approx 5\%$. Dieser Wert fasst alle seriellen Anteile zusammen, darunter das Laden der bis zu 326.000 Zeilen aus PostgreSQL, die Serialisierung der Matrizen für die Prozesskommunikation und den Aufbau der Trefferliste. Eine getrennte Zuordnung auf die einzelnen Phasen erlauben die vorliegenden Daten nicht, weil die Phasenzeiten im aktuellen Code nicht protokolliert werden.

Tabelle 6: Kennzahlen der Suche in Abhängigkeit von der Worker-Anzahl (jeweils 6 Anfragen, $C_\Sigma = 1.249.054$ Kandidaten).

p	$T(p)$ [s]	Mittel [s]	Median [s]	Max. [s]	$S(p)$	$E(p)$	$\Theta(p)$ [1/s]	$\kappa(p)$ [ms]
1	1604	267,4	324,9	403,2	1,00	1,00	779	1,28
2	843	140,5	169,4	212,0	1,90	0,95	1482	0,68
3	908	151,3	183,9	224,6	1,77	0,59	1376	0,73

7.5.3 Verteilung der Antwortzeiten

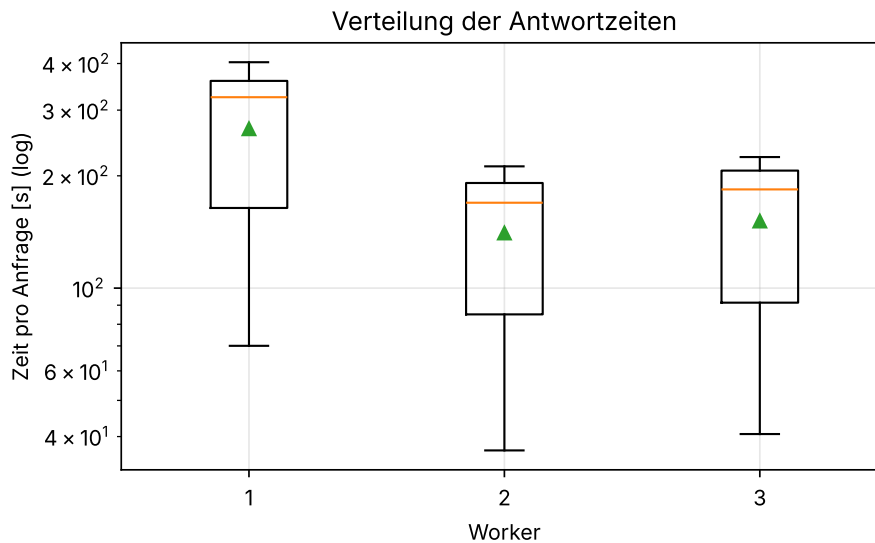


Abbildung 11: Verteilung der Antwortzeit pro Anfrage als Boxplot mit logarithmischer Zeitachse und Mittelwert (Dreieck) für 1, 2 und 3 Worker.

Abbildung 11 stellt die Antwortzeiten der einzelnen Anfragen als Boxplots dar. Die Zeitachse ist logarithmisch, weil die Zeiten über mehr als eine Größenordnung streuen. Die Box umfasst das untere bis obere Quartil, die orange Linie markiert den Median, das grüne Dreieck den Mittelwert, und die Antennen reichen bis zur kleinsten bzw. größten Messung. Mit einem Worker liegen die Zeiten zwischen 70,0 und 403,2 Sekunden; das untere Quartil beträgt rund 164 Sekunden, das obere rund 359 Sekunden, der Median 324,9 Sekunden und der Mittelwert 267,4 Sekunden. Mit zwei Workern verschiebt sich die gesamte Verteilung nach unten: Die Zeiten reichen von 36,7 bis 212,0 Sekunden, die Quartile liegen bei rund 85 und 191 Sekunden, der Median bei 169,4 und der Mittelwert bei 140,5 Sekunden. Auf der logarithmischen Achse ist die Verschiebung für Minimum, Median und Maximum nahezu gleich groß, die Boxen haben also dieselbe Form. Das bedeutet, dass die Parallelisierung alle Anfragen unabhängig von ihrer Größe um etwa denselben Faktor beschleunigt und nicht nur besonders große oder kleine. Mit drei Workern liegt die Verteilung geringfügig oberhalb derjenigen mit zwei Workern (Median 183,9 Sekunden, Maximum 224,6 Sekunden), aber deutlich unterhalb der Verteilung mit einem Worker.

Der Median liegt bei jeder Worker-Anzahl über dem Mittelwert. Die Verteilung ist also linksschief: Wenige kurze Anfragen, nämlich die beiden Queries mit 19 Knoten und vergleichsweise kleiner Kandidatenmenge, ziehen den Mittelwert herunter, während vier Anfragen mit großem Suchraum die Laufzeit bestimmen. Wegen der geringen Stichprobe von nur sechs Werten sind die Quartile grobe Orientierungswerte; das 95-%-Quantil, das das Skript ebenfalls ausgibt, ist bei sechs Werten identisch mit dem Maximum und wird deshalb hier nicht gesondert aufgeführt.

7.5.4 Skalierung mit der Größe des Suchraums

Abbildung 12 trägt für jede Anfrage und jede Worker-Anzahl die Antwortzeit gegen die Zahl der Kandidaten nach der SQL-Vorauswahl auf. Die Punkte jeder Worker-

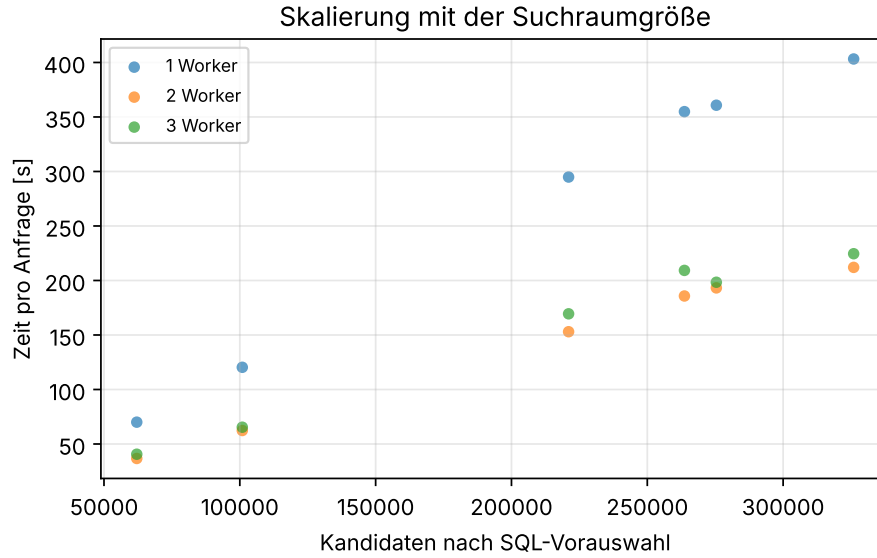


Abbildung 12: Streudiagramm der Antwortzeit gegen die Kandidatenzahl nach der SQL-Vorauswahl für 1, 2 und 3 Worker.

Anzahl liegen sehr nahe an einer Geraden, was Hypothese H3 stützt. Der Pearson-Korrelationskoeffizient zwischen C und Antwortzeit beträgt 0,996 für einen Worker, 0,996 für zwei und 0,990 für drei Worker. Die lineare Regression liefert für einen Worker eine Steigung von etwa 1,3 Millisekunden je Kandidat, für zwei Worker etwa 0,70 und für drei Worker etwa 0,75 Millisekunden. Die Gerade für einen Worker verläuft also etwa doppelt so steil wie die Geraden für zwei und drei Worker, die ihrerseits fast aufeinander liegen, wobei die Punkte für drei Worker durchgehend leicht höher liegen. Die Parallelisierung ändert damit den Anstieg, nicht die Form des Zusammenhangs: Eine Verdopplung der Kandidatenzahl verdoppelt weiterhin die Suchzeit, nur mit etwa der halben Steigung.

Zugleich zeigt das Diagramm, dass der Zusammenhang nicht exakt proportional ist. Die Kosten pro Kandidat (Tabelle 5, aus den Zeiten t_1 berechnet) betragen für die Anfragen mit 15 oder 16 Knoten zwischen 1,24 und 1,35 Millisekunden, für die beiden Anfragen mit 19 Knoten dagegen nur 1,13 und 1,19 Millisekunden. Die Regressionsgerade besitzt entsprechend einen negativen Achsenabschnitt. Aus der Kubikschranke $O(n^3)$ ließe sich für Queries mit mehr Knoten ein höherer, nicht ein niedrigerer Aufwand pro Vergleich erwarten. Die Schranke ist aber eine Worst-Case-Abschätzung für den Vergleich; die Messung zeigt, dass der tatsächliche Aufwand auf diesen Daten auch von anderen Faktoren abhängt, etwa davon, wie früh ein Vergleich entschieden werden kann. Eine belastbare Erklärung erlaubt die vorliegende Messreihe nicht, weil dafür Zeiten einzelner Vergleiche in Abhängigkeit von n_Q und n_k erhoben werden müssten.

7.5.5 Dieselben Anfragen im direkten Vergleich

Abbildung 13 ordnet die Anfragen auf der horizontalen Achse nach ihrer Laufzeit mit einem Worker und zeichnet für jede Worker-Anzahl einen Linienzug durch die zugehörigen Zeiten. Weil dieselben Anfragen in allen drei Messreihen verwendet wurden, zeigt jeder vertikale Schnitt dieselbe Anfrage unter drei Bedingungen. Die Linie für einen Worker liegt an jeder Stelle oberhalb der beiden anderen, die Linien für zwei und drei

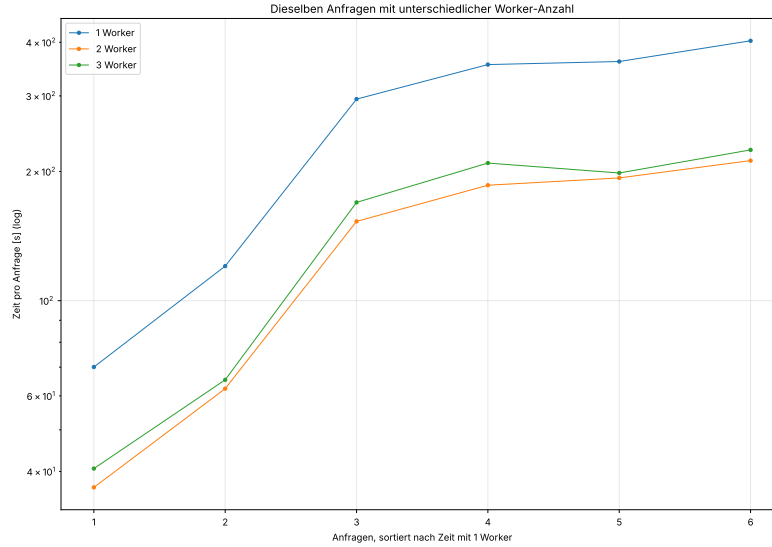


Abbildung 13: Antwortzeit jeder einzelnen Anfrage bei 1, 2 und 3 Workern, sortiert nach der Zeit mit einem Worker, logarithmische Zeitachse.

Worker verlaufen nahezu parallel zueinander, wobei die Linie für drei Worker für alle sechs Anfragen über der für zwei Worker liegt. Auf der logarithmischen Achse entspricht ein konstanter Faktor einem konstanten vertikalen Abstand; dass die Abstände zwischen den Linien über alle Positionen annähernd gleich bleiben, bestätigt, dass die Beschleunigung von der Größe der Anfrage weitgehend unabhängig ist. Der Speedup $S(2)$ liegt für die einzelnen Anfragen zwischen 1,87 und 1,93 (Tabelle 5), der Speedup $S(3)$ zwischen 1,70 und 1,84. Diese enge Streuung ist ein Hinweis, dass der Gesamtwert $S(2) = 1,90$ keine Folge einzelner Ausreißer ist. Besonders deutlich ist der Sprung von der zweiten zur dritten Position: Die Anfrage 3 ist mit 221.027 Kandidaten wesentlich aufwendiger als die Anfrage 6 mit 100.911 Kandidaten, und der Abstand zwischen den Linien bleibt trotzdem derselbe.

7.6 Diskussion

Hypothese H1. Der Übergang von einem auf zwei Worker liefert einen Speedup von 1,90 bei einer Effizienz von 95 %, und alle sechs Anfragen liegen zwischen 1,87 und 1,93. Die Hypothese ist damit auf dem untersuchten Rechner bestätigt: Der parallelisierbare Vergleichsanteil dominiert die Laufzeit, und der serielle Anteil von etwa 5 % (Laden der Kandidaten, Prozesskommunikation, Aufbau der Trefferliste) begrenzt den Gewinn nur geringfügig. Die Messung stützt das in Abschnitt 7.1 angegebene Modell $T_{\text{parallel}} = O(\frac{C}{p}n^3) + O(n^3)$, wobei der additive Term nur den langsamsten Einzelvergleich erfasst. Die Messung zeigt, dass zusätzlich ein mit C wachsender serieller Anteil existiert, der in diesem Modell nicht abgebildet ist. Er fällt bei $p = 2$ kaum ins Gewicht, würde bei größerem p aber nach dem Gesetz von Amdahl zunehmend den erreichbaren Speedup bestimmen.

Hypothese H2. Mit drei Workern auf einem Rechner mit zwei logischen Prozessoren steigt die Gesamtzeit um rund 7,7 % gegenüber zwei Workern, und der Speedup fällt von 1,90 auf 1,77. Auch diese Hypothese wird bestätigt. Die Vergleiche sind rein rechengebunden; ein dritter Prozess erzeugt keine zusätzliche Rechenkapazität, sondern konkurriert

mit den anderen Prozessen um dieselben zwei Prozessoren und verursacht zusätzliche Kontextwechsel. Hinzu kommt, dass auch der Hauptprozess, der die Ergebnisse einsammelt, und der PostgreSQL-Server auf demselben Rechner Rechenzeit benötigen. Welchen Anteil diese Effekte jeweils haben, lässt sich aus den Daten nicht trennen. Die Beobachtung lässt sich aber unmittelbar in eine Konfigurationsempfehlung übersetzen: Die Zahl der Worker sollte die Zahl der verfügbaren logischen Prozessoren nicht überschreiten. Die mitgelieferte `.env`-Datei setzt `SUBGRAPH_MAX_WORKERS=14`; dieser Wert ist nur auf einem Rechner mit mindestens vierzehn logischen Prozessoren sinnvoll und sollte für andere Systeme angepasst werden.

Hypothese H3. Die Antwortzeit hängt mit einem Korrelationskoeffizienten von mindestens 0,99 linear von der Kandidatenzahl ab. Das stimmt mit dem Term $O(C \cdot n^3)$ aus Satz 2 bei beschränktem n überein. Die Abweichungen in den Kosten pro Kandidat zeigen allerdings, dass die Konstante nicht allein durch die Knotenzahl bestimmt wird (siehe oben).

Bedeutung für das Korollar zur Datenbankgröße. Das Korollar zu Satz 2 behauptet eine von N unabhängige Suchzeit, sofern die Kandidatenzahl durch ein Limit K beschränkt ist. Das Experiment läuft bewusst ohne Limit und zeigt die Kehrseite: Bei $N \approx 10^6$ bleiben je nach Query bis zu 32,6 % der Datenbank als Kandidaten übrig, und die Suchzeit steigt dann proportional zu C auf bis zu 403 Sekunden mit einem und 212 Sekunden mit zwei Workern. Ohne Limit wächst C bei fester Selektivität des Pre-Filters linear mit N , und damit auch die Suchzeit. Die Parallelisierung verkleinert den Anstieg um etwa den Faktor p , hebt die Abhängigkeit von N aber nicht auf. Wer harte Latenzgrenzen einhalten will, braucht deshalb weiterhin einen schärferen Pre-Filter (etwa den in Abschnitt 9 diskutierten mehrdimensionalen Index) oder ein Kandidatenlimit mit den dort beschriebenen Einbußen bei der Vollständigkeit; die Parallelisierung ergänzt diese Maßnahmen, ersetzt sie aber nicht.

7.7 Grenzen der Untersuchung

Die Ergebnisse sind als erste empirische Einordnung zu lesen und aus folgenden Gründen nicht ohne Weiteres verallgemeinerbar.

- **Kleine Stichprobe.** Mit sechs Anfragen und einer Messung je Anfrage und Worker-Anzahl lassen sich keine Standardabweichungen über Wiederholungen und keine Signifikanztests angeben. Dass die Speedups der einzelnen Anfragen eng beieinander liegen, spricht für stabile Verhältnisse, ersetzt aber keine Wiederholungsmessungen. Die ursprünglich vorgesehenen 50 Anfragen sollten auf einem Rechner mit mehr Prozessoren nachgeholt werden.
- **Zwei logische Prozessoren.** Aussagen für $p > 2$ beschränken sich auf die Überbelegung; das Skalierungsverhalten bei vier, acht oder vierzehn echten Prozessoren, insbesondere der Einfluss des seriellen Anteils, wird nicht gemessen. Die Abschätzung $f \approx 5\%$ beruht auf einem einzigen Messpunkt.
- **Gemeinsam genutzter Rechner.** Datenbank, Hauptprozess und Worker teilen sich dieselben Prozessoren. Auf einem System mit separatem Datenbankserver würde sich der serielle Anteil anders verhalten. Die Absolutwerte sind außerdem

nicht mit dem Serverprotokoll aus Abschnitt 4.6 vergleichbar, das auf einem anderen System entstand.

- **Keine Phasenerlegung.** Der Code protokolliert die Zeiten für Laden, Vergleichen und Trefferaufbau nicht, weshalb das Diagramm „Zeit je Phase“ des Skripts leer bleibt und dieser Abschnitt auf seine Wiedergabe verzichtet. Der serielle Anteil kann daher nur indirekt aus dem Speedup abgeschätzt werden.
- **Eine synthetische Graphenfamilie.** Alle Netzwerke stammen aus dem Erdős-Rényi-Modell mit $p = 0,3$; für reale biologische Netzwerke mit anderer Dichte und Struktur können Kandidatenzahlen, Trefferquoten und Kosten pro Vergleich abweichen.
- **Treffer von Query und Kandidat.** Da die Queries aus der Datenbank stammen, enthält jede Kandidatenmenge das Query-Netzwerk selbst. Das ist für die Laufzeitmessung unerheblich, spricht aber gegen eine Interpretation der Trefferzahlen als allgemeine Trefferquote.

7.8 Zwischenfazit

Die parallele Ausführung der Kandidatenvergleiche über einen Prozess-Pool beschleunigt die Suche in Gen-DB auf dem untersuchten Rechner mit zwei logischen Prozessoren um den Faktor 1,90 (Effizienz 95 %); der Gewinn ist für alle sechs Anfragen nahezu gleich. Ein dritter Worker verschlechtert die Zeit um rund 8 %, da keine zusätzliche Rechenkapazität zur Verfügung steht. Die Suchzeit wächst unabhängig von der Worker-Zahl linear mit der Kandidatenzahl und erreicht ohne Kandidatenlimit bei großen Suchräumen mehrere Minuten, sodass die Parallelisierung die Antwortzeit um einen konstanten Faktor senkt, aber die Notwendigkeit eines selektiveren Pre-Filters oder eines Kandidatenlimits nicht beseitigt. Für die Praxis folgt daraus die Empfehlung, die Worker-Anzahl an die Zahl der logischen Prozessoren anzupassen. Die offene Frage, wie weit sich der Gewinn auf Rechnern mit deutlich mehr Prozessoren fortsetzt, soll eine Wiederholung der Messreihe mit der vollen Anfragezahl klären.

8 Experimenteller Vergleich Python und C++

Die vorangegangene Untersuchung hat gezeigt, dass sich die Suchzeit durch mehrere Worker-Prozesse deutlich reduzieren lässt. Sie beantwortet jedoch nicht, ob auch der einzelne Subgraph-Vergleich schneller ausgeführt werden kann. Dieses Kapitel untersucht deshalb die C++17-Implementierung aus dem Projekt `csubgraph`, die als Library direkt im langlebigen Gen-DB-Worker aufgerufen wird. Ein früherer Messlauf über die C++-CLI hatte den Prozessstart und JSON-Austausch als erhebliche Kostenquelle sichtbar gemacht; seine Messwerte sind von den hier ausgewerteten Library-Werten zu trennen.

Das Experiment vergleicht auf zwei Ebenen. Der Mikrobenchmark misst direkte Python-Aufrufe und Funktionsaufrufe der C++-Library auf denselben Graphpaaren sowie die Kosten eines trivialen Library-Aufrufs. Die Ende-zu-Ende-Messung vergleicht die vollständige Gen-DB-Suche für identische Queries, Kandidatenmengen und Worker-Zahlen. So werden sowohl der Effekt auf den Einzelvergleich als auch der praktisch

relevante Einfluss auf die Antwortzeit erfasst. Die abgeschlossene Messreihe vom 7. Oktober 2026 umfasst sechs Suchanfragen; beide Backends lieferten identische Treffer-IDs und es traten keine fehlgeschlagenen Vergleiche auf.

8.1 Motivation und Fragestellung

Der Algorithmus bildet den rechenintensiven Teil der Kandidatenprüfung in Gen-DB. Nach dem datenbankseitigen Pre-Filter wird jedes verbliebene Netzwerk mit der Query verglichen. Bei mehr als einer Million Kandidaten kann selbst eine geringe Zeitersparnis pro Paar einen großen Unterschied für die gesamte Suche bedeuten. Zugleich unterscheidet sich der Aufwand eines Algorithmusaufrufs von dem Aufwand, den die umgebende Anwendung tatsächlich misst: Daten müssen in das jeweilige Format überführt, an den ausführenden Prozess übergeben und das Ergebnis muss zurücktransportiert und interpretiert werden.

Das Experiment untersucht daher folgende Fragen:

1. Wie unterscheiden sich direkte Aufrufe der Python-Implementierung und der C++-Library für dieselben Graphpaare?
2. Wie groß sind die Übergabekosten der Library im Verhältnis zur Laufzeit eines trivialen Vergleichs?
3. Führt der Einsatz der C++-Library bei identischen Queries, Kandidaten und Worker-Zahlen zu einer kürzeren Ende-zu-Ende-Suchzeit?
4. Liefern beide Backends für dieselben Queries dieselben Treffer-IDs, und treten bei einem Backend fehlgeschlagene Einzelvergleiche auf?

Daraus ergeben sich vier Hypothesen. **H1:** Ein direkter Aufruf des nativen Algorithmuskerns benötigt weniger Zeit als ein Python-Aufruf. **H2:** Der Vorteil bleibt nach Einbeziehung der Library-Schnittstelle groß genug, um die vollständige Gen-DB-Suche zu beschleunigen. **H3:** Beide Implementierungen liefern für alle Queries dieselben Suchtreffer. **H4:** Die Kosten der Library-Schnittstelle sind im Verhältnis zur Rechenzeit klein, während ein Prozess-pro-Vergleich-CLI-Aufruf erheblichen Zusatzaufwand verursachen kann. Der abgeschlossene Mikrobenchmark und Suchlauf liefern für H1 bis H3 direkte Messwerte; H4 wird durch die Library-Aufrufkosten und den separat dokumentierten früheren CLI-Mikrobenchmark eingeordnet.

8.2 Implementierungen und Ausführungspfade

Beide Backends führen denselben Subgraph Algorithmus aus. Wie in Abschnitt 2 beschrieben, vergleicht dieser die aus den Adjazenzmatrizen abgeleiteten Zeilenkomponenten unter zyklischen Rotationen und entscheidet anhand einer dynamischen Übereinstimmungsprüfung über die Relation der Graphen. Die Python-Implementierung wird über `Subgraph().compare_graphs(A, B)` aufgerufen. Für C++ wird die statische Bibliothek `libsubgraphlib.a` mit einem C-Wrapper zu einer DLL gelinkt und über `ctypes` angesprochen. Die DLL wird einmal pro Worker-Prozess geladen; ein Vergleich erfolgt danach als direkter Funktionsaufruf.

Beide Varianten laufen in einem langlebigen `ProcessPoolExecutor` mit zwei Workern und erhalten Kandidaten blockweise mit Chunk-Größe 256. Anders als bei der

früheren CLI-Integration wird für C++ weder je Kandidat ein weiterer Prozess gestartet noch werden Matrizen und Resultate über JSON-Pipes serialisiert. Der einmalige Bau und das Linken des Wrappers finden vor dem Start des Pools statt und sind nicht Teil der gemessenen Suchzeit. Der Mikrobenchmark misst den direkten Funktionsspfad einschließlich der Übergabe der Arrays an den Wrapper und vergleicht somit reale Aufrufpfade, nicht abstrakt bereinigte Sprachlaufzeiten.

Das Experiment kontrolliert, dass in Haupt- und Worker-Prozessen tatsächlich das jeweils angeforderte Backend aktiv ist, zählt fehlgeschlagene Vergleiche und gleicht für jede Anfrage die gefundenen Netzwerk-IDs ab. Damit lässt sich neben der Laufzeit auch die Ergebniskonformität der beiden Implementierungen beurteilen.

8.3 Versuchsaufbau

Das Experiment wird durch `src/experiment_search_backends.py` gesteuert. Es verwendet dieselbe Datenbank für beide Implementierungen, wählt mit Seed 42 sechs Netzwerke als Queries und setzt die Untergrenze auf 15 Knoten. Beide Backends werden mit zwei Workern und in derselben Reihenfolge ausgewählten Queries ausgeführt. Vor jeder Messreihe wird eine Aufwärmanfrage verworfen. Die JSON-Datei wird nach jeder abgeschlossenen Anfrage aktualisiert, damit ein Abbruch die bis dahin geschriebenen Messwerte nicht verliert. Zusätzlich prüft das Skript die Treffer-IDs, kontrolliert die tatsächlich in den Worker-Prozessen aktive Implementierung und zählt fehlgeschlagene Einzelvergleiche.

Der Mikrobenchmark wird ohne Datenbank ausgeführt. Mit Seed 42 werden 200 Zufallspaare erzeugt. Die erste Matrix besitzt zwischen 15 und 19 Knoten, die zweite 20 Knoten; jede mögliche gerichtete Kante wird mit Wahrscheinlichkeit 0,3 gewählt. Beide Backends verarbeiten dieselben Paare. Für C++ wird außerdem der direkte Bibliotheksaufruf mit zwei trivialen 1×1 -Matrizen gemessen. Dieser Wert nähert die Kosten von Array-Marshalling und Funktionsübergabe an, nicht jedoch alle denkbaren Aufwände größerer Matrizen.

Tabelle 7: Versuchsparameter und Umgebung des Backend-Vergleichs.

Parameter	Wert
Backend-Reihenfolge	Python, anschließend C++-Library
Worker je Backend	2
Queries	6, zufällig aus der Datenbank, Seed 42
Mindestgröße der Queries	15 Knoten
Aufwärmung	eine verworfene Suche je Backend
Mikrobenchmark	200 Zufallspaare, Seed 42
Knotenzahl Mikrobenchmark	$n_A \in \{15, \dots, 19\}$, $n_B = 20$
Kantenwahrscheinlichkeit	0,3
Library-Referenz	200 Aufrufe mit trivialen 1×1 -Matrizen
Betriebssystem	Windows 11
Python	3.14.8
Logische Prozessoren	4
C++-Bibliothek	<code>libsubgraphlib.a</code> , Wrapper-DLL über <code>ctypes</code>

Die volle Suche misst mit `time.perf_counter()` die Wanduhrzeit von `crud.search_subgraph()`. Sie umfasst damit die Datenbankabfrage, die Vergleich-

sphase, die Kommunikation mit den Worker-Prozessen und den Aufbau der Trefferliste. Zusätzlich werden die in `compare_many` verbrauchte Zeit und der als Differenz berechnete Rest der Suche protokolliert. Für jede Query werden Knotenzahl, Kantenzahl, Zahl der Kandidaten nach SQL-Vorauswahl, Trefferzahl und Fehlerzahl festgehalten. Die Summe der Antwortzeiten je Backend erlaubt den direkten Gesamtvergleich; die jeweiligen Treffer-IDs dienen dem Abgleich der funktionalen Ergebnisse.

8.4 Metriken

Für dieselbe Query q bezeichne $t_{\text{Py},q}$ die gemessene Suchzeit mit der Python-Implementierung und $t_{\text{C++},q}$ die Zeit mit der C++-Library. Der relative Such-Speedup ist

$$S_{\text{C++}} = \frac{\sum_q t_{\text{Py},q}}{\sum_q t_{\text{C++},q}}.$$

Ein Wert größer als 1 bedeutet, dass die C++-Library in der gemessenen Gen-DB-Ausführung schneller war; ein Wert kleiner als 1 bedeutet eine langsamere Suche. Der Quotient vergleicht die gesamte Integration und nicht allein den nativen Algorithmenkern. Ergänzend misst das Skript den Kandidatendurchsatz $\Theta = C_\Sigma/T$ und die Kosten pro Kandidat $\kappa = T/C_\Sigma$, wobei C_Σ die Summe der Kandidatenzahlen und T die Summe der Antwortzeiten eines Backends ist.

Die funktionale Übereinstimmung wird pro Query über die sortierten Listen der gefundenen Netzwerk-IDs geprüft. Zusätzlich vergleicht das Skript die erwartete Backend-Kennung in Haupt- und Worker-Prozessen und protokolliert fehlgeschlagene Vergleiche. Dadurch ist ein stiller Rückfall auf Python nicht mit einem erfolgreichen C++-Lauf zu verwechseln. Der Mikrobenchmark beantwortet eine andere Frage als der Such-Speedup: Er isoliert den wiederholten direkten Funktionsaufruf im Hauptprozess; die Ende-zu-Ende-Messung umfasst zusätzlich Datenbankzugriff, Worker-Kommunikation und Ergebnisaufbau.

8.5 Ergebnisse der abgeschlossenen Messreihe

Die ausgewertete Datei `search_backends_20261007_104804.json` ist mit `complete=true` markiert. Sechs Queries mit mindestens 15 Knoten wurden bei zwei Workern in beiden Backends ausgeführt; zusammen durchlief jede Implementierung exakt 931.899 Kandidaten. Tabelle 8 fasst die Such- und Kalibrierungskennzahlen zusammen. Die Laufzeiten pro Query in Tabelle 10 zeigen ergänzend, dass der beobachtete Gewinn nicht von einer einzelnen Anfrage getragen wird.

Tabelle 8: Kennzahlen der Suche je Implementierung (jeweils 6 Anfragen, $C_\Sigma = 931.899$ Kandidaten, Speedup S relativ zu Python).

Variante	p	T [s]	Mittel [s]	Median [s]	Max. [s]	S	Θ [1/s]	κ [ms]	seriell [%]
Python	2	1.180	196,7	145,3	379,2	1,00	790	1,267	11,6
C++	2	186	30,9	26,2	63,6	6,36	5.020	0,199	66,7

Die Python-Suche benötigte insgesamt 1.180,28 Sekunden, also 19 Minuten und 40 Sekunden; die C++-Library benötigte 185,62 Sekunden, rund 3 Minuten und 6 Sekunden. Der aggregierte Speedup beträgt damit $6,36\times$ und die Suchzeit sinkt um etwa

Tabelle 9: Zeit je Einzelvergleich im Hauptprozess (200 Zufallspaare mit $n_A \in \{15, \dots, 19\}$, $n_B = 20$, Kantenwahrscheinlichkeit 0,3).

Messung	Median [ms]	Mittel [ms]	Max. [ms]
Python, Einzelvergleich	2,793	2,694	3,703
C++ (Bibliothek), Einzelvergleich	0,068	0,074	0,331
C++ (Bibliothek), trivialer 1×1 -Aufruf	0,018	0,018	0,073

Tabelle 10: Antwortzeiten und Ergebnisse der sechs identischen Suchanfragen mit jeweils zwei Workern.

Nr.	Netzwerk-ID	n_Q	m_Q	Kandidaten	Treffer	Python [s]	C++ [s]	Speedup
1	654.525	18	106	69.173	1	97,39	11,11	8,76×
2	773.692	19	104	74.482	1	85,22	12,33	6,91×
3	432.763	15	63	302.555	1	379,18	51,08	7,42×
4	858.300	20	110	38.496	1	50,66	7,31	6,93×
5	438.602	15	66	287.228	1	374,62	63,62	5,89×
6	89.411	18	83	159.965	1	193,21	40,16	4,81×

84,3 %. Der Geschwindigkeitsgewinn ist für jede der sechs Einzelanfragen sichtbar: Die individuellen Speedups reichen von $4,81\times$ bis $8,76\times$. Ihre Streuung macht zugleich deutlich, dass ein einzelner Gesamtfaktor keine für jede Query identische Beschleunigung verspricht.

Die Query mit den meisten Kandidaten (302.555) benötigt mit Python 379,18 Sekunden und mit C++ 51,08 Sekunden; die zweite Anfrage mit annähernd ebenso vielen Kandidaten zeigt mit 287.228 Kandidaten und 374,62 beziehungsweise 63,62 Sekunden ein ähnliches, aber etwas kleineres Verhältnis. Die Query mit den wenigsten Kandidaten (38.496) ist in beiden Backends die schnellste. Die Messreihe passt damit zur erwarteten Abhängigkeit von der Größe des Suchraums. Wegen nur sechs Queries und variierender Knotenzahlen ist sie jedoch keine kontrollierte Regression, aus der eine allgemeingültige Zeit pro Kandidat abgeleitet werden könnte.

Abbildung 14 stellt die aufsummierte Suchzeit in Minuten dar; die Beschriftungen über den Balken geben den Speedup relativ zur Python-Referenz an. Die Differenz ist groß genug, dass sie nicht durch die Wahl einer einzelnen Aggregationskennzahl erklärt wird: Verglichen werden die Summe derselben sechs Antwortzeiten, dieselben Kandidaten und dieselbe Worker-Zahl. Die Grafik belegt den Anwendungseffekt dieser konkreten Library-Integration auf dieser Hardware, nicht eine universelle Beschleunigung für alle Datenbanken oder Query-Arten.

Abbildung 15 verwendet eine logarithmische Zeitachse und zeigt Median, Quartile, Spannweite und Mittelwert der Antwortzeiten. Die beiden Verteilungen liegen klar getrennt: Die mittlere Antwortzeit sinkt von 196,7 auf 30,9 Sekunden, der Median von 145,3 auf 26,2 Sekunden. Die logarithmische Darstellung macht sowohl die Beschleunigung als auch die Unterschiede zwischen kleinen und großen Suchräumen lesbar; die sechs Beobachtungen pro Backend erlauben allerdings keine robuste Aussage über die Verteilung zukünftiger Anfragen.

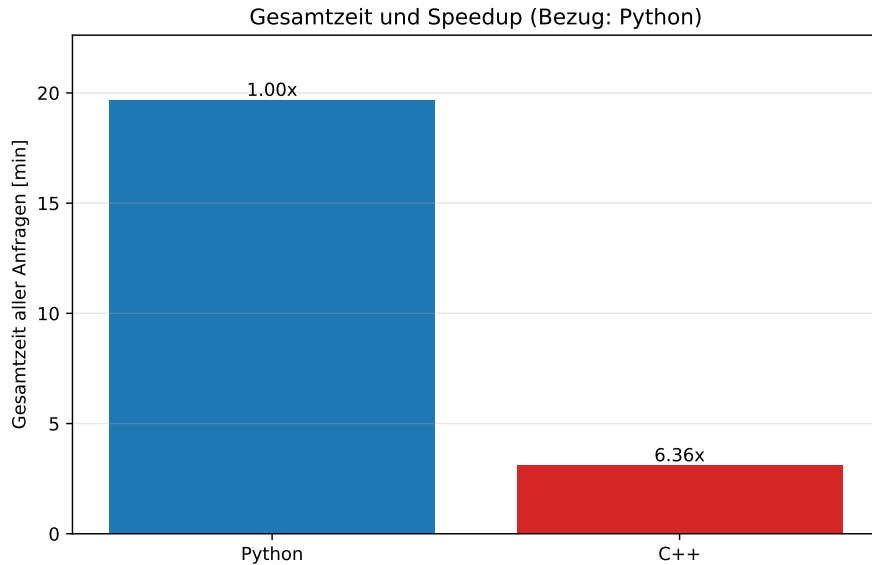


Abbildung 14: Die Gesamtlauzeit der sechs Suchen sinkt mit der C++-Library gegenüber Python auf rund ein Sechstel.

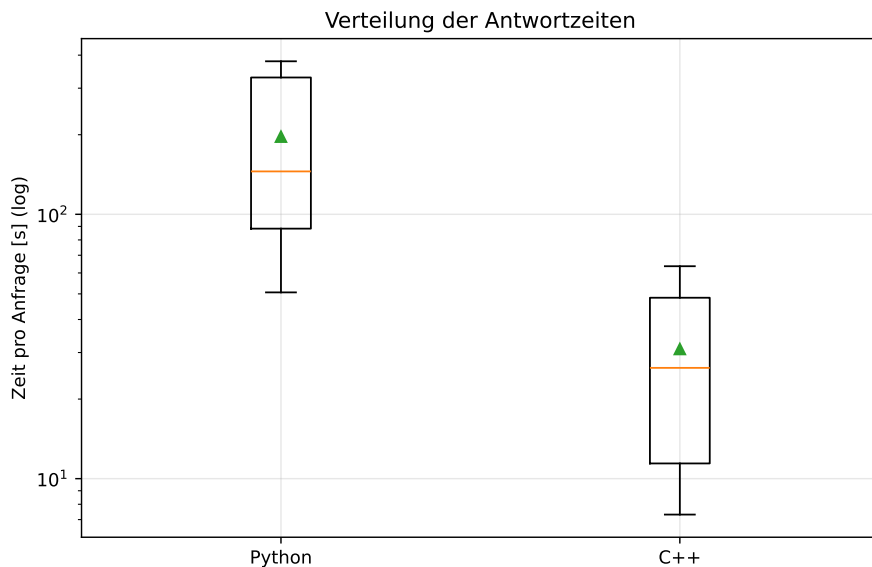


Abbildung 15: Die Verteilung der sechs Antwortzeiten zeigt für beide Backends eine deutliche Verkürzung bei C++.

8.6 Phasenanalyse und Skalierung mit der Kandidatenzahl

Abbildung 16 stellt die mittlere Antwortzeit je Anfrage als gestapelte Vergleichsphase und Restzeit dar. Die Python-Vergleichsphase beansprucht aggregiert 88,4 % der Gesamtlauzeit; bei C++ sind es nur 33,3 %. Entsprechend sinkt die Vergleichszeit über die sechs Anfragen von insgesamt rund 1.043 Sekunden auf rund 61,9 Sekunden, während die gemessene Restzeit von etwa 137 auf 123,7 Sekunden zurückgeht. Der Rest umfasst Laden, Datenübertragung beziehungsweise Aufgabenverteilung und Ergebnisaufbau und ist nicht mit einer reinen Datenbankzeit gleichzusetzen. Da der Rechenkern nun erheblich schneller ist, wird dieser Anteil relativ dominant: Beschleunigungen weiterer

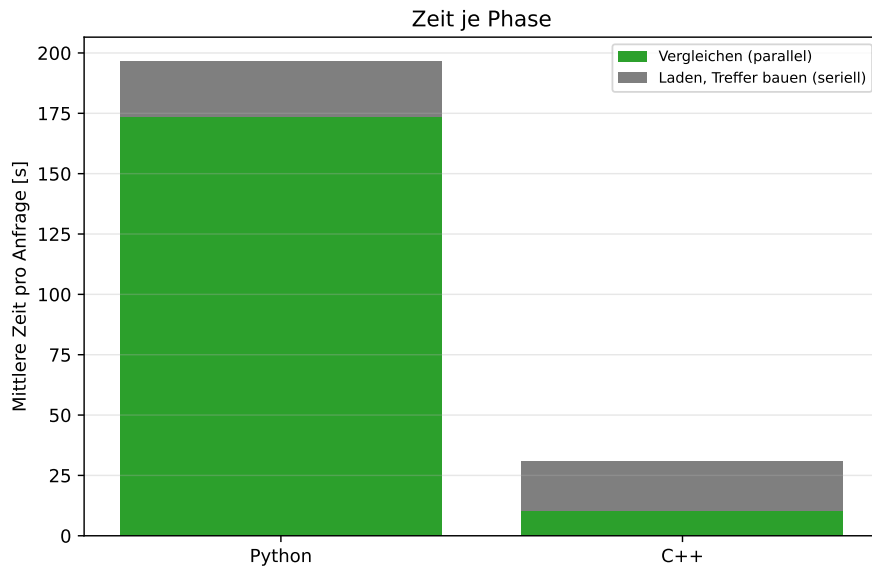


Abbildung 16: Die Phasenmessung zeigt, wie sich Vergleichszeit und übrige Suchzeit zwischen den Backends verschieben.

Vergleichsoperationen allein haben abnehmenden Einfluss auf die gesamte Antwortzeit. Strömendes Laden, effizientere Aufgabenübergabe und Phaseninstrumentierung werden dadurch als nächste Optimierungsziele relevanter.

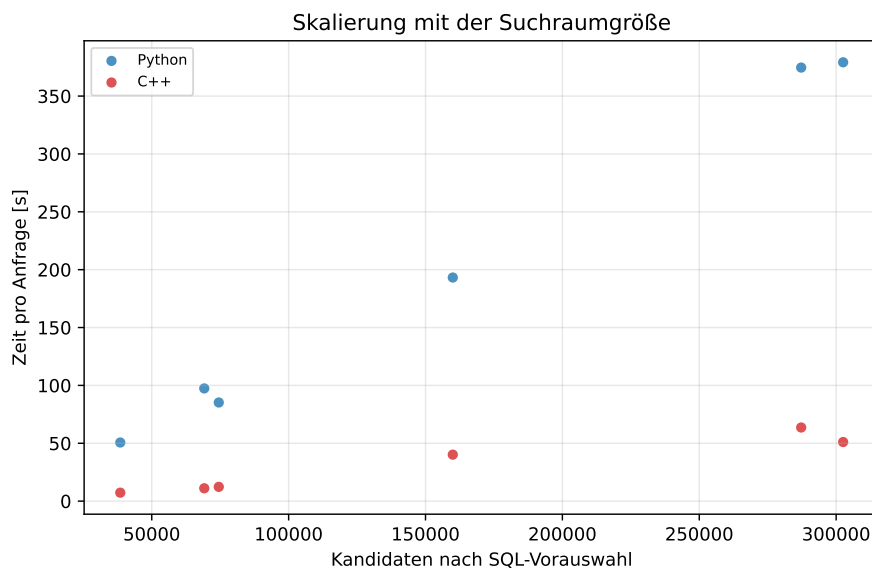


Abbildung 17: Die Antwortzeit nimmt für beide Backends mit der nach der SQL-Vorauswahl verbleibenden Kandidatenzahl zu.

Abbildung 17 ordnet jede Anfrage nach der Zahl ihrer Kandidaten und zeigt für Python wie C++ einen klaren Anstieg der Antwortzeit mit wachsendem Suchraum. Die Punkte bilden keine streng proportionale Linie: Queries unterscheiden sich nicht nur in ihrer Kandidatenzahl, sondern auch in Knotenzahl und Struktur, und außerhalb der Vergleichsphase fallen weitere Kosten an. Insgesamt verarbeitet Python rund 790 Kandidaten pro Sekunde, während die Library rund 5.020 Kandidaten pro Sekunde erreicht;

der aggregierte Aufwand sinkt von 1,267 auf 0,199 Millisekunden je Kandidat. Diese Quotienten beschreiben die sechs vollständigen Suchläufe einschließlich ihrer nichtvergleichenden Anteile und sind deshalb keine isolierten Kosten des Algorithmuskerns.

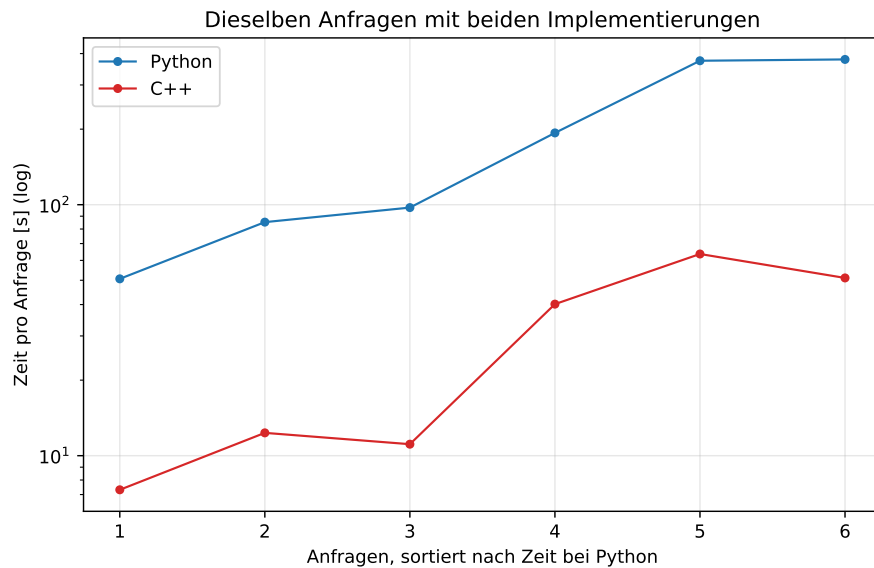


Abbildung 18: Für jede einzelne Query liegt die Antwortzeit der C++-Library unter derjenigen der Python-Implementierung.

Abbildung 18 zeigt gepaart dieselben sechs Queries, nach der Python-Antwortzeit sortiert und auf logarithmischer Skala. Beide Kurven folgen derselben Reihenfolge: Die aufwendigen Queries bleiben auch mit C++ aufwendiger als die kleineren, jedoch liegt jeder C++-Messpunkt deutlich unter seinem Python-Pendant. Dies schließt aus, dass der aggregierte Vorteil allein aus einer anderen Reihenfolge, einem anderen Query-Sample oder weniger Kandidaten im C++-Lauf entsteht.

8.7 Mikrobenchmark, funktionale Kontrolle und Diskussion

Der Mikrobenchmark isoliert die direkten Vergleichsaufrufe auf 200 identischen Zufallspaaren. Tabelle 9 enthält Mittelwert, Median und Maximum. Der C++-Library-Aufruf benötigt im Mittel 0,074 Millisekunden je Paar, der direkte Python-Aufruf 2,694 Millisekunden. Das Verhältnis der Mittelwerte entspricht einer Beschleunigung von ungefähr $36,5\times$; anhand der Mediane ergibt sich etwa $40,8\times$. Die Maximalwerte betragen 0,331 beziehungsweise 3,703 Millisekunden. Damit bestätigt H1 für diesen Benchmark deutlich einen schnelleren direkten C++-Aufruf.

Der triviale 1×1 -Library-Aufruf benötigt im Mittel 0,018 Millisekunden und im Median 0,018 Millisekunden. Das entspricht etwa einem Viertel der mittleren gemessenen C++-Zeit für die größeren Vergleichspaare und zeigt, dass die Funktionsübergabe samt Marshalling nicht null ist, aber weit unter der gemessenen Python-Aufrufzeit liegt. Anders als eine Differenz aus CLI-Messungen ist dieser Wert keine Prozessstartzeit: Er ist ein direkter Aufruf durch die Wrapper-Schnittstelle. Der triviale Aufruf enthält außerdem andere Matrixgrößen als die eigentlichen Paare; er dient daher nur als Größenordnung für die Mindestkosten und nicht als exakt abziehbarer Anteil.

H1 und H2: Beschleunigung des Einzelvergleichs und der Suche. Der Mikrobenchmark belegt den Geschwindigkeitsvorteil des direkten C++-Aufrufpfads in der Stichprobe. Auch die Ende-zu-Ende-Suche ist mit der Library deutlich schneller: Die Gesamtzeit reduziert sich von 1.180,28 auf 185,62 Sekunden. Der Such-Speedup von $6,36\times$ fällt kleiner aus als die etwa $36,5\times$ des Mikrobenchmarks, weil die vollständige Anfrage Datenbankzugriff, Prozesskommunikation und Ergebnisaufbau einschließt und weil die Vergleichsphase nach ihrer Beschleunigung nur noch rund ein Drittel der C++-Gesamtzeit ausmacht. Die beiden Speedup-Werte beschreiben damit unterschiedliche Ebenen und widersprechen einander nicht.

H3: funktionale Übereinstimmung. Für jede Query stimmen die mit Python und C++ gefundenen Netzwerk-IDs überein. Beide Backends liefern je Query einen Treffer, zusammen also sechs Treffer in den sechs jeweiligen Suchläufen. In beiden Reihen beträgt die Fehlerzahl null. Zusätzlich bestätigt die Laufzeitkontrolle, dass die angeforderte Library tatsächlich in den C++-Workern aktiv war. Das ist eine positive differenzielle Prüfung der konkreten sechs Suchläufe, aber noch kein formaler Korrektheitsbeweis für alle Eingaben oder für alle Randfälle des Algorithmus.

H4: Kosten und Grenze der Anbindung. Der Library-Aufruf verursacht messbare Übergabekosten, doch die direkte Mikrobenchmark-Zeit bleibt deutlich unter der Python-Vergleichszeit. Damit zeigen die neuen Resultate den praktisch relevanten Unterschied zum früheren CLI-Pfad: Dessen Prozess-pro-Vergleich- und JSON-Aufwand hatte einen trivialen Aufruf von rund 10 Millisekunden ergeben; dieser Wert gehört ausdrücklich nicht zur hier untersuchten Library. Die Funktionseinbindung vermeidet diesen Prozessstart. Sie beseitigt aber weder alle Python/C++-Übergabekosten noch die nichtvergleichenden Phasen der Suchpipeline.

8.8 Grenzen der Messreihe

Die Ergebnisse sind positiv, müssen aber innerhalb des Versuchsaufbaus interpretiert werden:

- **Sechs Queries und jeweils ein Suchlauf.** Der Speedup beruht auf sechs mit Seed 42 reproduzierbar ausgewählten Datenbank Anfragen und einer Messung je Backend. Es gibt keine Wiederholungen je Query, Konfidenzintervalle oder Signifikanztests; Cache-, System- und Datenbankzustände können einzelne Zeiten beeinflussen.
- **Synthetische Datenbank und begrenzte Querygrößen.** Die Suchreihe stammt aus der synthetischen Gen-DB-Datenbank; die sechs Queries besitzen 15 bis 20 Knoten. Auf realen biologischen Netzwerken, anderen Dichten und größeren Matrizen kann sich das Verhältnis ändern.
- **Spezifische Hardware und Implementierung.** Gemessen wurde unter Windows 11 mit Python 3.14.8, vier logischen Prozessoren und zwei Workern. Compileroptionen, Prozessorarchitektur, Betriebssystem, Datenbanklast und Wrapper beeinflussen die Laufzeiten.

- **Mikrobenchmark ist kein vollständiger Algorithmusvergleich.** Die 200 Zufallspaare decken Matrizen mit 15 bis 20 Knoten und Kantendichte um 0,3 ab. Sie belegen den Vorteil unter dieser Verteilung, aber nicht für alle Graphklassen. Der Vergleich schließt zudem die konkrete Python-zu-C++-Übergabe ein.
- **Phasenrest ist zusammengesetzt.** Die Differenz zwischen Suchzeit und Vergleichszeit verbindet mehrere Vorgänge und darf weder als reine Datenbanklaufzeit noch als reine Parallelisierungskosten ausgelegt werden. Eine feinere Instrumentierung wäre nötig, um die verbliebenen rund zwei Drittel der C++-Zeit kausal aufzuteilen.
- **Trefferabgleich ist kein universeller Nachweis.** Identische Treffer-IDs und null Fehler in dieser Stichprobe erhöhen das Vertrauen in die Integration, ersetzen aber keine gemeinsamen Konformitäts- und Eigenschaftstests für alle Eingabekombinationen.

8.9 Fazit

Der Wechsel von Python zur C++-Library beschleunigt in der gemessenen Konfiguration sowohl den direkten Vergleich als auch die vollständige Suche. Bei denselben sechs Anfragen und 931.899 Kandidaten sinkt die Gesamtzeit um den Faktor 6,36; in allen sechs Fällen stimmen die Treffer-IDs überein, und es gibt keine fehlgeschlagenen Vergleiche. Der Mikrobenchmark misst einen noch größeren Vorteil für direkte Aufrufe, während die Phasenanalyse zeigt, dass Laden, Übergabe und Ergebnisaufbau den Ende-zu-Ende-Speedup begrenzen. Die Messergebnisse stützen damit die Library-Integration als sinnvollen Beschleunigungsweg für diesen Anwendungsfall. Sie sind wegen Stichprobengröße, synthetischer Daten und einzelner Hardwarekonfiguration eine belastbare Tendenz, aber keine universelle Leistungsgarantie.

Dass der Bytecode für die C++-Library größer ist, ist deshalb so, weil dadurch im Arbeitsspeicher ein größerer Bereich reserviert wird, der viel effizienter von der Architektur zur Ausführung prozessiert werden kann. Wenn der reservierte Bereich dagegen kleiner ist, wie für die Python Implementierung des Subgraph Algorithmus, ist der reservierte Speicherbereich kleiner.

Der Strom im reservierten Bereich im Arbeitsspeicher für den Algorithmus liegt parallel an. Ist dieser Bereich kleiner, kann die effiziente Architektur mit Adressbus, Datenbus, Steuerbus und Controller diesen Bereich nicht so schnell ausführen und bearbeiten wie für reservierte Bereiche die größer sind.

Es ist so, dass diese hochentwickelte und für Geschwindigkeit extrem ausgelegte Hardware-Architektur des Prozessors gerade darauf wartet, durch die parallele Bestromung der reservierten Bereiche im Arbeitsspeicher möglichst viele Instruktionen dadurch gleichzeitig prozessieren zu können. Etwas enttäuscht ist sie dann doch, wenn der reservierte Bereich kleiner ist und sie gerade für ihre eigentliche Bestimmung so nicht so sehr zum Ziel kommen kann, was die Ausführung des Subgraph Algorithmus verzögert.

8.10 Fremde Beschreibungen

Die Anreicherung mit einem Fremdmodul in einer anderen Beschreibung erscheint mir sehr unpassend und beschreibungstechnisch nicht durchgängig. Eine lose Kopplung dage-

gen, angenommen durch eine C++-CLI oder C++-Library, erlaubt eine klarere Trennung, bei der alle beteiligten Mitgestalter dieses Softwaresystems mit einem klaren Trennungsbewusstsein dieses System mitgestalten. Ein System, das aus Bereichen oder Teilen besteht, die durch unterschiedliche Beschreibungen entwickelt sind oder arbeiten, schafft Mitgestaltern dieses Systems ein in der Regel eher unruhigeres Trennungsbewusstsein in der Analyse, Weiterentwicklung und Wiederverwendung. Dieses unruhigere Trennungsbewusstsein kann sich in der Praxis in einer unklaren Verantwortlichkeit und Kostenverteilung an der Grenze zwischen den beiden Beschreibungen äußern. Die Messungen dieses Kapitels dagegen zeigen für Gen-DB zugleich einen deutlichen Laufzeitvorteil der Library. Die weitergehende Position dieser Arbeit lautet, dass klar benannte Grenzen Analyse, Weiterentwicklung und Wiederverwendung unterstützen und dadurch ein gemeinsames Trennungsbewusstsein fördern. Diese Aussage ist eine begründete Gestaltungsposition; die Laufzeitdaten allein messen sie nicht.

Die Einbindung eines Fremdmoduls aus einer anderen Beschreibung wirft eine Gestaltungsfrage auf: Wie bleiben Verantwortlichkeiten und Kosten an der Grenze für alle Mitgestaltenden erkennbar? Eine lose Kopplung über eine C++-Library statt über einen separaten C++-CLI-Prozess hält die technische Schnittstelle schmalere und ermöglicht es, den C++-Kern innerhalb des Python-Workers aufzurufen. Die Messungen dieses Kapitels zeigen für Gen-DB zugleich einen deutlichen Laufzeitvorteil der Library. Die weitergehende Position dieser Arbeit lautet, dass klar benannte Grenzen Analyse, Weiterentwicklung und Wiederverwendung unterstützen und dadurch ein gemeinsames Trennungsbewusstsein fördern. Diese Aussage ist eine begründete Gestaltungsposition; die Laufzeitdaten allein messen sie nicht.

Dieser Abschnitt führt die Aussage weiter aus. Er klärt zunächst, was hier unter einer *Beschreibung* verstanden wird, wendet die Aussage dann auf die Messungen dieses Kapitels an und überträgt sie anschließend auf die übrigen Teile der Arbeit. Abschließend werden Gegenargumente und die Grenzen der Position benannt. Wichtig ist die Reihenfolge der Argumente: Die Position beruht in erster Linie auf einer Überlegung zur Gestaltung des Systems. Die Messwerte aus diesem Kapitel illustrieren sie, sie beweisen sie nicht.

8.10.1 Was hier unter einer Beschreibung verstanden wird

Eine *Beschreibung* bezeichnet im Folgenden die Gesamtheit der Vereinbarungen, in denen ein Teil eines Systems formuliert ist: seine Sprache, seine Datentypen, sein Fehlermodell, die Lebensdauer seiner Zustände und die Art, wie er aufgerufen, getestet und ausgeliefert wird. Zwei Teile liegen in derselben Beschreibung, wenn sie diese Vereinbarungen teilen. Dann kann ein Mitgestalter, der den einen Teil versteht, den anderen mit denselben Begriffen lesen. Ein *Fremdmodul* ist ein Teil, der in einer anderen Beschreibung arbeitet und dennoch Teil desselben Systems sein soll.

Gen-DB besteht in der Beschreibung des Backends aus Python-Code. Matrizen liegen dort als `numpy`-Arrays vor, Fehler werden über Python-Ausnahmen signalisiert, und der Subgraph Algorithmus wird als Funktion oder Objekt aufgerufen. Die frühere C++-CLI arbeitete dagegen in einer anderen Beschreibung: Matrizen wurden zu JSON-Text, Fehler zu Exit-Codes und Ausgaben auf `stderr`, und ein Zustand überlebte keinen Aufruf. Die nun eingesetzte Library behält den nativen C++-Kern bei, stellt dem Backend aber eine direkte Funktionsschnittstelle im Worker-Prozess bereit. Sie benötigt weiterhin eine Sprachgrenze und eine passende Fehler- und Datentypabbildung, vermeidet

jedoch das pro Vergleich wiederholte Textprotokoll und den zusätzlichen Prozesslebenszyklus.

Die C++-Library ist ebenfalls in C++ geschrieben, ihre Schnittstelle kann jedoch in der Beschreibung des Aufrufers erscheinen, etwa als importierbare Funktion, die Arrays entgegennimmt und Werte oder Ausnahmen zurückgibt. Der Unterschied liegt also nicht in der Programmiersprache des Algorithmuskerns, sondern darin, *wo die Grenze zwischen beiden Beschreibungen verläuft*. Bei der Library verläuft sie an einer schmalen, typisierten Funktionsschnittstelle. Bei der CLI verläuft sie am Betriebssystem, also an Prozessgrenze, Textformat und Dateideskriptoren.

8.10.2 Anwendung auf die Messungen dieses Kapitels

Die Untersuchung unterscheidet zwischen der früheren CLI-Anbindung und der nun gemessenen Library-Anbindung. Die CLI legte eine breite Grenze aus Prozessionsstart, Textformat und Fehlerauswertung offen; die Library verläuft innerhalb des Worker-Prozesses über einen Funktionsaufruf. Der Mikrobenchmark und die Ende-zu-Ende-Messung quantifizieren den Laufzeitunterschied dieser konkreten Varianten.

Der Messaufbau überwacht die aktive Implementierung. Die Versuchsanordnung prüft die tatsächlich in den Worker-Prozessen aktive Implementierung, zählt fehlgeschlagene Einzelvergleiche und vergleicht die Treffer-IDs beider Backends (Abschnitt 8.3). Diese Kontrollen sind nötig, um den konkreten Lauf korrekt zuzuordnen und Backend-Konformität empirisch zu prüfen. Sie zeigen zugleich, dass sprachübergreifende Integration Tests an der Grenze benötigt; der abgeschlossene Lauf bestätigt für die sechs Queries gleiche Treffer-IDs und null Fehler.

Fachliche Regeln müssen an der Schnittstelle eindeutig sein. Der C++-Kern liefert Ergebniswerte, die auf die von Gen-DB verwendeten Zustände abgebildet werden; bei identischen Graphen muss außerdem dieselbe Auswahlregel wie im Python-Backend gelten. Solche Regeln dürfen bei austauschbaren Backends nicht implizit auseinanderlaufen. Die Library beseitigt nicht automatisch diese semantische Verantwortung, ermöglicht aber eine gemeinsame Funktionsschnittstelle und direkte Konformitätstests ohne JSON-Protokoll. Eine ausdrücklich dokumentierte Spezifikation bleibt daher erforderlich.

Die Library-Schnittstelle ist messbar, aber klein gegenüber dem CLI-Prozesspfad. Der direkte Mikrobenchmark (Tabelle 9) misst für C++ im Mittel 0,074 Millisekunden je Vergleich und 0,018 Millisekunden für einen trivialen 1×1 -Aufruf. Letzterer ist eine Größenordnung für Funktionsübergabe und Marshalling; er entspricht ungefähr einem Viertel der mittleren Zeit des regulären Library-Aufrufs. Dem steht der frühere CLI-Mikrobenchmark mit 10,081 Millisekunden für den trivialen Prozessionsaufruf gegenüber. Die Werte stammen aus verschiedenen Messläufen und Aufrufpfaden; sie dürfen nicht als exakt kontrollierter A/B-Vergleich subtrahiert werden. Zusammen mit dem gemessenen Ende-zu-Ende-Speedup von $6,36\times$ zeigen sie jedoch, dass die Library die Kostenstruktur der CLI vermeidet und den nativen Rechenkern praktisch nutzbar macht.

Der Prozess-Pool wird mit der Library wiederverwendet. Gen-DB betreibt einen langlebigen `ProcessPoolExecutor`, dessen Worker Kandidaten blockweise mit je 256 Vergleichen bearbeiten. Die Library wird einmal je Worker geladen und für die Vergleiche innerhalb dieses Prozesses wiederverwendet. Die frühere CLI öffnete zusätzlich einen Prozess pro Vergleich; dieser zweite Prozess-Lebenszyklus entfällt im nun gemessenen Pfad. Das passt zum Modell der Anwendung und vermeidet wiederholte Betriebssystem- und JSON-Kosten.

8.10.3 Anwendung auf die gesamte Arbeit

Die Position betrifft nicht nur dieses Kapitel. Sie knüpft an den Gedanken an, den die Schlussbemerkung als zentral bezeichnet, nämlich die Trennung zweier Aufgaben: eine günstige Vorauswahl in der Datenbank und der Subgraph Algorithmus als Entscheidung über jeden verbliebenen Kandidaten. Diese Trennung funktioniert, weil sie klar ist. Eine Anbindung, die sie an anderer Stelle durch eine unscharfe Grenze unterläuft, widerspricht dem Prinzip der Gesamtarbeit.

Systemarchitektur (Abschnitt 3). Die drei Schichten Frontend, Backend und Datenbank sind jeweils durch eine benannte Schnittstelle getrennt: REST-Aufrufe zwischen Frontend und Backend, SQL zwischen Backend und Datenbank. Der Algorithmus wird vom Modul `crud.py` über seine Backend-API aufgerufen. Die C++-Library wird innerhalb dieses Backend-Bausteins geladen und fügt keine weitere Prozessgrenze in die Suchpipeline ein.

Formale Analyse (Abschnitt 4). Satz 2 modelliert die Suchzeit als Summe aus dem Pre-Filter und C Vergleichen mit je $O(n^3)$ Aufwand. Das Modell trägt, solange die Kosten eines Vergleichs dem Algorithmus zugerechnet werden können. Eine Prozess-pro-Vergleich-Anbindung fügt jedem Vergleich einen Term hinzu, den das Modell nicht enthält.

Bemerkung 5 (Erweitertes Kostenmodell). Bezeichne δ die Schnittstellenkosten je Vergleich. Bei der CLI umfassen sie Prozessesstart, JSON-Serialisierung und Antwortauswertung; bei der Library die Übergabe der Arrays und den Funktionsaufruf. Dann lautet eine erweiterte Form des Modells aus Satz 2

$$T'_{\text{Suche}}(N, C, n) = O(\log N + C) + O(C \cdot (n^3 + \delta)).$$

Der direkte Mikrobenchmark misst für die Library 0,018 ms für einen trivialen Funktionsaufruf und 0,074 ms im Mittel für ein Zufallspaar. Der frühere CLI-Mikrobenchmark maß rund 10 ms für den trivialen Prozessaufufruf. Diese konkreten Zeiten hängen von Hardware, Datenformat und Umgebung ab; sie zeigen jedoch, dass δ je nach Anbindung einen erheblichen Unterschied im konstanten Faktor erzeugt. Aus den Werten lässt sich kein allgemeiner Vergleich von δ mit der abstrakten Zahl n^3 ableiten.

Das erweiterte Modell erhält die asymptotische Aussage, dass die Suchzeit linear mit C wächst. Es macht aber sichtbar, dass die Wahl der Anbindung den konstanten Faktor bestimmt. Ein Modell mit einem Term, den die Beschreibung der Arbeit nicht benennt, wäre ein Beispiel für das beschriebene Problem im Kleinen.

Empirische Evaluation und VF2-Vergleich (Abschnitte 5 und 6). Der Vergleich mit VF2 ist aussagekräftig, weil beide Verfahren in derselben Beschreibung laufen: Beide werden in Python aufgerufen, auf identischen Eingaben und mit derselben Zeitmessung. Die Library-Messung vergleicht zwei unterschiedliche Implementierungssprachen, aber beide Aufrufe erfolgen innerhalb der Suchworker und auf denselben Daten. Sie misst damit den realen Einfluss der Backend-Wahl auf Gen-DB und nicht ausschließlich den algorithmischen Aufwand. Der frühere CLI-Pfad hatte zusätzlich Prozessstart und JSON-Transport in die Messung eingebracht.

Parallelisierung (Abschnitt 7). Die dort gemessenen Ergebnisse, ein Speedup von 1,90 mit zwei Workern und eine Verschlechterung beim dritten Worker auf einem Rechner mit zwei logischen Prozessoren, beschreiben die Python-Arbeitslast. Der Backend-Versuch setzte zwei Worker ein und zeigte auch bei der Library einen deutlichen Suchgewinn; er untersuchte jedoch nicht systematisch unterschiedliche Worker-Zahlen. Daher bleibt offen, ob die optimale Parallelisierung und der serielle Anteil mit C++ unverändert sind. Die Worker-Zahl sollte weiterhin an verfügbare Prozessoren und die Last des Datenbankservers angepasst werden.

Schluss und Ausblick (Abschnitte 9.2 und 9.3). Die Grenzen der Arbeit nennen eine in reinem Python geschriebene Referenzimplementierung als obere Abschätzung des praktisch Erreichbaren. Der Ausblick führt als Hebel die Vektorisierung mit `numpy`, die Kompilierung mit `numba` und die C++-Erweiterung auf (Abschnitt 9.3.8). Alle drei sind Anbindungen, die in der Beschreibung des Aufrufers bleiben. Die hier vertretene Position stützt diese Reihenfolge: Weiterentwicklungen des Vergleichs sollten zuerst dort ansetzen, wo die Grenze schmal bleibt, und die Prozessgrenze nur dort einführen, wo ihr Zweck, etwa Isolation, den Beschreibungsbruch rechtfertigt.

8.10.4 Kriterien einer losen Kopplung

Aus den vorangehenden Überlegungen lassen sich Kriterien ableiten, an denen sich beide Anbindungsformen vergleichen lassen. Die Tabelle fasst Gestaltungseigenschaften zusammen; die Laufzeitangaben werden durch die Mikro- und Suchmessungen aus Abschnitt 8.5 ergänzt.

Die Zeile zur Isolation zeigt, dass die Gegenüberstellung nicht einseitig ausfällt. Sie ist die stärkste Gegenposition und wird im nächsten Abschnitt behandelt.

8.10.5 Gegenargumente und Grenzen der Position

Isolation und Sprachneutralität. Eine CLI trennt Prozesse hart. Ein Fehler im nativen Code reißt dann nicht den Worker mit, und ein Timeout kann den Prozess beenden. Außerdem ist eine CLI von jeder Sprache aus aufrufbar. Beide Eigenschaften sind echte Vorteile. Sie sprechen für die CLI als Werkzeug, etwa für Skripte, manuelle Prüfungen oder Experimente. Die hier vertretene Position betrifft die Einbindung in das System. Für diese zählt, ob die Grenze in der gemeinsamen Beschreibung des Systems liegt. Die CLI bleibt als eigenständiges Programm sinnvoll, auch wenn Gen-DB den Algorithmus über die Library aufruft.

Tabelle 11: Vergleich der früheren CLI- und der nun umgesetzten Library-Anbindung nach Gestaltungskriterien.

Kriterium	C++-CLI (Prozess pro Vergleich)	C++-Library (im Worker-Prozess)
Datenaustausch	Matrizen als JSON-Text über <code>stdin</code> und <code>stdout</code>	typisierte Arrays über eine Funktionsschnittstelle
Fehlermodell	Exit-Code, <code>stderr</code> , Timeout (<code>CSUBGRAPH_TIMEOUT</code>)	Ausnahmen oder Rückgabewerte in der Beschreibung des Aufrufers
Lebensdauer	kein Zustand über einen Vergleich hinaus	Instanz je Worker, wiederverwendbar
Aufrufkosten δ	Prozessstart je Vergleich, früher etwa 10 ms je trivialem Aufruf	Funktionsaufruf im Prozess, trivial im Mittel 0,018 ms
Ergebnisregeln	Übersetzung der Werte und Regel für identische Graphen im Backend	Abbildung kann Teil der Schnittstelle sein und liegt dann beim Algorithmus
Testbarkeit	Tests brauchen ein gebautes Executable und einen Prozessstart	Tests rufen die Funktion direkt auf, gleiche Mittel wie beim Python-Algorithmus
Auslieferung	plattformabhängiger Pfad zum Executable (<code>subgraph-cli.exe</code>) über die Konfiguration	Abhängigkeit des Projekts, Binärdatei je Plattform
Isolation	Absturz des Algorithmus betrifft nur den Einzelprozess	Absturz betrifft den Worker-Prozess
Messbarkeit	Schnittstellenkosten verdecken den Rechenkern	Rechenkern lässt sich unter gleichen Bedingungen wie Python messen

Aufwand der Library. Eine Library braucht eine Anbindung an Python, etwa über einen Erweiterungsmechanismus, und eine Bauumgebung je Plattform. Das ist Aufwand, den die CLI zunächst nicht verlangt. Die Position behauptet nicht, dass dieser Aufwand gering sei. Sie behauptet, dass er einmalig anfällt und die Grenze danach schmal hält, während die Kosten der CLI bei jedem Vergleich und bei jeder Weiterentwicklung wiederkehren.

Die Evidenz ist begrenzt, aber nicht mehr vorläufig. Die Library wurde im Mikrobenchmark und in einer vollständigen Suchreihe gemessen; beide Messungen zeigen einen deutlichen Zeitgewinn. Die Aussagekraft ist jedoch auf sechs Queries, einen Lauf je Backend und eine Hardwarekonfiguration begrenzt. Die Ergebnisse belegen daher den Vorteil in diesem Versuchsaufbau, nicht für jede Plattform oder Graphenfamilie.

Die Position ist eine Gestaltungsentscheidung. Begriffe wie Trennungsbewusstsein lassen sich in dieser Arbeit nicht messen. Sie bezeichnen eine Eigenschaft der Zusammenarbeit von Mitgestaltern, die durch Fallbeispiele wie die oben genannten plausibel gemacht, aber nicht durch eine Studie belegt wird. Die Arbeit formuliert sie deshalb als begründete Position und nicht als Ergebnis.

8.10.6 Folgerungen für Gen-DB

Aus der Position und den Messungen ergeben sich fünf Folgerungen für die weitere Arbeit am System:

1. **Eine Schnittstelle, mehrere Implementierungen.** Python- und C++-Algorithmen sollten hinter derselben Funktionsschnittstelle liegen. Der Aufrufer in Gen-DB wählt die Implementierung über die Konfiguration, und der Rückfall auf Python bleibt erhalten. Er ist dann ein Wechsel hinter einer gemeinsamen Schnittstelle und kein Wechsel der Beschreibung.
2. **Fachliche Regeln gehören zum Algorithmus.** Die Abbildung der Ergebniswerte und die Auswahl nach der Kantenzahl bei identischen Graphen sollten in der Schnittstelle des Algorithmus festgelegt sein und nicht im Übersetzungscode der Anwendung. Beide Implementierungen liefern dann per Konstruktion dieselben Werte, und die Kontrolle über die Treffer-IDs wird zu einem Test statt zu einer Laufzeitprüfung.
3. **Die Library-Messung verbreitern.** Die erste gepaarte Messreihe liegt vor. Als nächstes sollte sie mit mehr Queries, Wiederholungen, Konfidenzintervallen und weiteren Plattformen wiederholt werden; die Vergleiche sollten dabei nach Suchphasen aufgeschlüsselt werden.
4. **Schnittstellenkosten ausweisen.** Künftige Messungen sollten δ getrennt von der Rechenzeit des Algorithmus berichten, wie es die Bemerkung 5 nahelegt. Dadurch lassen sich Ergebnisse verschiedener Anbindungen vergleichen, ohne Algorithmus und Schnittstelle zu vermischen.
5. **Die CLI als Werkzeug behalten.** Die CLI bleibt für Einzelprüfungen, Skripte und Fehlersuche nützlich. Sie sollte jedoch nicht der Pfad sein, über den die produktive Suche einen Vergleich nach dem anderen ausführt.

8.10.7 Zwischenfazit zur Kopplung

Die Aussage des Abschnitts lässt sich so zusammenfassen: Ein System bleibt analysierbar, weiterentwickelbar und wiederverwendbar, wenn seine Grenzen schmal und benannt sind und die Zuständigkeiten klar bleiben. Die Messungen zeigen am Beispiel der C++-Anbindung sowohl einen strukturellen als auch einen Laufzeitunterschied: Die CLI verlegt die Grenze an Prozess, JSON-Text und Exit-Code; der frühere triviale Aufruf benötigte rund 10 ms. Die Library wird dagegen direkt im Worker aufgerufen; ihr trivialer Aufruf lag im Mittel bei 0,018 ms, und die vollständige Suche war 6,36-mal schneller als mit Python. Der Suchlauf bestätigt damit einen konkreten Leistungsvorteil dieser Library-Integration. Die allgemeinere Behauptung, dass dadurch Zusammenarbeit und Wiederverwendung erleichtert werden, bleibt eine begründete Gestaltungsposition. Die Library setzt in der Gesamtarbeit das Prinzip fort, das Architektur, formale Analyse und Messmethodik tragen: Schnittstellen und ihre Kosten sichtbar zu machen, statt sie im Gesamtergebnis zu verbergen.

9 Schluss

Die vorliegende Arbeit hat Gen-DB, ein System zur Subgraph-Suche in Datenbanken biologischer Netzwerke, von der Architektur über die formale Analyse der Such-Pipeline bis zu mehreren empirischen Untersuchungen betrachtet. Dieser abschließende Abschnitt gliedert sich in fünf Teile: Zunächst fasst Abschnitt 9.1 die Ergebnisse aller vorangegangenen Abschnitte zusammen und gleicht sie mit den in der Einleitung formulierten Beiträgen ab. Abschnitt 9.2 benennt die Grenzen der Arbeit, sodass die Ergebnisse richtig eingeordnet werden können. Abschnitt 9.3 entwickelt darauf aufbauend ausführlich mögliche Erweiterungen und offene Forschungsfragen, geordnet nach Themenfeldern. Abschnitt 9.4 schlägt schließlich eine Reihenfolge für die Umsetzung vor, und Abschnitt 9.5 schließt die Arbeit mit einer Gesamteinschätzung. Ein Faden, der sich durch alle fünf Teile zieht, ist die in Abschnitt 8.10 begründete Position zur losen Kopplung: Die Zusammenfassung ordnet sie in die Ergebnisse ein, die Würdigung benennt ihre Belastbarkeit, der Ausblick leitet daraus das Themenfeld G ab, die Priorisierung gibt ihr einen Platz in der Reihenfolge, und die Schlussbemerkung stellt sie neben den zentralen Gedanken der Trennung.

9.1 Zusammenfassung

9.1.1 Ausgangspunkt und Zielsetzung

Biologische Datenbanken enthalten Millionen von Netzwerken: metabolische Pfade, Protein-Interaktionsnetzwerke und Gen-Regulationsnetzwerke. Eine zentrale Aufgabe bei ihrer Analyse ist die Subgraph-Suche, bei der zu einem Query-Netzwerk alle gespeicherten Netzwerke gefunden werden sollen, die das Query als Teilstruktur enthalten. Diese Arbeit hat untersucht, wie sich der Subgraph Algorithmus [1], der zwei Adjazenzmatrizen über Spaltensignaturen und zyklische längste gemeinsame Teilsequenzen in $O(n^3)$ vergleicht, in eine relationale Datenbank einbetten lässt, ob die entstehende Such-Pipeline formal korrekt und analysierbar ist und wie sie sich bei einer Datenbank mit einer Million Einträgen tatsächlich verhält. Im Verlauf der Untersuchung trat eine weitere Frage hinzu: auf welche Weise ein in C++ geschriebener Rechenkern in ein Python-System eingebunden werden sollte, damit Analyse, Weiterentwicklung und Wiederverwendung klar bleiben.

9.1.2 Systemarchitektur und Datenmodell

Gen-DB besteht aus drei Schichten: einem Web-Frontend in HTML, CSS und JavaScript, einem FastAPI-Backend in Python und einer PostgreSQL-Datenbank. Das Datenmodell speichert Metadaten (Name, Netzwerktyp, Organismus, Knoten- und Kantenzahl) in der Tabelle `biological_networks` und die strukturellen Repräsentationen (Knotenlabels, Adjazenzmatrix, Signatur-Array und Signatur-Hash) in der 1:1-verknüpften Tabelle `network_matrices`. Entscheidend für die Suche ist, dass Knoten- und Kantenzahl als eigene, indizierte Spalten vorliegen, sodass sie ohne Zugriff auf die Matrizen ausgewertet werden können. Die UML-Paket- und Klassendiagramme in Abschnitt 3.3 dokumentieren, dass die Zerlegung in Präsentation, Logik und Persistenz sauber getrennt ist. Dadurch lässt sich die Such-Pipeline als eigenständiger Baustein formalisieren. Für die Ausführung der Vergleiche besitzt das Backend einen Prozess-Pool, der die Kandidaten blockweise auf mehrere Worker-Prozesse verteilt (Abschnitt 7).

9.1.3 Formale Ergebnisse zur Such-Pipeline

Die Pipeline besteht aus zwei Stufen. Die erste ist eine datenbankseitige Vorauswahl, die nur Netzwerke mit mindestens so vielen Knoten und Kanten wie das Query zulässt. Die zweite ist der strukturelle Vergleich der verbliebenen Kandidaten mit dem Subgraph Algorithmus. Gezeigt wurde Folgendes:

- Die Vorauswahl ist korrekt: Aus $A_Q \preceq A_k$ folgt $n_Q \leq n_k$ und $m_Q \leq m_k$ (Lemma 2). Damit scheidet der Pre-Filter keinen Treffer der Richtung $A_Q \preceq A_k$ fälschlich aus (Korollar 1).
- Die Gesamtlaufzeit beträgt $T_{\text{Suche}}(N, C, n) = O(\log N + C) + O(C \cdot n^3)$ (Satz 2). Bei beschränkter Knotenzahl und beschränkter Kandidatenzahl K ist die Suchzeit unabhängig von der Datenbankgröße N .
- Ohne Kandidatenlimit ($K = \infty$) findet die Pipeline alle Treffer der Richtung $A_Q \preceq A_k$ (Satz 3). Mit Limit gilt dies nur noch eingeschränkt, weil ein sortiertes LIMIT Treffer mit großer Knotenzahl systematisch abschneiden kann (Lemma zur Verzerrung durch LIMIT).
- Die symmetrische Richtung $A_k \preceq A_Q$ wird von der Vorauswahl nicht abgedeckt und braucht eine eigene Filterbedingung.

9.1.4 Empirische Evaluation der Datenbank

Als Testdatenbank dienten 1.000.000 synthetische Netzwerke (zuzüglich zweier Beispielnetzwerke) mit gleichverteilter Knotenzahl $n_k \in \{3, \dots, 20\}$, Kantenwahrscheinlichkeit $p = 0,3$ und gleichverteilten Netzwerktypen und Organismen. Im Mittel hatten sie 11,5 Knoten und 44,3 Kanten. Die Verteilungen entsprachen den Erwartungen aus dem Generierungsmodell, einschließlich der Abweichung der mittleren Kantenzahl vom naiven Erwartungswert, die sich durch die Jensen-Ungleichung erklärt. Die Auswertung zeigte, dass allein die Knotenzahlbedingung den Suchraum je nach Querygröße um bis zum Faktor 18 verkleinert, und die Kantenbedingung die Reduktion weiter verstärkt. Die Modellkurven zur Skalierung bestätigten die Aussage des Korollars zu Satz 2: Mit Kandidatenlimit bleibt die Suchzeit näherungsweise konstant, ohne Limit wächst sie linear in N .

9.1.5 Experimenteller Vergleich mit VF2

In einer direkten Messung gegen VF2 [10] auf Erdős–Rényi-Graphen $G(n, 0,3)$ war der Subgraph Algorithmus bei festem $n_A = 5$ und $n_B \in [5, 13]$ um den Faktor 1,9 bis 4,7 schneller. Der Vorsprung schmolz bei $n_B = 14$ bis 16 auf etwa 1,1 ab, und bei $n_B = 17$ und 18 war VF2 mit Speedups von 0,83 und 0,82 etwas schneller. Bei gemeinsam wachsendem $n = n_A = n_B$ stieg die Laufzeit des Subgraph Algorithmus von 0,055 ms ($n = 4$) auf 1,309 ms ($n = 12$), also um den Faktor 23,8 und damit in der Größenordnung der kubischen Referenz $3^3 = 27$, während VF2 im selben Bereich nur um den Faktor 2,8 wuchs. Die Ursachenanalyse hat dies auf vier Punkte zurückgeführt: eine Referenzimplementierung ohne vorzeitigen Abbruch, das frühe Terminieren von VF2 auf zufälligen dichten Graphen, den Interpreter-Overhead der Python-Implementierung und den grundsätzlichen Unterschied zwischen Worst-Case-Schranke und Durchschnittsverhalten. Die Literaturübersicht hat gezeigt, dass der Subgraph Algorithmus unter den

betrachteten Verfahren das einzige mit unbedingt polynomieller Worst-Case-Schranke ist, dafür aber eine strukturell schwächere Relation entscheidet als die volle Subgraph-Isomorphie. Daraus wurde eine Hybridstrategie abgeleitet, bei der je nach Graphgröße zwischen beiden Verfahren gewählt und der Subgraph Algorithmus als Fallback mit garantierter Schranke behalten wird.

9.1.6 Parallelisierung der Kandidatenvergleiche

Die Messreihe mit sechs aus der Datenbank gezogenen Queries (mindestens 15 Knoten, 62.064 bis 325.964 Kandidaten, kein Kandidatenlimit) hat die parallele Ausführung quantifiziert. Mit zwei Workern sank die Gesamtzeit aller Anfragen von 1604 auf 843 Sekunden, das entspricht einem Speedup von 1,90 und einer Effizienz von 95 %, für jede einzelne Anfrage zwischen 1,87 und 1,93. Aus dem Speedup ergibt sich ein serieller Anteil von etwa 5 %. Ein dritter Worker auf dem Rechner mit zwei logischen Prozessoren verschlechterte die Gesamtzeit um rund 8 % ($S(3) = 1,77$), weil keine zusätzliche Rechenkapazität entsteht. Die Antwortzeit wuchs mit einem Korrelationskoeffizienten von mindestens 0,99 linear mit der Kandidatenzahl. Eine einzelne Suche dauerte ohne Parallelisierung bis zu 403 Sekunden und mit zwei Workern bis zu 212 Sekunden.

9.1.7 Vergleich der Python- und C++-Implementierung

Die abgeschlossene Suchmessung mit zwei Workern verglich sechs identische Queries und insgesamt 931.899 Kandidaten. Die Python-Ausführung benötigte 1.180,28 Sekunden, die C++-Library 185,62 Sekunden; der Ende-zu-Ende-Speedup beträgt $6,36\times$. Der individuelle Speedup lag je nach Query zwischen $4,81\times$ und $8,76\times$. Im direkten Mikrobenchmark über 200 gemeinsame Zufallspaare sank die mittlere Vergleichszeit von 2,694 ms in Python auf 0,074 ms in C++; der triviale Library-Aufruf lag bei 0,018 ms. Alle sechs Suchpaare lieferten dieselben Netzwerk-IDs und beide Reihen hatten null fehlgeschlagene Vergleiche. Der Unterschied zwischen Mikrobenchmark- und Such-Speedup erklärt sich daraus, dass Datenbankzugriff, Worker-Kommunikation und Ergebnisaufbau beim schnelleren Rechenkern einen größeren Anteil an der vollständigen Antwortzeit ausmachen.

9.1.8 Kopplung und Beschreibungsgrenzen

Der Vergleich wirft neben der Laufzeitfrage eine Gestaltungsfrage auf, die Abschnitt 8.10 ausführt: Wie lässt sich ein in C++ implementierter Rechenkern in ein Python-System einbinden, ohne die Zuständigkeiten und die Messung unnötig zu verkomplizieren? Die Messungen sprechen für die Library-Anbindung: Sie hält den wiederholten Aufruf innerhalb des Worker-Prozesses, vermeidet den Prozess-pro-Vergleich- und JSON-Pfad der CLI und erreicht in den sechs Suchläufen den Speedup von $6,36\times$. Das ist ein empirischer Vorteil unter den dokumentierten Bedingungen. Die weitergehende Behauptung, eine bestimmte Grenze erleichtere Zusammenarbeit, Analyse und Wiederverwendung generell, bleibt eine begründete Gestaltungsposition und ist nicht durch eine Nutzer- oder Entwicklerstudie geprüft.

9.1.9 Zentrale Erkenntnisse

Aus der Gesamtschau lassen sich neun Erkenntnisse festhalten:

1. **Die Architektur trägt die Analyse.** Die klare Trennung zwischen relationaler Kandidatenreduktion und algorithmischem Strukturvergleich erlaubt es, beide Stufen getrennt zu beweisen, zu messen und zu optimieren.
2. **Der Pre-Filter ist notwendig, aber allein nicht ausreichend.** Er verkleinert den Suchraum erheblich, doch bei kleinen oder spärlichen Queries bleiben Hunderttausende Kandidaten übrig. Bei den sechs Messanfragen lagen zwischen 6,2 % und 32,6 % der Datenbank im Suchraum.
3. **Der Anteil der Vergleichsphase hängt vom Backend ab.** Sie beansprucht 88,4 % der Python-Zeit, aber nur 33,3 % der C++-Suchzeit. Die Gesamtsuche wächst weiterhin mit der Kandidatenzahl; nach der Beschleunigung durch die Library begrenzen Datenzugriff, Übergabe und Ergebnisaufbau den End-to-End-Speedup stärker.
4. **Garantie und Geschwindigkeit sind zwei verschiedene Eigenschaften.** Der Subgraph Algorithmus bietet eine bewiesene polynomielle Schranke und vorhersagbare Laufzeiten. Auf kleinen dichten Zufallsgraphen kann ein Verfahren mit exponentieller Worst-Case-Schranke wie VF2 im Durchschnitt gleich schnell oder schneller sein.
5. **Parallelisierung ist ein verlässlicher, aber begrenzter Hebel.** Sie senkt die Antwortzeit auf dem untersuchten System um einen konstanten Faktor nahe der Prozessorzahl, ändert aber nichts an der linearen Abhängigkeit von C . Sie ersetzt deshalb weder einen schärferen Pre-Filter noch ein Kandidatenlimit.
6. **Die Worker-Zahl muss zur Hardware passen.** Mehr Worker als logische Prozessoren verschlechtern die Laufzeit. Die mitgelieferte Voreinstellung von vierzehn Workern ist deshalb nur auf entsprechend ausgestatteten Rechnern sinnvoll.
7. **Vollständigkeit und Geschwindigkeit stehen im Spannungsfeld.** Ein Kandidatenlimit macht die Suchzeit unabhängig von N , kostet aber die Vollständigkeitsgarantie. Auch die Suchrichtung ist nur für $A_Q \preceq A_k$ abgesichert.
8. **Die Form der Sprachgrenze hat praktische Folgen.** Der direkte Aufruf der C++-Library beschleunigt die sechs gemessenen Suchen gegenüber Python um den Faktor 6,36, während der frühere CLI-Mikrobenchmark hohe Kosten pro Prozessaufruf zeigte. Die Library hält den wiederholten Aufruf im Worker-Prozess und erzielt damit sowohl eine schmale Grenze als auch einen gemessenen Anwendungsvorteil (Abschnitt 8.10).
9. **Schnittstellenkosten gehören ins Kostenmodell.** Der direkte C++-Aufruf benötigt im Mittel 0,074 ms, wovon der triviale Library-Aufruf rund 0,018 ms als Größenordnung der Übergabe zeigt. Das Modell aus Satz 2 wird durch einen Term δ je Vergleich erweitert (Bemerkung 5); die Anbindung beeinflusst den konstanten Faktor, nicht die Asymptotik.

9.1.10 Abgleich mit den Beiträgen der Arbeit

In der Einleitung wurden Beiträge zur Architektur, formalen Analyse, Datenbankevaluation, zum VF2-Vergleich, zur Parallelisierung und zu den C++-Backends angekündigt.

Die Architektur und Einbettung in die relationale Datenbank werden in Abschnitt 3 beschrieben; Korrektheit und Komplexität der Pipeline sind in Abschnitt 4 bewiesen, ausdrücklich für die Suchrichtung $A_Q \preceq A_k$ und unter Berücksichtigung eines optionalen Kandidatenlimits. Die Evaluation an einer Million synthetischen Netzwerken steht in Abschnitt 5. VF2-Vergleich, Parallelisierung und Backend-Vergleich finden sich in den Abschnitten 6, 7 und 8. Der Backend-Beitrag ist nun durch Mikrobenchmark, abgeschlossene Ende-zu-Ende-Messung, Trefferabgleich und Schnittstellenanalyse erfüllt; die Reproduzierbarkeit und Aussagekraft bleiben durch die kleine Stichprobe begrenzt. Zusammenfassung, Würdigung der Grenzen und Ausblick bilden dieses Kapitel. Dabei ist zu unterscheiden: Die Skalierungskurven in Abschnitt 5 beruhen auf dem analytischen Modell, während VF2-, Parallelisierungs- und Backend-Werte aus Messungen stammen.

9.2 Kritische Würdigung und Grenzen

Die Ergebnisse dieser Arbeit sind belastbar, soweit es um die formalen Aussagen geht. Für die empirischen Aussagen gelten die folgenden Einschränkungen, die bei der Interpretation berücksichtigt werden müssen.

1. **Die entschiedene Relation ist nicht die Subgraph-Isomorphie.** Die zyklische, auf den Zeilenkomponenten der Signaturen beruhende Relation $\preceq$ ist strukturell schwächer als die klassische Subgraph-Isomorphie. Treffer von Gen-DB sind daher Treffer im Sinne von $\preceq$ und nicht ohne Weiteres im Sinne der Isomorphie. Die Laufzeitvergleiche mit VF2 vergleichen deshalb Größenordnungen und nicht die Mächtigkeit der Relationen.
2. **Nur eine Suchrichtung ist abgesichert.** Die Vollständigkeitsaussage gilt für $A_Q \preceq A_k$. Für die umgekehrte Richtung fehlt eine bewiesene Vorauswahl.
3. **Kandidatenlimit und Implementierungsstand.** Die Analyse in Abschnitt 4 modelliert ein optionales Kandidatenlimit K , und das dort gezeigte SQL-Statement enthält eine LIMIT-Klausel. Die für die Messungen in Abschnitt 7 verwendete Programmfassung führt dagegen den Vergleich über alle Kandidaten aus. Beides ist mit den Sätzen vereinbar, die Textstellen sollten bei einer Überarbeitung jedoch auf einen einheitlichen Stand gebracht werden.
4. **Teilweise modellbasierte Auswertung.** Die Skalierungskurven in Abschnitt 5 beruhen auf dem analytischen Laufzeitmodell und nicht auf Messungen. Gemessen sind der Vergleich mit VF2 (Abschnitt 6) und die Parallelisierung (Abschnitt 7).
5. **Synthetische Daten.** Alle Netzwerke stammen aus dem Erdős-Rényi-Modell mit $p = 0,3$. Reale biologische Netzwerke sind häufig skalenfrei, haben hohe Clusterkoeffizienten und Hub-Knoten. Kandidatenzahlen, Trefferquoten und Kosten pro Vergleich können dort deutlich abweichen.
6. **Geringe Stichprobengröße.** Der Vergleich mit VF2 beruht auf 8 bzw. 5 Wiederholungen je Datenpunkt, die Parallelisierungsmessung auf 6 Anfragen mit je einer Messung. Konfidenzintervalle und Signifikanztests fehlen.
7. **Begrenzte Hardware.** Die Parallelisierung wurde auf einem Rechner mit zwei logischen Prozessoren gemessen, auf dem auch die Datenbank lief. Aussagen zur

Skalierung auf viele Prozessoren oder auf getrennte Datenbankserver sind daher Extrapolationen.

8. **Ungenutzte Knotenlabels.** Die Relation $\preceq$ ignoriert Knotenlabels. Zwei strukturell gleiche, biologisch aber verschiedene Netzwerke gelten als äquivalent, was für die praktische Anwendung eine wesentliche Einschränkung ist.
9. **Referenzimplementierung ohne Optimierung.** Die Laufzeitwerte des Subgraph Algorithmus stammen aus einer in reinem Python geschriebenen Implementierung ohne vorzeitigen Abbruch. Sie sind daher eher eine obere Abschätzung des praktisch Erreichbaren.
10. **Begrenzte Backend-Messreihe.** Die C++-Library und die Ende-zu-Ende-Suche sind inzwischen gemessen (Abschnitt 8.5). Der Speedup von 6,36 und die übereinstimmenden Treffer gelten für sechs Queries, eine Messung je Backend und eine Windows-11-Konfiguration mit zwei Workern. Wiederholungen, Konfidenzintervalle, weitere Plattformen und reale biologische Daten fehlen.
11. **Phasenmodell und Schnittstellenkosten.** Die Messung zerlegt die Suchzeit in Vergleichsphase und Rest; der Rest fasst Datenbankzugriff, Datenübergabe, Aufgabenverteilung und Ergebnisaufbau zusammen. Die 0,018 ms für einen trivialen Library-Aufruf sind nur eine Orientierung für die Übergabekosten und dürfen nicht als exakte Korrektur auf größere Matrizen übertragen werden. Auch der frühere CLI-Wert von rund 10 ms ist eine Messung eines anderen Aufrufpfads und keine kontrollierte Schätzung für alle Plattformen.
12. **Die Kopplungsposition ist eine Gestaltungsposition.** Begriffe wie Trennungsbewusstsein lassen sich in dieser Arbeit nicht messen. Die Fallbeispiele aus dem Versuchsaufbau machen die Position plausibel, belegen sie aber nicht durch eine Studie. Die Gegenargumente (Isolation, Sprachneutralität, Bauaufwand der Library) sind nicht entkräftet, sondern nur dem Einsatzzweck zugeordnet.
13. **Betriebssystemabhängigkeit der C++-Anbindung.** Die Library-Messung stammt von einem Windows-11-Rechner. Bau, Linken, DLL-Laden und Wrapper-Aufruf können sich auf anderen Betriebssystemen unterscheiden. Der strukturelle Unterschied zum CLI-Prozessstart bleibt bestehen, aber die gemessene Größenordnung des Beschleunigungsgewinns ist nicht ohne plattformübergreifende Wiederholung übertragbar.

9.3 Ausblick

Die Erweiterungsmöglichkeiten lassen sich in sieben Themenfelder ordnen: die Absicherung von Korrektheit und Suchsemantik, die Verschärfung der Vorauswahl und Indexierung, die Beschleunigung der Vergleiche, die Skalierung auf Systemebene, die Weiterentwicklung der Evaluation, die Anwendungsebene sowie die Kopplung und die Schnittstellen der Implementierungen. Die folgenden Unterabschnitte behandeln jedes Feld einzeln. Wo eine Idee bereits durch die Ergebnisse dieser Arbeit gestützt oder relativiert wird, ist darauf verwiesen.

Themenfeld A: Korrektheit und Suchsemantik

9.3.1 Vollständige Suche in beide Richtungen

Die Pipeline findet bisher nur Netzwerke, die das Query enthalten. Für die umgekehrte Richtung, bei der ein gespeichertes Netzwerk Teil des Querys ist, lässt sich dieselbe Argumentation wie in Lemma 2 anwenden: Aus $A_k \preceq A_Q$ folgt $n_k \leq n_Q$ und $m_k \leq m_Q$. Die zugehörige Kandidatenmenge ist

$$\mathcal{C}'(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : n_k \leq n_Q \wedge m_k \leq m_Q\},$$

und eine vollständige Suche in beide Richtungen verarbeitet die Vereinigung $\mathcal{C}(Q, \mathcal{D}) \cup \mathcal{C}'(Q, \mathcal{D})$. Die beiden Mengen überschneiden sich genau in den Netzwerken mit $n_k = n_Q$ und $m_k = m_Q$. Für die Praxis ist wichtig, dass $\mathcal{C}'$ bei großen Queries sehr groß wird: Für $n_Q = 20$ erfüllt fast jedes Netzwerk die Knotenbedingung $n_k \leq 20$. Eine Suche in Gegenrichtung ist deshalb nur mit einer Parallelisierung und mit einer Ergebnisbegrenzung sinnvoll und sollte in API und Oberfläche als eigener Suchmodus gekennzeichnet werden. Ob die Gegenrichtung für biologische Anwendungen überhaupt gebraucht wird (die Frage lautet dann: „Welche gespeicherten Motive stecken in meinem großen Netzwerk?“), ist eine fachliche Frage, die mit Anwendern geklärt werden sollte.

9.3.2 Differenzielle Tests der Vorauswahl und Eigenschaftstests

Die Korrektheit des Pre-Filters wurde bewiesen, aber nicht automatisiert geprüft. Ein wirksamer Test vergleicht auf kleinen Datenbanken (etwa tausend zufällige Netzwerke) die Ergebnisse der Pipeline mit einer vollständigen Suche ohne Vorauswahl, bei der jedes Netzwerk mit dem Subgraph Algorithmus verglichen wird. Beide Ergebnismengen müssen für die Richtung $A_Q \preceq A_k$ identisch sein. Ergänzend können Eigenschaften, die aus der Definition der Relation folgen, als Eigenschaftstests (*property-based tests*) formuliert werden, etwa dass jedes Netzwerk Subgraph von sich selbst ist und dass der Vergleich nicht von der zyklischen Anordnung der Zeilenkomponenten abhängt. Solche Tests würden auch jede spätere Optimierung des Vergleichs (Abschnitt 9.3.8) absichern.

9.3.3 Erweiterung auf gewichtete und beschriftete Subgraph-Isomorphie

Die aktuelle Subgraph-Relation $\preceq$ berücksichtigt ausschließlich die *Struktur* (Adjazenzmatrix), nicht jedoch die *Knotenlabels* L (z. B. Glucose, p53). Zwei strukturell identische, aber biologisch unterschiedliche Netzwerke (z. B. Glykolyse vs. ein Zufallsgraph gleicher Adjazenzmatrix) würden aktuell als äquivalent erkannt. Eine biologisch sinnvolle Erweiterung wäre eine *gefärbte* Subgraph-Relation

$$(L_Q, A_Q) \preceq_\ell (L_k, A_k) \iff A_Q \preceq A_k \wedge \exists \text{Knotenabbildung } \phi : L_Q(i) = L_k(\phi(i)) \ \forall i,$$

deren Prüfung zusätzlich eine Label-Kompatibilitätsprüfung in $O(n)$ pro Rotation erfordert (Gesamtkomplexität bleibt $O(n^3)$, da $O(n) \subseteq O(n^2)$). Für kontinuierliche Kantengewichte $w : E \rightarrow \mathbb{R}_{\geq 0}$ (z. B. Reaktionsraten) wäre die binäre Signatur $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$ durch eine gewichtete Variante $\sigma_j^w = \sum_i w_{ij} 2^i + j 2^n$ zu ersetzen, wobei die Injektivität aus Lemma 1 nur noch für endliche Wertebereiche $w_{ij} \in \{0, \dots, W\}$ mit Basis $W + 1$ statt 2 erhalten bliebe.

Themenfeld B: Vorauswahl und Indexierung

9.3.4 Indexierung über Signatur-Hashes

Aktuell wird der Signatur-Hash $h(A)$ zwar berechnet und gespeichert (Spalte `signature_hash`, indiziert über `idx_matrices_hash`), jedoch nicht für die Subgraph-Suche genutzt — er eignet sich nur für *exakte* Übereinstimmungen (inklusive Rotation, falls die Hashfunktion rotationsinvariant gestaltet wird). Eine Erweiterung bestünde darin, einen *rotationsinvarianten* Hash

$$h^*(A) = \min_{r \in \{0, \dots, n-1\}} \text{SHA256}(\text{str}(\text{rot}_r(\rho(A))))$$

zu definieren und zu indizieren. Damit ließen sich **exact**-Treffer (Korollar zu Satz 3 für $A_Q \equiv A_k$ bis auf Rotation) in $O(\log N)$ statt $O(C \cdot n^3)$ finden, indem zunächst eine Hash-Gleichheitssuche durchgeführt wird und der teure Subgraph-Vergleich nur für **subgraph**-Treffer (echte Teilstruktur) verbleibt. Die Korrektheit dieser Optimierung folgt aus der Tatsache, dass $h^*(A_Q) = h^*(A_k) \Rightarrow A_Q \preceq A_k \wedge A_k \preceq A_Q$ (Äquivalenz unter Rotation), was eine direkte Anwendung von Lemma 1 auf die Zeilenkomponenten ist.

9.3.5 Mehrdimensionale Indexstrukturen (R-Baum über (n_k, m_k))

Der aktuelle Pre-Filter nutzt zwei unabhängige B-Baum-Indizes (`node_count`, `edge_count`) und kombiniert sie via konjunktiver **WHERE**-Klausel, was PostgreSQL typischerweise durch Bitmap-Index-Scans mit anschließendem **AND** realisiert. Eine direktere Lösung wäre ein zweidimensionaler Index (z. B. GiST/R-Baum über das Tupel (n_k, m_k) als Punkt im $\mathbb{R}^2$), der die Bedingung $n_k \geq n_Q \wedge m_k \geq m_Q$ als achsenparallele Bereichsabfrage (“dominance query”) in $O(\log N + C)$ statt der aktuellen Bitmap-Kombination beantwortet. Formal entspricht $\mathcal{C}(Q, \mathcal{D})$ exakt der Menge der Punkte (n_k, m_k) , die den Punkt (n_Q, m_Q) im Sinne der Pareto-Dominanz dominieren — ein klassisches Problem der rechnerischen Geometrie, für das R-Bäume und k -d-Bäume $O(\log N)$ -Anfragezeiten (amortisiert) erreichen.

9.3.6 Zusätzliche notwendige Bedingungen und Label-Indizes

Die Vorauswahl nutzt derzeit nur Knoten- und Kantenanzahl. Jede weitere Größe, die sich billig aus der Datenbank lesen lässt und für $A_Q \preceq A_k$ notwendig ist, würde den Suchraum weiter verkleinern. Solche Bedingungen müssen vor ihrer Verwendung aus der Definition von $\preceq$ bewiesen werden, denn eine zu scharfe Bedingung würde echte Treffer ausschließen. Bei Einführung der beschrifteten Relation $\preceq_\ell$ (Themenfeld A) ist dagegen eine sehr wirksame Bedingung unmittelbar ableitbar: Das Multiset der Knotenlabels des Querys muss im Multiset der Labels des Kandidaten enthalten sein. PostgreSQL unterstützt für Array-Spalten wie `node_labels` GIN-Indizes mit dem Enthaltenseins-Operator `@>` [4], sodass diese Bedingung direkt im Index ausgewertet werden könnte. Biologische Queries enthalten meist seltene Labels (etwa p53), wodurch die Kandidatenmenge in diesem Fall um Größenordnungen kleiner würde als bei der reinen Zählung von Knoten und Kanten. Einfacher umzusetzen ist ein zusammengesetzter Index über (`node_count`, `edge_count`), der die beiden bisher getrennten B-Baum-Indizes ersetzt, bevor der aufwendigere R-Baum-Ansatz nötig wird.

Themenfeld C: Beschleunigung der Vergleiche

9.3.7 Weiterführende Parallelisierung

Die Parallelisierung der Kandidatenvergleiche ist in Gen-DB umgesetzt und in Abschnitt 7 vermessen. Die Messung zeigt zugleich, an welchen Stellen weitere Gewinne liegen. Das Modell $T_{\text{parallel}}(C, n, p) = O(\frac{C}{p}n^3) + O(n^3)$ aus Abschnitt 7 übersieht, dass das Laden und Übertragen der Kandidaten ebenfalls mit C wächst. Ein realistischeres Modell lautet

$$T_{\text{parallel}}(C, n, p) = \underbrace{O(C \cdot n^2)}_{\text{Laden, Übertragung, Ergebnisaufbau (seriell)}} + \underbrace{O\left(\frac{C}{p} \cdot n^3\right)}_{\text{Vergleiche}} + O(n^3),$$

in dem der serielle Anteil nach dem Gesetz von Amdahl [15] den erreichbaren Speedup für große p auf etwa $1/f$ begrenzt. Mit dem geschätzten Anteil $f \approx 5\%$ wäre der Gewinn rechnerisch bei etwa dem zwanzigfachen Wert gedeckelt; da dieser Wert aus einem einzigen Messpunkt stammt, ist er nur ein Anhaltspunkt. Folgende Maßnahmen setzen am seriellen Anteil an:

- **Strömendes Laden.** Die Kandidaten werden derzeit mit einem einzigen Abruf vollständig in den Speicher gelesen. Ein serverseitiger Cursor würde sie in Blöcken liefern, sodass das Laden mit den Vergleichen überlappt und der Speicherbedarf bei $C \approx 3 \cdot 10^5$ sinkt.
- **Query nur einmal pro Task übertragen.** Die Query-Matrix wird gegenwärtig für jeden Kandidaten mitgeführt; bei einem Block von 256 Vergleichen wird sie 256-mal serialisiert. Eine Übergabe pro Block oder pro Worker (etwa über einen Initialisierer des Prozess-Pools bei Beginn der Suche) würde den Kommunikationsaufwand senken.
- **Blockgröße anpassen.** Die Blockgröße von 256 Vergleichen ist ein Kompromiss zwischen Kommunikationsaufwand und Lastverteilung. Eine adaptive Wahl (kleinere Blöcke am Ende, um Leerlauf zu vermeiden) und eine Messreihe zur Blockgröße sind einfach umzusetzen.
- **Worker-Zahl automatisch an die Hardware anpassen.** Die Ergebnisse zeigen, dass mehr Worker als logische Prozessoren schaden. Die Konfiguration sollte deshalb standardmäßig an `os.cpu_count()` gebunden und eine Überschreitung mit einer Warnung quittiert werden. Dabei ist zu bedenken, dass der Hauptprozess und der Datenbankserver ebenfalls Rechenzeit beanspruchen.
- **Gleichzeitige Suchanfragen.** Der Prozess-Pool wird von allen Anfragen gemeinsam genutzt. Laufen mehrere lange Suchen gleichzeitig, konkurrieren sie um dieselben Worker. Eine Warteschlange mit Priorisierung kurzer Suchen oder ein Limit gleichzeitiger Suchen würde die Antwortzeit kleiner Anfragen schützen.

Für sehr große Kandidatenmengen oder viele Nutzer kommt zusätzlich die Verteilung über mehrere Rechner in Betracht, etwa mit Apache Spark und einer benutzerdefinierten Funktion für `compare_graphs`. Dabei muss der Kommunikationsaufwand für den Versand der Adjazenzmatrizen (je $O(n^2)$ Bit) gegen den Rechengewinn $O(n^3/p)$ abgewogen

werden. Für $n \leq 20$ stehen $n^2 = 400$ Bit nur $n^3 = 8000$ Operationen gegenüber, sodass sich die Verteilung bei $p \ll C$ lohnt. Eine solche Verteilung führt allerdings eine weitere Beschreibung in das System ein; ob sie diesen Bruch rechtfertigt, ist nach den Prüffragen in Abschnitt 9.3.26 zu klären. Das Gesetz von Gustafson [16] legt zudem nahe, bei mehr Prozessoren auch die Problemgröße zu steigern, also etwa größere Netzwerke oder Datenbanken zu bearbeiten, anstatt nur dieselbe Suche zu beschleunigen.

9.3.8 Optimierung der Implementierung des Vergleichs

Die Ursachenanalyse in Abschnitt 6.3 hat gezeigt, dass die Referenzimplementierung des Subgraph Algorithmus die Tabelle der längsten gemeinsamen Teilsequenz für jede Rotation vollständig aufbaut und in reinem Python arbeitet. Daraus ergeben sich drei Hebel, die die asymptotische Schranke $O(n^3)$ nicht verändern, die gemessene Zeit aber erheblich senken können. Erstens kann die Suche über die Rotationen abgebrochen werden, sobald eine Rotation die Bedingung erfüllt; im günstigen Fall genügt dann eine einzige Tabelle. Zweitens lässt sich die Tabelle zeilenweise abbrechen, wenn bereits feststeht, dass die geforderte Mindestlänge nicht mehr erreichbar ist. Drittens kann die innere Schleife mit `numpy` vektorisiert oder mit `numba` bzw. als C++-Erweiterung kompiliert werden. Die Wahl der Anbindung ist beim C++-Weg keine Nebensache: Abschnitt 9.3.23 begründet, weshalb eine Library der CLI vorzuziehen ist. Da der Vergleich nach der Abschätzung in Abschnitt 7 etwa 95 % der Laufzeit ausmacht, wirkt jede Beschleunigung des Einzelvergleichs fast vollständig auf die Gesamtzeit durch; zusammen mit den Prozessen multiplizieren sich die Gewinne. Diese Maßnahmen müssen allerdings durch die differenziellen Tests aus Abschnitt 9.3.1 abgesichert werden, damit die Optimierung das Ergebnis nicht verändert.

9.3.9 GPU-Beschleunigung der Signatur- und LCS-Berechnung

Die Signaturberechnung (Schritt 1 des Subgraph Algorithmus) ist eine Matrix-Vektor-Operation der Form $\sigma_j = \sum_i A_{ij}2^i + j2^n$, die sich als gewichtete Spaltensummen mit Bitmasken-Gewichten 2^i darstellen lässt — eine klassische BLAS-Level-2-Operation, die auf GPUs (CUDA, via CuPy oder PyTorch) hochparallel ausgeführt werden kann. Für C Kandidaten mit $n \times n$ -Matrizen ergibt sich ein Tensor der Form (C, n, n) , dessen Signaturen in einem einzigen GPU-Kernel-Aufruf berechnet werden könnten, was Schritt 1 von $O(C \cdot n^2)$ sequentiellen auf $O(C \cdot n^2 / P_{\text{GPU}})$ effektiv parallele Operationen reduziert (P_{GPU} = Anzahl CUDA-Kerne). Die LCS-Berechnung (Schritt 3) ist aufgrund der sequentiellen Abhängigkeiten der DP-Tabelle ($dp[i][j]$ hängt von $dp[i-1][j-1]$ ab) schwieriger zu parallelisieren, jedoch existieren Wellenfront-Parallelisierungen (Anti-Diagonalen der DP-Tabelle sind unabhängig berechenbar), die auf GPUs in $O(n)$ Schritten mit $O(n)$ Prozessoren statt $O(n^2)$ sequentiellen Schritten realisierbar wären (Brent-Theorem).

9.3.10 Hybridstrategie: Adaptive Verfahrenswahl nach Kandidatengröße

Die in Abschnitt 6 beobachtete Umkehrung des Speedup-Verhältnisses bei $n_B \approx 14$ (Tabelle 2, Abbildung 8) motiviert eine *adaptive Hybridstrategie*: Da sowohl n_A (Query-Größe n_Q) als auch n_B (Kandidatengröße n_k) zum Zeitpunkt des Vergleichs bekannt sind (sie sind Teil des Tupels $(id_k, L_k, A_k, n_k, m_k) \in \mathcal{C}(Q, \mathcal{D})$, vgl. Abschnitt 3), könnte die Vergleichsfunktion `compare_graphs` zur Laufzeit zwischen dem Subgraph Algorithmus und VF2 umschalten:

$$\text{Verfahren}(n_Q, n_k) = \begin{cases} \text{Subgraph Algorithmus} & \text{falls } \max(n_Q, n_k) < \theta \\ \text{VF2} & \text{falls } \max(n_Q, n_k) \geq \theta \end{cases}$$

mit einem aus den Messdaten von Abschnitt 6 kalibrierten Schwellwert $\theta \approx 14$.

Satz 4 (Optimalität der Hybridstrategie auf Messbasis). Sei $T_{\text{Sub}}(n)$ und $T_{\text{VF2}}(n)$ die (für eine feste Graphenfamilie $G(n, p)$ erwartete) Laufzeit beider Verfahren. Definiere $T_{\text{Hybrid}}(n) = \min(T_{\text{Sub}}(n), T_{\text{VF2}}(n))$. Dann gilt für jede Folge von Vergleichsanfragen $(n^{(1)}, \dots, n^{(C)})$:

$$\sum_{i=1}^C T_{\text{Hybrid}}(n^{(i)}) \leq \min \left(\sum_{i=1}^C T_{\text{Sub}}(n^{(i)}), \sum_{i=1}^C T_{\text{VF2}}(n^{(i)}) \right).$$

Beweis. Da $T_{\text{Hybrid}}(n^{(i)}) \leq T_{\text{Sub}}(n^{(i)})$ und $T_{\text{Hybrid}}(n^{(i)}) \leq T_{\text{VF2}}(n^{(i)})$ für jedes i nach Definition des Minimums gilt, folgt durch Summation über $i = 1, \dots, C$ sofort $\sum_i T_{\text{Hybrid}}(n^{(i)}) \leq \sum_i T_{\text{Sub}}(n^{(i)})$ und ebenso $\leq \sum_i T_{\text{VF2}}(n^{(i)})$, also auch $\leq$ dem Minimum beider Summen. $\square$

Bemerkung 6. Satz 4 ist im strengen Sinne trivial (eine punktweise Minimum-Strategie ist niemals schlechter als jede ihrer Komponenten), seine praktische Bedeutung liegt jedoch darin, dass er die Hybridstrategie als *strikt dominant* gegenüber jeder festen Verfahrenswahl ausweist — *vorausgesetzt*, der Umschalt-Overhead (Bestimmung von $\max(n_Q, n_k)$ und Verzweigung) ist $O(1)$, was hier der Fall ist, da n_Q und n_k bereits in $\mathcal{D}_t$ materialisiert vorliegen (Abschnitt 3). Die Schranke $O(n^3)$ aus Satz 2 bleibt für die Hybridstrategie *nicht* formal erhalten, da T_{VF2} keine polynomielle Worst-Case-Schranke besitzt; für $n \geq \theta$ würde die Pipeline daher weiterhin den Subgraph Algorithmus als *Fallback mit garantierter Schranke* benötigen, etwa mit einem Zeitbudget τ : Falls VF2 nach τ nicht terminiert hat, wird auf den Subgraph Algorithmus zurückgegriffen (*Race-Verfahren*). Dies verbindet die empirische Average-Case-Effizienz von VF2 (Abschnitt 6) mit der formalen Worst-Case-Garantie des Subgraph Algorithmus.

Themenfeld D: Skalierung auf Systemebene

9.3.11 Asynchrone Suche mit Fortschrittsanzeige und Teilergebnissen

Eine Suche mit mehreren hunderttausend Kandidaten dauert Minuten (bis zu 403 Sekunden in Abschnitt 7) und blockiert den HTTP-Aufruf bis zum Ende. Für interaktive Nutzung eignet sich ein asynchrones Modell: Die API nimmt die Suche an, liefert sofort eine Kennung zurück und stellt den Fortschritt (verglichene Kandidaten von C) sowie die bisher gefundenen Treffer über Abfrage, Server-Sent Events oder WebSockets bereit. Der Prozess-Pool liefert Ergebnisse bisher in der Reihenfolge der Kandidaten; mit `as_completed` oder blockweiser Ausgabe könnten Treffer sofort sichtbar werden. Da die Kandidaten nach Knotenzahl sortiert sind, ließe sich zusätzlich ein „Anytime“-Verhalten erreichen: Der Nutzer sieht früh erste Treffer und kann die Suche abbrechen. Das senkt die wahrgenommene Antwortzeit unabhängig von der tatsächlichen Gesamtzeit und ist ohne Änderung des Algorithmus möglich.

9.3.12 Approximative Suche und geschichtete Stichproben

Die Beschränkung $K = 10.000$ (Abschnitt 4) gefährdet die Vollständigkeitsgarantie aus Satz 3, falls $|\mathcal{C}(Q, \mathcal{D})| > K$. Eine Abmilderung wäre eine *geschichtete Stichprobe*: Partitioniere $\mathcal{C}(Q, \mathcal{D})$ in Schichten nach n_k (also $\mathcal{C}_3, \dots, \mathcal{C}_{20}$) und ziehe aus jeder nichtleeren Schicht $\mathcal{C}_i$ mindestens $\lceil K/18 \rceil$ Kandidaten, bevor das verbleibende Budget proportional zur Schichtgröße verteilt wird. Dies garantiert, dass Treffer mit seltenen Knotenzahlen n_k (z. B. $n_k = 20$, am dünnsten besetzte Schicht) nicht systematisch unterrepräsentiert werden — ein Problem, das bei einfachem LIMIT K nach ORDER BY node_count ASC auftritt (kleine n_k werden bevorzugt, große n_k ggf. nie erreicht).

Lemma 4 (Verzerrung durch LIMIT). Sei $\mathcal{C}(Q, \mathcal{D})$ sortiert nach n_k aufsteigend und $K < |\mathcal{C}(Q, \mathcal{D})|$. Dann enthält das durch LIMIT K gelieferte Präfix $\mathcal{C}_K$ keinen Kandidaten mit $n_k > n^*$, wobei n^* das kleinste n mit $|\{k : n_k \leq n\}| \geq K$ ist.

Beweis. Da ORDER BY node_count ASC eine totale Ordnung auf $\mathcal{C}(Q, \mathcal{D})$ induziert und LIMIT K die ersten K Elemente dieser Ordnung zurückgibt, ist $\mathcal{C}_K = \{k \in \mathcal{C}(Q, \mathcal{D}) : \text{rank}(n_k) \leq K\}$. Per Definition von n^* als kleinstem Wert mit kumulativer Häufigkeit $\geq K$ folgt $n_k \leq n^*$ für alle $k \in \mathcal{C}_K$. $\square$

Dieses Lemma motiviert formal die Notwendigkeit der oben skizzierten Schichtung für eine unverzerrte approximative Suche.

9.3.13 Caching häufiger Queries

Da biologische Analysen oft wiederholt nach denselben oder ähnlichen Motiven suchen (z. B. Standard-Pfadabschnitte der Glykolyse), bietet sich ein LRU-Cache auf Ebene des Signatur-Arrays $\Sigma(A_Q)$ an: Wird $\Sigma(A_Q)$ bereits für eine vorherige Anfrage berechnet, kann Schritt 1 des Subgraph Algorithmus übersprungen werden (Ersparnis $O(n_Q^2)$ pro wiederholter Anfrage). Bei einer angenommenen Cache-Trefferquote $\alpha \in [0, 1]$ reduziert sich die erwartete Laufzeit aus Satz 2 zu

$$\mathbb{E}[T_{\text{Suche}}] = (1 - \alpha) \cdot O(n_Q^2) + O(C \cdot n^3),$$

was insbesondere bei kleinem C (großes n_Q , vgl. Abbildung 4) relativ stärker zu Buche schlägt.

9.3.14 Inkrementelle Aktualisierung der Signatur-Indizes

Bei INSERT bzw. DELETE einzelner Netzwerke (wie in der Gen-DB-API über POST/DELETE /api/networks) werden Signaturen aktuell vollständig neu berechnet (Aufwand $O(n^2)$ pro Operation, vernachlässigbar). Bei einer zukünftigen Erweiterung um *Batch-Updates* (Änderung mehrerer Kanten gleichzeitig) wäre jedoch eine inkrementelle Aktualisierung der Signaturen wünschenswert: Da $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$ linear in den Einträgen A_{ij} ist, lässt sich eine Änderung $A_{ij} \rightarrow A'_{ij}$ als

$$\sigma'_j = \sigma_j + (A'_{ij} - A_{ij}) \cdot 2^i$$

in $O(1)$ pro geänderter Kante nachführen, ohne die gesamte Spalte neu zu durchlaufen — eine direkte Konsequenz der Linearität der Signaturfunktion.

9.3.15 Betrieb, Reproduzierbarkeit und Beobachtbarkeit

Für den produktiven Betrieb sind mehrere organisatorische Erweiterungen sinnvoll. Die Dockerdateien des Projekts erlauben bereits einen reproduzierbaren Aufbau; sie sollten um einen automatisierten Testlauf und eine Messung der Suchzeit in einer Continuous-Integration-Pipeline ergänzt werden. Der Server sollte die Zeiten der einzelnen Phasen (Laden, Vergleichen, Ergebnisaufbau) und die Zahl der Kandidaten protokollieren oder als Metriken veröffentlichen. Das würde die in Abschnitt 7 fehlende Phasenerlegung liefern und den seriellen Anteil direkt messbar machen, anstatt ihn aus dem Speedup abzuschätzen. Eine Begrenzung von Anfragegröße, Laufzeit und Zahl gleichzeitiger Suchen schützt den Dienst vor Überlastung.

Themenfeld E: Weiterentwicklung der Evaluation

9.3.16 Wiederholung und Erweiterung der Parallelisierungsmessung

Die Messreihe aus Abschnitt 7 ist als erste Einordnung gedacht und sollte in folgenden Punkten erweitert werden: (i) Wiederholung mit den ursprünglich vorgesehenen 50 Anfragen, damit Streuungsmaße und Konfidenzintervalle berechnet werden können; (ii) Durchführung auf Rechnern mit vier, acht, sechzehn und mehr echten Prozessoren, um die Kurve $S(p)$ jenseits von $p = 2$ zu bestimmen und den seriellen Anteil mit mehr als einem Messpunkt anzupassen; (iii) Trennung von Datenbankserver und Anwendung auf zwei Rechner, um zu klären, wie stark das Laden die Messung beeinflusst; (iv) Variation der Queries (Knotenzahlen oberhalb und unterhalb von 15, verschiedene Kantendichten), damit auch Suchen mit wenigen und mit sehr vielen Kandidaten abgedeckt sind; (v) Messung mit und ohne Kandidatenlimit; (vi) Vergleich verschiedener Blockgrößen. Mit diesen Daten ließen sich Amdahl- und Gustafson-Modell tatsächlich an Messwerte anpassen.

9.3.17 Benchmark-Suite und kontinuierliche Performanzmessung

Schließlich wäre die Entwicklung einer reproduzierbaren Benchmark-Suite sinnvoll, die die in Abschnitt 5 dargestellten Diagramme automatisch aus Live-Messungen (statt aus dem hier verwendeten analytischen Modell) generiert, etwa durch Instrumentierung der `search_subgraph`-Funktion mit `time.perf_counter()` und Protokollierung von $(N, n_Q, m_Q, C, T_{\text{gemessen}})$ -Tupeln in einer separaten Metriktabelle. Eine solche Suite ermöglichte die empirische Validierung der in Satz 2 hergeleiteten asymptotischen Schranken und die Kalibrierung des Kandidatenlimits K in Abhängigkeit von einem Antwortzeit-Budget (z. B. 200 ms für interaktive Anwendungen). Die in Abschnitt 6 durchgeführten Experimente liefern hierfür einen konkreten Ausgangspunkt: `bench_compare.py` kann als Kern einer kontinuierlichen Integrationspipeline (CI) dienen, die bei jeder Änderung der Implementierung des Subgraph Algorithmus automatisch prüft, ob die gemessene Laufzeit weiterhin innerhalb der durch Satz 1 vorhergesagten Größenordnung liegt (Regressions-Schranke, z. B. $T_{\text{Sub}}(n) \leq c \cdot n^3$ für eine aus historischen Messungen kalibrierte Konstante c).

9.3.18 Erweiterung des Vergleichsbenchmarks

Die in Abschnitt 6 durchgeführten Experimente beschränken sich auf den Vergleich mit VF2 auf Erdős–Rényi-Zufallsgraphen $G(n, 0,3)$ mit $n \leq 18$. Eine umfassendere Evalua-

tion sollte folgende Erweiterungen umfassen:

- **Weitere Referenzalgorithmen:** Integration von Ullmann [11] (z.B. über `networkx.algorithms.isomorphism.GraphMatcher` mit ISMAGS oder externe Implementierungen von RI [13]), um die in Tabelle 3 rein literaturbasierte Einordnung durch eigene Messungen auf identischer Hardware zu ersetzen.
- **Reale biologische Netzwerke:** Statt $G(n, p)$ -Zufallsgraphen sollten Benchmark-Datensätze wie die KEGG-Pathway-Datenbank (metabolische Netzwerke) oder der DIP/STRING-Datensatz (Protein-Interaktionsnetzwerke) verwendet werden, da reale biologische Netzwerke charakteristische Eigenschaften (Skalenfreiheit, hohe Clusterkoeffizienten, Hub-Knoten) aufweisen, die sowohl das Pruning-Verhalten von VF2/RI als auch die Verteilung der Zeilenkomponenten $\rho(A)$ des Subgraph Algorithmus systematisch beeinflussen können.
- **Größere n :** Die hier durchgeführten Experimente decken $n \leq 18$ ab (entsprechend dem Wertebereich $\{3, \dots, 20\}$ aus der Gen-DB-Generierung, Abschnitt 5). Für $n > 20$ (z. B. größere Stoffwechselnetzwerke mit 50–200 Knoten) wäre zu erwarten, dass die kubische Schranke des Subgraph Algorithmus zunehmend dominiert, während VF2/RI auf solchen Graphen — je nach Dichte und Symmetriegrad — entweder weiterhin sub-exponentiell bleiben (geringe Dichte, viele eindeutige Labels) oder in den exponentiellen Worst Case geraten (hohe Symmetrie, viele Automorphismen). Eine systematische Messreihe für $n \in \{20, 30, 50, 100\}$ wäre daher ein wichtiger nächster Schritt.
- **Statistische Signifikanz:** Die hier verwendeten 8 bzw. 5 Wiederholungen pro Datenpunkt (Tabelle 2, Abbildung 9) genügen für eine erste Tendenzangabe, jedoch nicht für eine belastbare statistische Aussage (z.B. Konfidenzintervalle, t -Test auf Mittelwertunterschiede). Eine erweiterte Messreihe mit ≥ 100 Wiederholungen pro Konfiguration und expliziter Angabe von Konfidenzintervallen (95 %) wäre für eine Publikation in einer Fachzeitschrift erforderlich.
- **Implementierungssprache:** Die hier verwendete Referenzimplementierung des Subgraph Algorithmus ist reines Python mit `numpy`, während `networkx` ebenfalls in Python implementiert ist (mit C-Erweiterungen für einige Kernoperationen). Ein fairer Vergleich auf Implementierungsebene (z.B. beide Verfahren in C++ oder beide in einer JIT-kompilierten Umgebung wie `numba`) würde den Einfluss des Implementierungs-Overheads von der algorithmischen Komplexität trennen und die in Satz 1 bewiesene asymptotische Aussage von konstanten Faktoren isolieren. Für C++ ist dabei die Library-Anbindung (Abschnitt 9.3.23) die geeignete Messgrundlage, weil sie Prozessstart und JSON-Pfad aus dem Vergleich entfernt.

9.3.19 Formale Charakterisierung des Übergangsbereichs θ

Die empirisch beobachtete Umkehrung des Geschwindigkeitsvorteils bei $n_B \approx 14$ (Abschnitt 6) wirft die Frage auf, ob sich der Schwellwert θ aus Abschnitt 9.3.10 formal als Funktion der Graphdichte p charakterisieren lässt. Intuitiv gilt: Je größer p , desto eher findet VF2 *sehr früh* einen Match oder einen Konsistenzbruch (da fast jeder Knoten mit fast jedem anderen verbunden ist, sind die meisten Teilabbildungen ϕ bereits nach

wenigen Schritten konsistent oder inkonsistent), während die Kosten des Subgraph Algorithmus $\Theta(n^3)$ *unabhängig* von p sind (Satz 1 macht keine Annahme über p). Dies legt die folgende Vermutung nahe:

Bemerkung 7 (Vermutung: Dichteabhängigkeit von θ). Für $G(n, p)$ mit $p \rightarrow 1$ (vollständige Graphen) konvergiert die erwartete VF2-Laufzeit gegen $\Theta(n^2)$ (jede Teilabbildung ist sofort konsistent, das Pruning entscheidet nach $O(n)$ Rekursionsschritten mit $O(n)$ Aufwand pro Schritt für Konsistenzprüfung), während $T_{\text{Sub}}(n) = \Theta(n^3)$ unverändert bleibt. Für $p \rightarrow 0$ (sehr dünne Graphen) hingegen terminiert VF2 bei den meisten Knotenpaaren sofort durch Verletzung der Kantenbedingung, was ebenfalls zu $\Theta(n^2)$ oder besser führt. Der Schwellwert $\theta(p)$ wäre demnach für p nahe 0 und nahe 1 *kleiner* als für $p \approx 0,3\text{--}0,5$ (Bereich maximaler struktureller Unsicherheit, in dem VF2 die meisten Rekursionsschritte benötigt) — d. h. $\theta(p)$ wäre als Funktion von p vermutlich *unimodal* mit Maximum bei mittlerer Dichte. Ein formaler Beweis dieser Vermutung würde eine probabilistische Analyse des VF2-Suchbaums auf $G(n, p)$ erfordern, analog zu den in [10] angedeuteten, aber nicht vollständig ausgeführten Average-Case-Betrachtungen.

Eine systematische Messreihe über das zweidimensionale Gitter $(n, p) \in \{4, \dots, 20\} \times \{0,1,0,2, \dots, 0,9\}$ — eine direkte Erweiterung des in Abschnitt 6 durchgeführten Experiments — würde die in Bemerkung 7 formulierte Vermutung empirisch prüfbar machen und gleichzeitig eine Tabelle $\theta(p)$ liefern, mit der die Hybridstrategie aus Abschnitt 9.3.10 dichteadaptiv kalibriert werden könnte (d. h. θ würde nicht als globale Konstante, sondern als Funktion der für den jeweiligen Kandidaten k aus $m_k/(n_k(n_k-1))$ geschätzten Dichte $\hat{p}_k$ nachgeschlagen).

Themenfeld F: Anwendungsebene

9.3.20 Visuelle Exploration und Graph-Layout

Das aktuelle Frontend zeigt Adjazenzmatrizen als Zahlentabellen und Listen. Eine Erweiterung um eine graphische Darstellung (Force-Directed-Layout, z. B. mittels D3.js oder Cytoscape.js) würde die Interpretierbarkeit von Suchergebnissen erheblich verbessern, insbesondere für die in Abschnitt 5 diskutierten **subgraph**-Treffer, bei denen die *Einbettung* des Query-Graphen im größeren Kontextgraphen visuell hervorgehoben werden könnte (z. B. durch Hervorhebung der n_Q Knoten, auf die ϕ aus Abschnitt 9 (Label-Erweiterung) abbildet).

9.3.21 Validierung mit realen Anwendungsfällen

Der größte offene Punkt auf Anwendungsebene ist der Nachweis, dass die Suche für Biologinnen und Biologen nützliche Treffer liefert. Dazu sollten bekannte Motive (etwa Teilpfade der Glykolyse oder des p53-Netzwerks) gegen Datensätze aus öffentlichen Datenbanken gesucht und die Treffer von Fachleuten bewertet werden. Maßgeblich sind dabei die Präzision (wie viele Treffer sind fachlich sinnvoll) und die Vollständigkeit (werden bekannte Vorkommen gefunden). Die Ergebnisse ließen sich mit den Treffern etablierter Verfahren wie VF2 oder RI vergleichen und würden zeigen, an welchen Stellen die schwächere Relation $\preceq$ zu Abweichungen führt. Erst eine solche Validierung beantwortet, ob die zyklische Relation für die Praxis hinreichend ist oder ob die gefärbte Variante und eine Verfeinerung durch VF2 als Nachprüfung der Treffer nötig sind.

Themenfeld G: Kopplung und Schnittstellen

Dieses Themenfeld überträgt die Position aus Abschnitt 8.10 in konkrete Maßnahmen. Die C++-Library-Anbindung und eine erste gepaarte Messreihe sind umgesetzt. Offen bleiben die dauerhafte Absicherung der Backend-Konformität, die Wiederholung auf breiterer Datenbasis sowie die Verbesserung der Phaseninstrumentierung. Die CLI ist damit eine optionale Werkzeug- oder Isolationsschnittstelle, nicht mehr Voraussetzung für den produktiven Suchpfad.

9.3.22 Gemeinsame Schnittstellenspezifikation und Konformitätstests

Gegenwärtig ist die Gleichheit der beiden Backends eine Laufzeitprüfung des Experiments: Das Skript vergleicht die Treffer-IDs und die aktive Implementierung (Abschnitt 8.4). Besser ist es, die Zusicherung an die Schnittstelle zu binden. Die Spezifikation legt fest, welche Ergebniswerte der Vergleich liefert (A ist in B enthalten, B ist in A enthalten, keine Teilgraph-Beziehung, strukturell gleich mit der jeweiligen Wahl des beizubehaltenden Graphen), nach welcher Regel bei identischen Graphen anhand der Kantenzahl entschieden wird, wie Fehler gemeldet werden und welche Eingaben zulässig sind (quadratische 0/1-Matrizen).

Aus der Spezifikation folgt eine Suite von Konformitätstests, die gegen *jede* Implementierung läuft, also gegen den Python-Algorithmus, die Library und gegebenenfalls die CLI. Die Suite enthält feste Beispielpaare mit bekannten Ergebnissen, zufällige Paare mit festem Seed und die Eigenschaftstests aus Themenfeld A, etwa dass jedes Netzwerk Teil von sich selbst ist und dass der Vergleich nicht von der zyklischen Anordnung der Zeilenkomponenten abhängt. Zwei Implementierungen gelten als austauschbar, wenn sie die Suite bestehen. Der Abgleich der Treffer-IDs im Experiment wird dadurch von einer Laufzeitkontrolle zu einer zusätzlichen Absicherung. Zugleich wandert die fachliche Regel für identische Graphen an die Stelle, an der sie hingehört, nämlich in die Schnittstelle des Algorithmus.

9.3.23 Ausbau der Library-Anbindung des C++-Kerns

Die Library-Anbindung ist umgesetzt: Die statische C++-Bibliothek wird mit einem Wrapper für den direkten Aufruf aus Python vorbereitet, und die Suchworker laden sie innerhalb ihres Prozesses. Der gemessene Such-Speedup von $6,36\times$ gegenüber Python bestätigt, dass dieser Integrationsweg für den untersuchten Datensatz praktisch wirksam ist. Die folgenden Maßnahmen betreffen nun nicht mehr die Grundentscheidung, sondern Robustheit, Portabilität und weitere Optimierung:

- **Konformität automatisieren.** Der Experimentlauf vergleicht bereits Treffer-IDs und prüft, welches Backend aktiv ist. Diese Laufzeitkontrollen sollten durch gemeinsame Tests mit festen Fällen, Randbedingungen und Eigenschaftstests ergänzt werden, damit jede Änderung am C++-Wrapper oder Python-Backend vor dem Suchlauf gegen dieselbe Semantik geprüft wird.
- **Datenübergabe und Kopien messen.** Die aktuelle C-Schnittstelle übergibt einzelne Matrizen an den nativen Aufruf. Zu untersuchen ist, welcher Anteil der verbleibenden 66,7% nichtvergleichender beziehungsweise serieller Suchzeit auf Arraykonvertierung und Übergabe entfällt. Erst danach ist zu entscheiden, ob

Batch-Verarbeitung oder ein zusammenhängendes Array für Kandidaten messbare Vorteile bietet.

- **Bau und Auslieferung absichern.** Die Binärdatei ist plattformabhängig; Bau, Laden und Fehlerfälle sollten in der Continuous-Integration-Pipeline für unterstützte Betriebssysteme automatisiert werden. Fehlende oder inkompatible Bibliotheken müssen explizit gemeldet werden; ein stiller Wechsel auf Python würde Ergebnisse und Performance-Messungen verfälschen.
- **Schnittstelle stabil halten.** Ergebniswerte, ungültige Eingaben und Fehlerverhalten sollten unabhängig vom Backend dokumentiert werden. Die Python-Referenz bleibt als Rückfall und Vergleichsimplementierung verfügbar, muss aber dieselben Konformitätstests bestehen.

Offen bleibt der einmalige Aufwand für Bau und Test je Plattform sowie der wiederkehrende Wartungsaufwand bei Änderungen am C++-Kern. Die gemessenen Laufzeiten beantworten die Performance-Frage für die aktuelle Windows-Konfiguration, nicht aber die Portabilität. Automatisierte Builds und direkte Konformitätstests bilden deshalb die Voraussetzung dafür, den beobachteten Beschleunigungsgewinn zuverlässig in künftigen Releases zu erhalten.

9.3.24 Batch- und persistente Prozessschnittstellen als Übergangslösung

Falls eine Library kurzfristig nicht umsetzbar ist, lässt sich die CLI-Anbindung entschärfen, ohne ihre Grenze zu ändern. Zwei Varianten stehen zur Wahl: ein Batch-Aufruf, bei dem ein Prozess viele Vergleiche bearbeitet und die Ergebnisse gesammelt zurückgibt, und ein persistenter Prozess, der Aufträge über `stdin` entgegennimmt und über `stdout` beantwortet. Bei B Vergleichen je Prozessaufruf teilt sich der Prozessstart auf alle Vergleiche auf. Eine Modellrechnung, die auf der gemessenen Mindestlatenz von 10,081 Millisekunden beruht und die Serialisierungskosten δ_{ser} je Paar getrennt führt, ergibt

$$\delta_{\text{eff}}(B) \approx \frac{10,081 \text{ ms}}{B} + \delta_{\text{ser}}.$$

Mit $B = 256$ läge der Prozessanteil rechnerisch bei rund 0,04 Millisekunden je Vergleich. δ_{ser} ist nicht gemessen; der Wert zeigt nur, dass sich der Prozessanteil durch Bündelung beseitigen lässt, während die JSON-Übersetzung bestehen bleibt. Die Übergangslösung löst damit das Kostenproblem der CLI, nicht aber das Beschreibungsproblem: Die fachlichen Regeln und die Fehlerabbildung bleiben im Übersetzungscode. Sie sollte deshalb nur gewählt werden, wenn die Library nach der Messung nicht tragfähig ist, und in diesem Fall ebenfalls gegen die Konformitätstests laufen.

9.3.25 Messprogramm für die Kopplungsfrage

Die abgeschlossene Messung vergleicht Python und Library bereits auf denselben sechs Queries und derselben Worker-Zahl. Eine Folgeuntersuchung sollte diese Messung mit mindestens mehreren Dutzend Queries und Wiederholungen pro Query erweitern, damit Streuungsmaße und Konfidenzintervalle berechnet werden können. CLI- und Batch-CLI-Varianten sind nur dann zusätzlich aufzunehmen, wenn sie als reale Alternativen für Isolation, Interoperabilität oder Fehlersuche bewertet werden; sie sind nicht nötig, um den inzwischen gemessenen Library-Vorteil festzustellen.

Die Folgeuntersuchung sollte die Phasen feiner aufteilen: Datenbankabfrage, Übergabe der Daten an den Rechenkern, Rechenzeit des Kerns und Ergebnisauswertung. Damit lässt sich der verbleibende Aufwand δ genauer bestimmen und prüfen, ob er sich durch eine gebündelte Übergabe senken lässt. Zu untersuchen sind außerdem der Backend-Speedup über unterschiedliche Knotenzahlen und Kantendichten, identische Treffer für einen größeren Query-Satz und die Skalierung mit mehr als zwei Workern. Mindestens zwei Betriebssysteme sollten einbezogen werden, um sowohl die Portabilität als auch den Einfluss des Betriebssystems auf den Wrapper zu bewerten.

9.3.26 Prüfung künftiger Fremdmodule

Mehrere Erweiterungen dieses Ausblicks führen weitere Bausteine in das System ein. Damit sie nicht jeweils neu begründet werden müssen, genügen fünf Prüffragen: Ist die Grenze schmal und benannt? Liegen fachliche Regeln, etwa die Ergebnisabbildung, auf einer Seite der Grenze? Ist der Schnittstellenaufwand δ im Kostenmodell ausgewiesen? Lässt sich der Baustein mit denselben Mitteln testen wie der Rest des Systems? Rechtfertigt der Zweck, etwa Isolation oder Skalierung über Rechner, einen Wechsel der Beschreibung? Für die Maßnahmen des Ausblicks ergibt sich folgende Einordnung:

- **Vektorisierung mit numpy und Kompilierung mit numba** (Abschnitt 9.3.8): bleiben in der Beschreibung des Aufrufers und erfüllen die Fragen ohne Weiteres.
- **C++-Erweiterung oder Library**: erfüllt die Fragen, sofern die fachliche Regel in die Schnittstelle wandert und die Konformitätstests bestehen.
- **GPU-Beschleunigung**: Die Übergabe als Tensor der Form (C, n, n) entspricht der blockweisen Schnittstelle und hält die Grenze schmal. Würde dagegen je Kandidat übertragen, überstiege der Transferaufwand voraussichtlich den Gewinn; die Frage nach δ ist deshalb vor der Umsetzung zu beantworten.
- **Verteilung mit Apache Spark**: führt eine weitere Beschreibung mit eigenem Datenformat ein. Der Zweck, Skalierung über Rechner, kann den Bruch rechtfertigen, sofern die Grenze grob ist, also einmal je Partition und nicht je Vergleich verläuft.
- **VF2 in der Hybridstrategie**: Über `networkx` bleibt der Baustein in der Beschreibung des Backends. Eine externe Implementierung wie RI [13] wäre je nach Anbindung ein Fremdmodul und würde dieselbe Prüfung durchlaufen.
- **Asynchrone Suche und Metriken**: betreffen Schichtgrenzen, die ohnehin über benannte Schnittstellen laufen.

9.4 Priorisierung der nächsten Entwicklungsschritte

Die beschriebenen Erweiterungen sollten in einer Reihenfolge umgesetzt werden, die zunächst die fachliche Korrektheit und Aussagekraft absichert, danach die Messbarkeit herstellt und erst anschließend die Laufzeit optimiert. Die Parallelisierung der Vergleiche, die zuvor als eine von mehreren möglichen Optimierungen aufgeführt war, ist inzwischen umgesetzt und gemessen; sie entfällt daher als eigener Entwicklungsschritt, nicht

Tabelle 12: Vorgeschlagene Reihenfolge der Weiterentwicklung.

Nr.	Maßnahme	Nutzen	Aufwand
1	Differenzielle Tests der Vorauswahl und Eigenschaftstests (Abschnitt 9.3, Feld A)	sichert Korrektheit und alle weiteren Optimierungen ab	gering
2	Gemeinsame Konformitätstests für Python und C++ (Feld G)	sichert identische Semantik bei jeder Änderung	gering bis mittel
3	Feinere Phasen- und Schnittstellenmessung (Feld D und G)	erklärt den Restanteil der C++-Suchzeit	gering bis mittel
4	Wiederholung der Backend- und Parallelisierungsmessung (Feld E und G)	belastbare Aussagen zu Speedup und $S(p)$	mittel
5	Bau und Laden der C++-Library für unterstützte Plattformen automatisieren (Feld G)	reproduzierbare Bereitstellung und Fehlererkennung	mittel
6	Batch- oder persistente CLI-Schnittstelle nur bei Bedarf (Feld G)	erhält Isolation oder Sprachneutralität bei amortisierten Startkosten	gering bis mittel
7	Suche in beide Richtungen (Feld A)	vollständige Suchsemantik	mittel
8	Optimierung des Einzelvergleichs: Abbruch, Vektorisierung (Feld C)	wirkt fast vollständig auf die Gesamtzeit	mittel
9	Verfeinerte Parallelisierung: strömendes Laden, Query pro Task, Blockgröße (Feld C)	kleinerer serieller Anteil	mittel
10	Asynchrone Suche mit Teilergebnissen (Feld D)	bessere Bedienbarkeit bei langen Suchen	mittel
11	Beschriftete Relation, Label-Indizes (Felder A und B)	fachlich sinnvollere und kleinere Trefferlisten	mittel bis hoch
12	Mehrdimensionaler Index, Hash-Index (Feld B)	schärfere Vorauswahl	mittel
13	Hybridstrategie mit VF2 und Fallback (Feld C)	Geschwindigkeit bei garantierter Schranke	hoch
14	Validierung mit realen Netzwerken (Feld F)	Nachweis der Praxistauglichkeit	hoch
15	GPU-Beschleunigung, verteilte Ausführung, jeweils nach den Prüffragen aus Feld G (Feld C)	Skalierung für sehr große Datenbanken	hoch

jedoch ihre Verfeinerung. Die Kopplungsfrage (Themenfeld G) steht früh in der Reihenfolge, weil sie bestimmt, auf welcher Anbindung spätere Messungen und Optimierungen aufbauen.

Die Reihenfolge trennt drei Ziele: Vollständigkeit und semantische Korrektheit (Nummern 1 und 7), reproduzierbare Austauschbarkeit der Backends (Nummern 2 und 5) sowie empirisch belastbare Leistungswerte und darauf aufbauende Optimierungen (Nummern 3, 4, 6 und 8 bis 15). Die Grundentscheidung und erste Messung der Library sind abgeschlossen; Priorität haben nun Konformitätstests, genauere Phasenmessung und Wiederholungen unter kontrollierten Bedingungen. Bei einer Hybridstrategie mit VF2 bleibt die Worst-Case-Garantie nur dann erhalten, wenn der Subgraph Algorithmus als

Fallback vorgesehen ist. Die Tests (Nr. 1 und 2) sollten vor algorithmischen Optimierungen (Nr. 8 und 13) stehen. Die detailliertere Instrumentierung (Nr. 3) ist Voraussetzung, um den Nutzen der späteren Parallelisierungsverfeinerung (Nr. 9) zuverlässig beurteilen zu können.

9.5 Abschließende Bemerkung

Diese Arbeit beschreibt Gen-DB als Full-Stack-System zur Suche in biologischen Netzwerken und verbindet Architektur, formale Analyse, eine Evaluation an einer Million synthetischer Netzwerke sowie Experimente zu VF2, Parallelisierung und Implementierungsbackends. Ihr zentraler Gedanke ist die Trennung zweier Aufgaben: Eine datenbankseitige Vorauswahl verkleinert den Suchraum, und der Subgraph Algorithmus entscheidet anschließend über die verbliebenen Kandidaten mit einer bewiesenen kubischen Zeitschranke. Die Analyse zeigt, unter welchen Bedingungen diese Pipeline korrekt und vollständig ist; die Experimente zeigen, wie stark die tatsächliche Laufzeit von Kandidatenzahl, Implementierung und Ausführungsmodell abhängt.

Besonders deutlich wird das im erneuerten Backend-Experiment. Bei sechs identischen Suchanfragen und 931.899 Kandidaten sinkt die Laufzeit mit der C++-Library von 1.180,28 auf 185,62 Sekunden, ein Speedup von $6,36\times$. Auch der direkte Mikrobenchmark zeigt einen erheblich schnelleren C++-Aufruf. Gleichzeitig ist der Ende-zu-Ende-Gewinn deutlich kleiner als der Mikrobenchmark-Speedup: Nach Beschleunigung des Vergleichskerns machen Datenbankzugriff, Übergabe und Ergebnisaufbau einen großen Teil der verbleibenden Suchzeit aus. Die Messung führt damit nicht nur zu einem positiven Ergebnis für die Library, sondern auch zu einer konkreten nächsten Frage: Welche nichtvergleichenden Phasen lassen sich sicher messen und optimieren, ohne die Korrektheit der Suche zu beeinträchtigen?

Die Ergebnisse bleiben in ihrem Geltungsbereich begrenzt. Die Backend-Messung umfasst sechs einmalig ausgeführte Queries auf synthetischen Daten und einer Windows-11-Konfiguration. Sie belegt weder dieselbe Beschleunigung auf realen biologischen Netzwerken noch auf anderer Hardware; auch stimmen identische Treffer in dieser Stichprobe keinem allgemeinen Korrektheitsbeweis gleich. Offen bleiben außerdem die biologische Angemessenheit der Relation $\preceq$, die Berücksichtigung von Knotenlabels, die Suchrichtung, mögliche Kandidatenlimits und eine breitere statistische Absicherung. Der nächste Entwicklungsschritt ist deshalb nicht allein weitere Beschleunigung, sondern zunächst Konformitätstests, wiederholbare Multi-Plattform-Messungen und feinere Phaseninstrumentierung. Darauf können eine schärfere Vorauswahl, asynchrone Teilergebnisse und reale Anwendungsvalidierung aufbauen.

Der zentrale Gedanke der Trennung gilt somit für die Suchpipeline wie für die Sprachgrenze: Eine klar definierte Schnittstelle macht Verantwortung und Kosten nachvollziehbar. Für den untersuchten Suchpfad hat die Library diese Gestaltungsentscheidung nicht nur strukturell, sondern auch experimentell gestützt; die frühere CLI-Messung zeigt zugleich, wie sehr ein ungünstiger Aufrufpfad den Rechenkern verdecken kann. Die Arbeit liefert damit keinen universellen Leistungsanspruch, sondern eine nachvollziehbare Grundlage für Gen-DB: einen formal analysierten Suchablauf, ein konkret gemessenes schnelleres Backend und einen priorisierten Weg von der Prototyp-Implementierung zu einem validierten und belastbar betriebenen Suchsystem.

Literatur

- [1] Stephan Epp. *Der Subgraph Algorithmus.*, 2026. Verfügbar unter: <https://github.com/naphets86/subgraph>.
- [2] Amir Abboud, Arturs Backurs, Virginia Vassilevska Williams. *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. In: Proceedings of FOCS 2015.
- [3] Paul Erdős, Alfréd Rényi. *On Random Graphs I*. Publicationes Mathematicae (Debrecen), 6:290–297, 1959.
- [4] The PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/15/>.
- [5] Sebastián Ramírez et al. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, 4. Auflage. MIT Press, 2022.
- [7] Richard P. Brent. *The Parallel Evaluation of General Arithmetic Expressions*. Journal of the ACM, 21(2):201–206, 1974.
- [8] Antonin Guttman. *R-Trees: A Dynamic Index Structure for Spatial Searching*. In: Proceedings of ACM SIGMOD, 1984.
- [9] Robert A. Wagner, Michael J. Fischer. *The String-to-String Correction Problem*. Journal of the ACM, 21(1):168–173, 1974.
- [10] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, Mario Vento. *A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10):1367–1372, 2004.
- [11] Julian R. Ullmann. *An Algorithm for Subgraph Isomorphism*. Journal of the ACM, 23(1):31–42, 1976.
- [12] Aric A. Hagberg, Daniel A. Schult, Pieter J. Swart. *Exploring Network Structure, Dynamics, and Function using NetworkX*. In: Proceedings of the 7th Python in Science Conference (SciPy), 2008.
- [13] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis Shasha, Alfredo Ferro. *A Subgraph Isomorphism Algorithm and Its Application to Biochemical Data*. BMC Bioinformatics, 14(Suppl 7):S13, 2013.
- [14] Huahai He, Ambuj K. Singh. *Graphs-at-a-time: Query Language and Access Methods for Graph Databases*. In: Proceedings of ACM SIGMOD, 2008.
- [15] Gene M. Amdahl. *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. In: Proceedings of the AFIPS Spring Joint Computer Conference, 1967.
- [16] John L. Gustafson. *Reevaluating Amdahl’s Law*. Communications of the ACM, 31(5):532–533, 1988.

- [17] Stephen A. Cook. *The Complexity of Theorem-Proving Procedures*. In: Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC), S. 151–158, 1971.
- [18] Richard M. Karp. *Reducibility Among Combinatorial Problems*. In: R. E. Miller, J. W. Thatcher (Hrsg.), Complexity of Computer Computations, Plenum Press, S. 85–103, 1972.
- [19] Michael R. Garey, David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [20] Russell Impagliazzo, Ramamohan Paturi. *On the Complexity of k -SAT*. Journal of Computer and System Sciences, 62(2):367–375, 2001.
- [21] Karl Bringmann, Marvin Künnemann. *Quadratic Conditional Lower Bounds for String Problems and Dynamic Time Warping*. In: Proceedings of FOCS 2015, S. 79–97.
- [22] Daniel S. Hirschberg. *A Linear Space Algorithm for Computing Maximal Common Subsequences*. Communications of the ACM, 18(6):341–343, 1975.
- [23] Brendan D. McKay, Adolfo Piperno. *Practical Graph Isomorphism, II*. Journal of Symbolic Computation, 60:94–112, 2014.
- [24] László Babai. *Graph Isomorphism in Quasipolynomial Time*. In: Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC), S. 684–697, 2016.
- [25] Vincenzo Carletti, Pasquale Foggia, Alessia Saggese, Mario Vento. *Challenging the Time Complexity of Exact Subgraph Isomorphism for Huge and Dense Graphs with VF3*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 40(4):804–818, 2018.
- [26] Jinsoo Lee, Wook-Shin Han, Romans Kasperovics, Jeong-Hoon Lee. *An In-depth Comparison of Subgraph Isomorphism Algorithms in Graph Databases*. Proceedings of the VLDB Endowment, 6(2):133–144, 2013.
- [27] Xifeng Yan, Philip S. Yu, Jiawei Han. *Graph Indexing: A Frequent Structure-based Approach*. In: Proceedings of ACM SIGMOD, S. 335–346, 2004.
- [28] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, Uri Alon. *Network Motifs: Simple Building Blocks of Complex Networks*. Science, 298(5594):824–827, 2002.
- [29] Albert-László Barabási, Réka Albert. *Emergence of Scaling in Random Networks*. Science, 286(5439):509–512, 1999.
- [30] Minoru Kanehisa, Susumu Goto. *KEGG: Kyoto Encyclopedia of Genes and Genomes*. Nucleic Acids Research, 28(1):27–30, 2000.
- [31] Damian Szklarczyk et al. *STRING v11: Protein–Protein Association Networks with Increased Coverage, Supporting Functional Discovery in Genome-Wide Experimental Datasets*. Nucleic Acids Research, 47(D1):D607–D613, 2019.

- [32] Charles R. Harris et al. *Array Programming with NumPy*. Nature, 585:357–362, 2020.
- [33] Siu Kwan Lam, Antoine Pitrou, Stanley Seibert. *Numba: A LLVM-based Python JIT Compiler*. In: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 2015.